

Table of Contents

机器学习（常用科学计算库的使用）基础定位、目标	1.1
机器学习概述	1.2
人工智能概述	1.2.1
人工智能发展历程	1.2.2
人工智能主要分支	1.2.3
机器学习工作流程	1.2.4
机器学习算法分类	1.2.5
模型评估	1.2.6
Azure机器学习模型搭建实验	1.2.7
深度学习简介	1.2.8
机器学习基础环境安装与使用	1.3
库的安装	1.3.1
jupyter notebook使用	1.3.2
Matplotlib	1.4
Matplotlib之HelloWorld	1.4.1
基础绘图功能 — 以折线图为例	1.4.2
常见图形绘制	1.4.3
Numpy	1.5
Numpy的优势	1.5.1
N维数组-ndarray	1.5.2
基本操作	1.5.3
ndarray运算	1.5.4
数组间的运算	1.5.5
数学：矩阵	1.5.6
Pandas	1.6
Pandas介绍	1.6.1
Pandas数据结构	1.6.2
基本数据操作	1.6.3
DataFrame运算	1.6.4
Pandas画图	1.6.5
文件读取与存储	1.6.6
高级处理-缺失值处理	1.6.7
高级处理-数据离散化	1.6.8
高级处理-合并	1.6.9
高级处理-交叉表与透视表	1.6.10
高级处理-分组与聚合	1.6.11
案例	1.6.12
拓展知识	1.7
完整机器学习项目的流程	1.7.1
独立同分布	1.7.2

机器学习（常用科学计算库的使用）基础定位、目标

定位

作为人工智能领域(数据挖掘/机器学习方向)的基础课程，为长期从事人工智能方向打下坚实的基础。

目标

- 了解机器学习定义以及应用场景
- 掌握机器学习基础环境的安装和使用
- 掌握利用常用的科学计算库对数据进行展示、分析

机器学习概述

学习目标

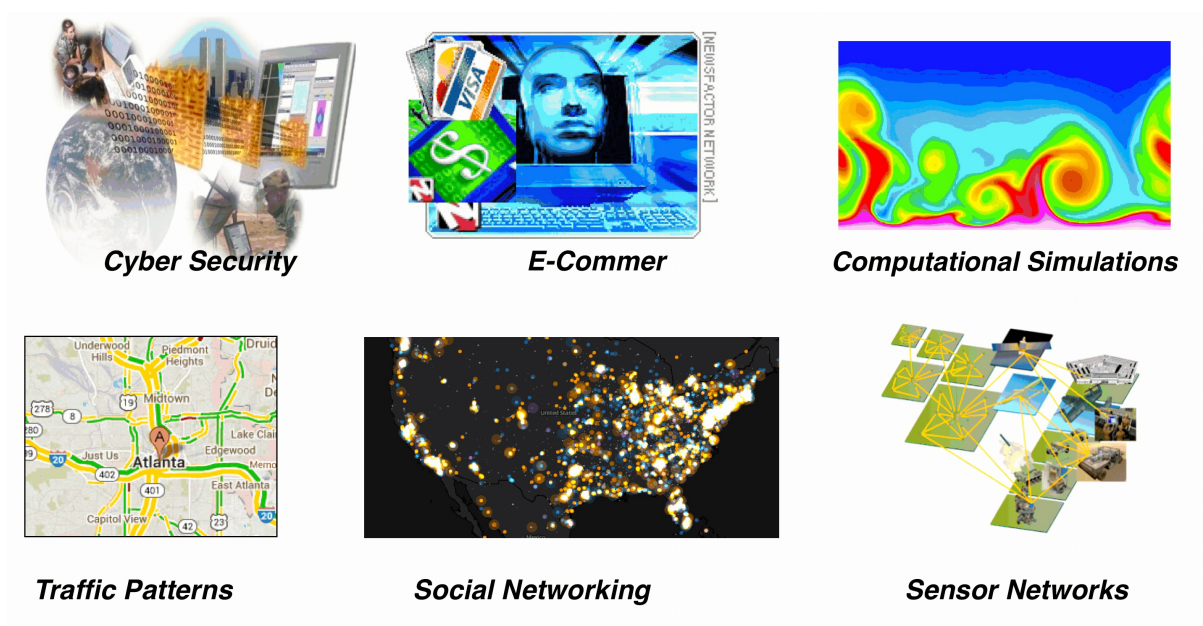
- 了解人工智能发展历程
- 了解机器学习定义以及应用场景
- 知道机器学习算法监督学习与无监督学习的区别
- 知道监督学习中的分类、回归特点
- 知道机器学习的开发流程

1.1 人工智能概述

学习目标

- 了解人工智能在现实生活中的应用
- 知道人工智能发展必备三要素
- 知道人工智能和机器学习、深度学习三者之间的关系

1 人工智能应用场景



2 人工智能小案例

案例一：

请你画的是：棕榈树

这是你画的图，而且神经网络识别出来了。



为什么神经网络知道棕榈树长什么样子？因为它会观察其他用户画的这些涂鸦示例，并从中进行学习。

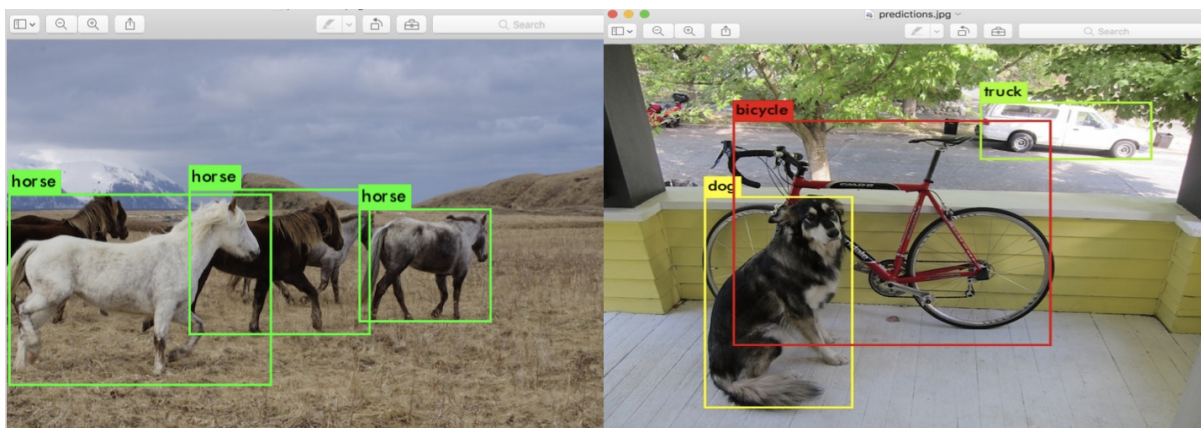
	
	
	
	

神经网络觉得你画的图也像下面这些东西：

猫中棕榈树 	第2接近的匹配项 雨伞 	第3接近的匹配项 火烈鸟 
--	---	--

参考链接：<https://quickdraw.withgoogle.com>

案例二：



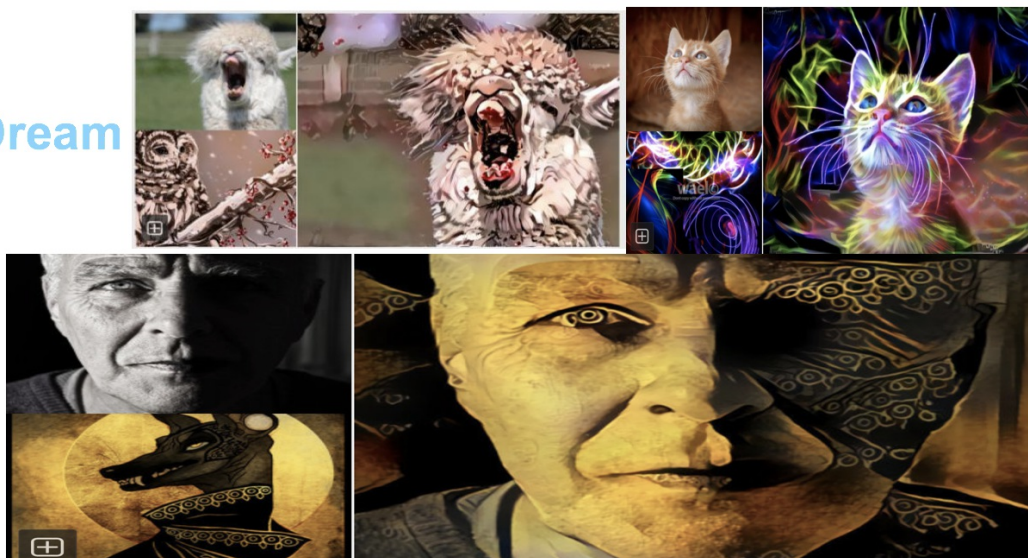
参考链接: <https://pjreddie.com/darknet/yolo/>

案例三:

风格迁移



DeepDream



查看更多: <https://deepdreamgenerator.com/>

3 人工智能发展必备三要素:

- 数据
- 算法
- 计算力
 - CPU, GPU, TPU

AI新动能 Driving forces behind AI



- 算力之CPU、GPU对比：
 - CPU主要适合I/O密集型的任务
 - GPU主要适合计算密集型任务
- 提问：什么类型的程序适合在GPU上运行？

(1) 计算密集型的程序。

所谓计算密集型(Compute-intensive)的程序，就是其大部分运行时间花在了寄存器运算上，寄存器的速度和处理器的速度相当，从寄存器读写数据几乎没有延时。可以做一下对比，读内存的延迟大概是几百个时钟周期；读硬盘的速度就不说了，即便是SSD，也实在是太慢了。

(2) 易于并行的程序。

GPU其实是一种SIMD(Single Instruction Multiple Data)架构，他有成百上千个核，每一个核在同一时间最好能做同样的事情。

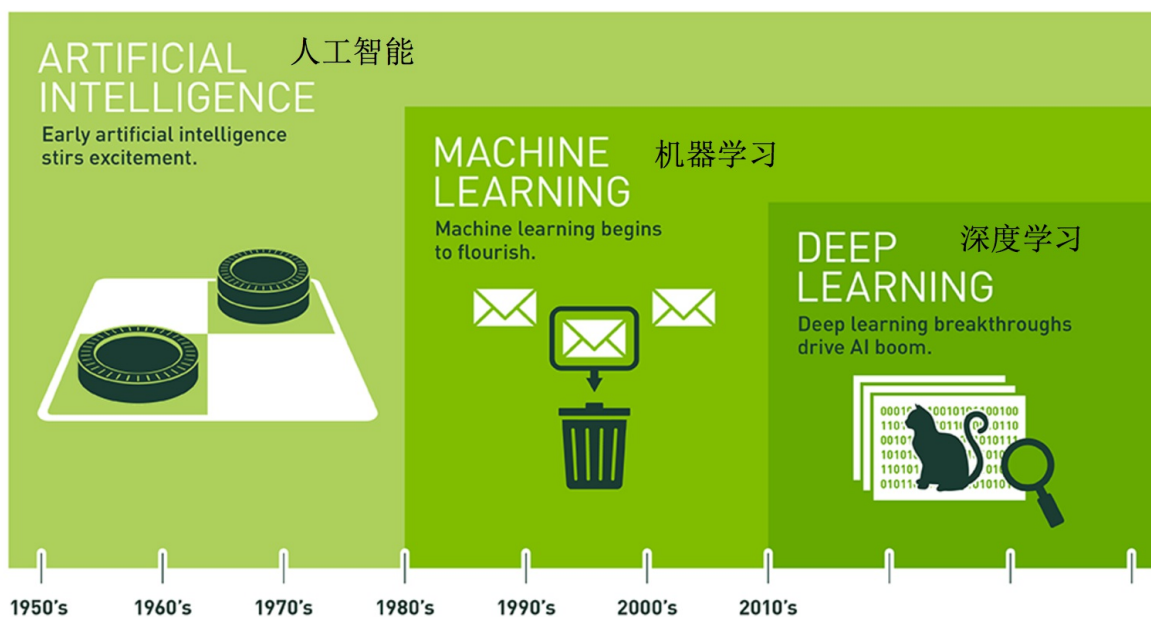
CPU和GPU的区别：

http://www.sohu.com/a/201309334_468740

Google TPU 介绍：

<https://buzzorange.com/techorange/2017/09/27/what-intel-google-nvidia-microsoft-do-for-ai-chips/>

4 人工智能、机器学习和深度学习



- 人工智能和机器学习，深度学习的关系
 - 机器学习是人工智能的一个实现途径
 - 深度学习是机器学习的一个方法发展而来

4 小结

- 人工智能应用场景【了解】
 - 网络安全、电子商务、计算模拟、社交网络
- 人工智能必备三要素【知道】
 - 数据、算法、计算力
- 人工智能和机器学习，深度学习的关系【知道】
 - 机器学习是人工智能的一个实现途径
 - 深度学习是机器学习的一个方法发展而来

1.2 人工智能发展历程

学习目标

- 了解人工智能阶段发展历程

1 人工智能的起源

1.1 图灵测试

测试者与测试者（一个人和一台机器）隔开的情况下，通过一些装置（如键盘）向被测试者随意提问。

多次测试（一般为5min之内），如果有超过30%的测试者不能确定被测试者是人还是机器，那么这台机器就通过了测试，并被认为具有人类智能。



1.2 达特茅斯会议

1956年8月，在美国汉诺斯小镇宁静的达特茅斯学院中，

约翰·麦卡锡（John McCarthy）

马文·闵斯基（Marvin Minsky，人工智能与认知学专家）

克劳德·香农（Claude Shannon，信息论的创始人）

艾伦·纽厄尔（Allen Newell，计算机科学家）

赫伯特·西蒙（Herbert Simon，诺贝尔经济学奖得主）等科学家正聚在一起，讨论着一个完全不食人间烟火的主题：

用机器来模仿人类学习以及其他方面的智能。

会议足足开了两个月的时间，虽然大家没有达成普遍的共识，但是却为会议讨论的内容起了一个名字：

人工智能

因此，1956年也就成为了人工智能元年。

2 发展历程

人工智能充满未知的探索道路曲折起伏。如何描述人工智能自1956年以来60余年的发展历程，学术界可谓仁者见仁、智者见智。我们将人工智能的发展历程划分为以下6个阶段：

- 第一是起步发展期：1956年—20世纪60年代初。

人工智能概念提出后，相继取得了一批令人瞩目的研究成果，如机器定理证明、跳棋程序等，掀起人工智能发展的第一个高潮。

- 第二是反思发展期：20世纪60年代—70年代初。

人工智能发展初期的突破性进展大大提升了人们对人工智能的期望，人们开始尝试更具挑战性的任务，并提出了一些不切实际的研发目标。然而，接二连三的失败和预期目标的落空（例如，无法用机器证明两个连续函数之和还是连续函数、机器翻译闹出笑话等），使人工智能的发展走入低谷。

- 第三是应用发展期：20世纪70年代初—80年代中。

20世纪70年代出现的专家系统模拟人类专家的知识和经验解决特定领域的问题，实现了人工智能从理论研究走向实际应用、从一般推理策略探讨转向运用专门知识的重大突破。专家系统在医疗、化学、地质等领域取得成功，推动人工智能走入应用发展的新高潮。

- 第四是低迷发展期：20世纪80年代中—90年代中。

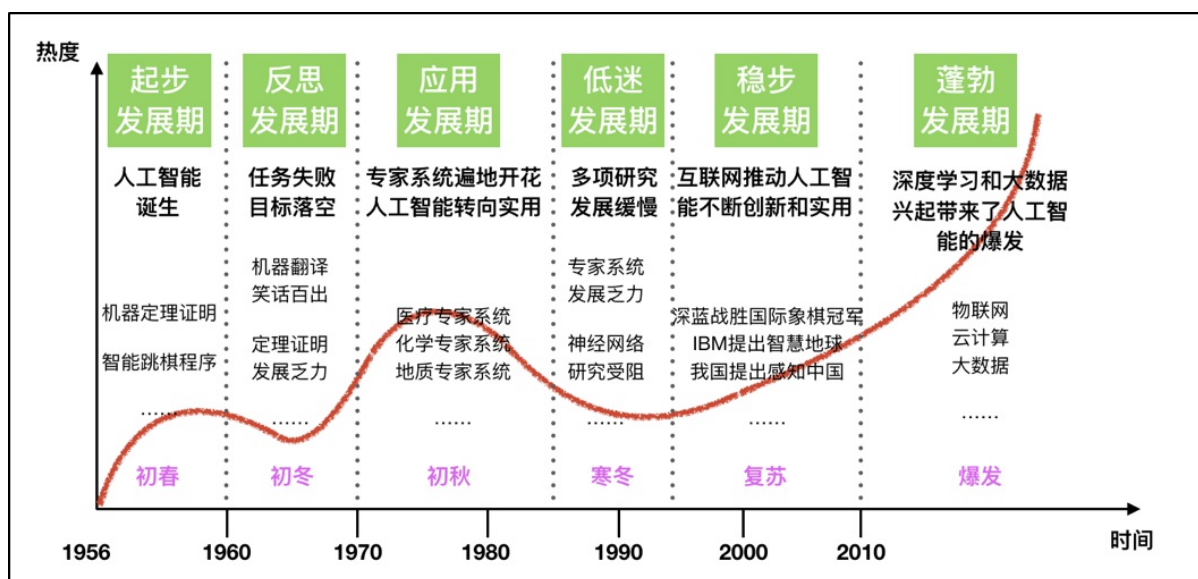
随着人工智能的应用规模不断扩大，专家系统存在的应用领域狭窄、缺乏常识性知识、知识获取困难、推理方法单一、缺乏分布式功能、难以与现有数据库兼容等问题逐渐暴露出来。

- 第五是稳步发展期：20世纪90年代中—2010年。

由于网络技术特别是互联网技术的发展，加速了人工智能的创新研究，促使人工智能技术进一步走向实用化。1997年国际商业机器公司（简称IBM）深蓝超级计算机战胜了国际象棋世界冠军卡斯帕罗夫，2008年IBM提出“智慧地球”的概念。以上都是这一时期的标志性事件。

- 第六是蓬勃发展期：2011年至今。

随着大数据、云计算、互联网、物联网等信息技术的发展，泛在感知数据和图形处理器等计算平台推动以深度学习为代表的人工智能技术飞速发展，大幅跨越了科学与应用之间的“技术鸿沟”，诸如图像分类、语音识别、知识问答、人机对弈、无人驾驶等人工智能技术实现了从“不能用、不好用”到“可以用”的技术突破，迎来爆发式增长的新高潮。



3 小结

- 人工智能的起源【了解】
 - 图灵测试
 - 达特茅斯会议
- 人工智能的发展经历了六个阶段【了解】
 - 起步发展期
 - 反思发展期
 - 应用发展期
 - 低迷发展期
 - 稳步发展期
 - 蓬勃发展期

1.3 人工智能主要分支

学习目标

- 了解人工智能的主要分支

1 主要分支介绍

通讯、感知与行动是现代人工智能的三个关键能力，在这里我们将根据这些能力/应用对这三个技术领域进行介绍：

- 计算机视觉(CV)、
- 自然语言处理(NLP)
 - 在 NLP 领域中，将覆盖文本挖掘/分类、机器翻译和语音识别。
- 机器人

1.1 分支一：计算机视觉

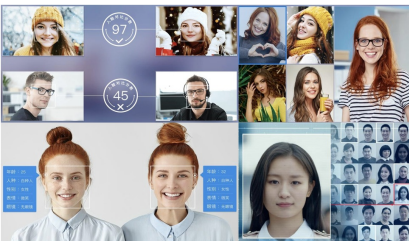
计算机视觉(CV)是指机器感知环境的能力。这一技术类别中的经典任务有图像形成、图像处理、图像提取和图像的三维推理。物体检测和人脸识别是其比较成功的研究领域。

当前阶段：

计算机视觉现已有很多应用，这表明了这类技术的成就，也让我们将其归入到应用阶段。随着深度学习的发展，机器甚至能在特定的案例中实现超越人类的表现。但是，这项技术离社会影响阶段还有一定距离，那要等到机器能在所有场景中都达到人类的同等水平才行(感知其环境的所有相关方面)。

发展历史：

阶段	年份	说明
研究	1959	Hubel 和 Wiesel 对猫进行了实验（为了研究视觉的工作方式）
	1963	计算机视觉领域的先驱 Larry Roberts 在他的博士论文中试图提取「积木世界（Block World）」的 3D 几何信息
	1966	Summer Vision 项目启动，人们普遍认为这意味着计算机视觉的诞生
工程	1970s (1977)	David Marr 的《视觉（Vision）》一书影响和激励了这一领域的一代研究者，该书暗示了以「层」的方式看待图像的思想
应用	1980s	光学字符识别（OCR）技术开始在工业应用中使用
	2001	Viola 和 Jones 开始了面部检测研究；计算机视觉的研究重心发生转移，从建模物体的 3D 形状转向了识别物体是什么
	2009	ImageNet 建立
	2012	AlexNet 在 ImageNet 竞赛中获胜
	2015	Google Cloud Vision API [18] 开放



1.2 分支二：语音识别

语音识别是指识别语音(说出的语言)并将其转换成对应文本的技术。相反的任务(文本转语音/TTS)也是这一领域内一个类似的研究主题。

当前阶段:

语音识别已经处于应用阶段很长时间了。最近几年,随着大数据和深度学习技术的发展,语音识别进展颇丰,现在已经非常接近社会影响阶段了。

语音识别领域仍然面临着声纹识别和「鸡尾酒会效应」等一些特殊情况的难题。

现代语音识别系统严重依赖于云,在离线时可能就无法取得理想的工作效果。

发展历史:

阶段	年份	说明
研究	1930s	贝尔实验室提出语音分析合成系统模型 (Homer Dudley)
	1952	贝尔实验室的团队开发了用于识别说出的数字的系统 Audrey (词表有大约 10 个词)
	1960	Gunnar Fant 开发出用于产生语音的声源-过滤器模型 (Source-filter model)
工程	1962	IBM 在 1962 年的西雅图世博会上展示了 Shoebox (词表有 16 个英语词)
	Late 1960s	苏联研究者发明了动态时间规整 (Dynamic Time Warping/DTW) (词表有 200 个词)
	1971-1976	DARPA 资助了五年的语音识别研究,目标是将系统的词表扩展到 1000 词。CMU 的 Harpy 实现了这一目标,将词表扩展到了 1011 词
	Early 1980s	隐马尔可夫模型 (HMM) 开始用于语音识别
	Mid 1980s	IBM 的 Tangora 将口语词表规模扩展到了 20000 词
	1987	Katz 开发了 back-off 模型 (一种生成式 n-gram 语言模型)
应用	1990	第一款语音识别消费级产品 Dragon Dictate 发布
	1993	黄学东开发出 Sphinx-II (第一个大词表连续语音识别系统)
	1996	BellSouth 开发了第一个语音门户 VAL,这是一种呼叫交互式语音识别系统
	1997	Juergen 开发出长短期记忆 (LSTM) 网络,后被广泛运用于语音识别领域
	2002	微软开始将语音识别集成到他们的产品中
	2007	谷歌推出 GOOG-411 服务
	2008	谷歌为 iPhone 推出语音搜索 (Voice Search) 应用
	2009	Geoffrey Hinton、邓力、俞栋 etc. (多伦多大学、微软、谷歌、IBM) 提出用于声学建模的深度前馈神经网络,显著提升了语言识别的准确度
	2010	谷歌在安卓手机上为 Voice Search 引入「个性化识别」功能,让其可以记录不同用户的声音
	2011	苹果推出 Siri
	2012	谷歌推出 Google Now
	2014	微软发布 Cortana
	2014	亚马逊发布 Echo (使用了 Alexa)
	2016	谷歌发布 Google Assistant

- 百度语音识别:
 - 距离小于1米,中文字准确率97%+

- 支持耳语、长语音、中英文混合及方言



1.3 分支三：文本挖掘/分类

这里的文本挖掘主要是指文本分类，该技术可用于理解、组织和分类结构化或非结构化文本文档。其涵盖的主要任务有句法分析、情绪分析和垃圾信息检测。

当前阶段：

我们将这项技术归类到应用阶段，因为现在有很多应用都已经集成了基于文本挖掘的情绪分析或垃圾信息检测技术。文本挖掘技术也在智能投顾的开发中有所应用，并且提升了用户体验。

文本挖掘和分类领域的一个瓶颈出现在歧义和有偏差的数据上。

发展历史：

年份	说明
1954	BOW 模型的早期引用出现
1955	语言模型出现
mid-1980s	隐马尔可夫模型用于 POS 消歧
1990s	文本分析开始被称为「文本数据挖掘」或「文本挖掘」
1998	出现隐语义索引 (Latent Semantic Indexing) 模型
1999	Thomas Hofmann 创造概率隐语义分析 (pLSA) 模型
2002	Andrew Ng et al. 开发出隐狄利克雷分布 (LDA) 模型
2016	Facebook 研究团队开发出一个名叫 fastText 的库，可用于快速文本表征和分类

1.4 分支四：机器翻译

机器翻译(MT)是利用机器的力量自动将一种自然语言(源语言)的文本翻译成另一种语言(目标语言)。

当前阶段：

机器翻译是一个见证了大量发展历程的应用领域。该领域最近由于神经机器翻译而取得了非常显著的进展，但仍然没有全面达到专业译者的水平；但是，我们相信在大数据、云计算和深度学习技术的帮助下，机器翻译很快将进入社会影响阶段。

在某些情况下，俚语和行话等内容的翻译会比较困难(受限词表问题)。

专业领域的机器翻译(比如医疗领域)表现通常不好。

发展历史：

阶段	年份	说明
研究	1949	Warren Weaver 提出了机器翻译的思想
	1954	IBM-701 系统表现出了机器翻译的可能性
	Late 1950s, early 1960s	Margaret Masterman et al. 设计了用于机器翻译的语义网络
工程	1976	蒙特利尔大学开发出 TAUM-MÉTÉO 系统，在翻译上取得了好的成绩，成为了这一领域的里程碑
	1993	Brown 和 Della Pietra 开始将统计方法应用于机器翻译
	2003	Koehn 提出基于短语的翻译模型；Yoshua Bengio 的团队开发了一个基于神经网络的语言模型 [86]
	2005	David Chang 提出基于层次短语的翻译
应用	2006	谷歌推出谷歌翻译
	2009	IBM 开发出 ViaVoice Translator
	2014	不同团队的研究者都开发出了神经机器翻译（NMT）；注意力机制也开始被引入神经机器翻译

1.5 分支五：机器人

机器人学(Robotics)研究的是机器人的设计、制造、运作和应用，以及控制它们的计算机系统、传感反馈和信息处理。

机器人可以分成两大类:固定机器人和移动机器人。固定机器人通常被用于工业生产(比如用于装配线)。常见的移动机器人应用有货运机器人、空中机器人和自动载具。机器人需要不同部件和系统的协作才能实现最优的作业。其中在硬件上包含传感器、反应器和控制器；另外还有能够实现感知能力的软件，比如定位、地图测绘和目标识别。

当前阶段：

自上世纪「Robot」一词诞生以来，人们已经为工业制造业设计了很多机器人。工业机器人是增长最快的应用领域，它们在 20 世纪 80 年代将这一领域带入了应用阶段。在安川电机、Fanuc、ABB、库卡等公司的努力下，我们认为进入 21 世纪之后，机器人领域就已经进入了社会影响阶段，此时各种工业机器人已经主宰了装配生产线。此外，软件机器人在很多领域也有广泛的应用，比如在医疗行业协助手术或在金融行业自动执行承销过程。

但是，法律法规和「机器人威胁论」可能会妨碍机器人领域的发展。还有设计和制造机器人需要相对较高的投资。

发展历史：

阶段	年份	说明
研究	1920/1921	艺术家 Josef Čapek 创造了「Robot」这个词，并通过其兄弟 Karel Čapek 的一部戏剧将其介绍给公众
	1941	Isaac Asimov 提出「机器人三定律」
工程	1948	William Grey Walter 创造的 Turtle 被认为是第一个自动的移动机器人
	1954	George Devol 和 Joe Engleberger 发明了第一个可编程的机器人——实际上是机器臂；通用（GM）在 1961 年买下了这个机器人
	1960s	Johns Hopkins Beast 移动式「前机器人（pre-robot）」问世
	late 1960s	SRI 开发出 Shakey（第一个通用移动式机器人）
应用	1961	通用（GM）首次在装配线上安装了工业机器人 Unimate
	1970s	早期自动驾驶汽车 Stanford Cart（Moravec）[19] 问世
	1990	世界上最成功的商业机器人吸尘器公司 iRobot 成立
	1995	CMU 改造了一辆旁蒂克 Trans Sport 小型厢车，得到了自动汽车 NavLab 5，它完成了 3100 英里的行程
	1997	火星探路者号（NASA 的一次火星任务，包含一个固定着陆器和一个移动火星车）成功在火星着陆
社会影响	2004	DARPA Ground Challenge 自动驾驶汽车挑战赛开始
	2005	STANLEY 赢得 DARPA 挑战赛，完成了 132 英里行程
	2007	库卡推出 KR titan（「世界上最大和最强的六轴工业机器人」），是吉尼斯世界纪录中世界最强壮的机器人 [37]
	2009	谷歌启动自动驾驶汽车项目 [39]
	2016	ABB 开发出 YuMi「世界上第一款真正的协作式工业机器人」[36]
	2017	机器人「Sophia」成为沙特阿拉伯公民，这是第一个获取公民身份的机器人

总的来说，人工智能领域的研究前沿正逐渐从搜索、知识和推理领域转向机器学习、深度学习、计算机视觉和机器人领域。

大多数早期技术至少已经处于应用阶段了，而且其中一些已经显现出了社会影响力。一些新开发的技术可能仍处于工程甚至研究阶段，但是我们可以看到不同阶段之间转移的速度变得越来越快。

2 小结

- 人工智能主要分支【了解】
 - 计算机视觉
 - 语音识别
 - 文本挖掘/分类
 - 机器翻译
 - 机器人

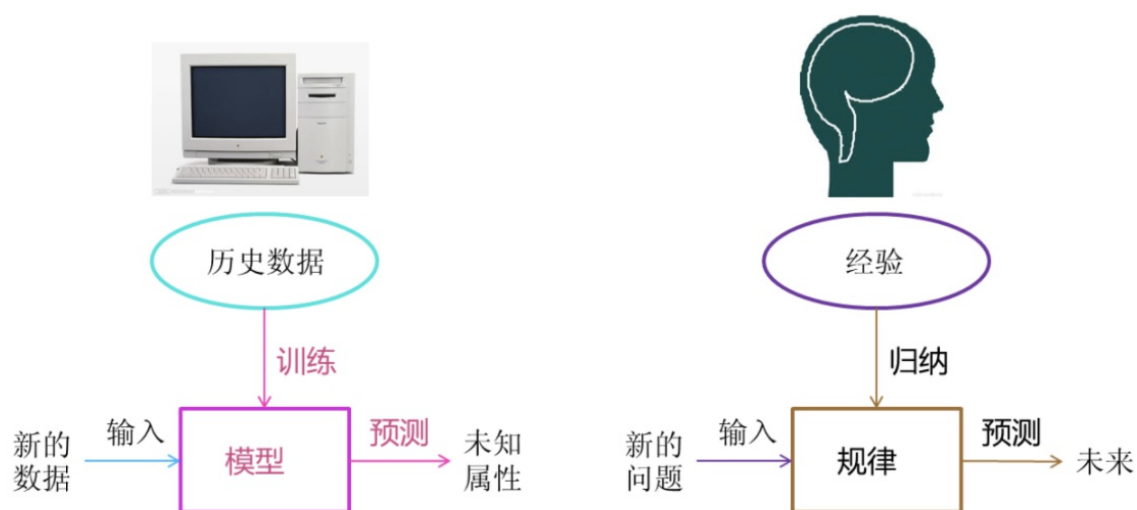
1.4 机器学习工作流程

学习目标

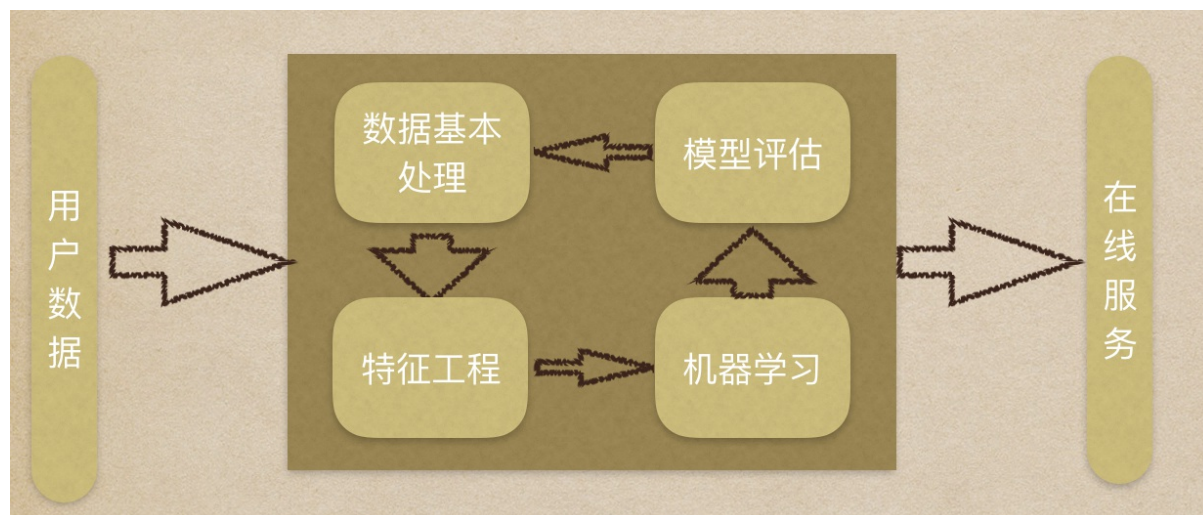
- 了解机器学习的定义
- 知道机器学习的工作流程
- 掌握获取到的数据集的特性

1 什么是机器学习

机器学习是从数据中自动分析获得模型，并利用模型对未知数据进行预测。



2 机器学习工作流程



- 机器学习工作流程总结
 - 1. 获取数据
 - 2. 数据基本处理
 - 3. 特征工程
 - 4. 机器学习(模型训练)
 - 5. 模型评估

- 结果达到要求，上线服务
- 没有达到要求，重新上面步骤

2.1 获取到的数据集介绍



房屋价格？

序号	电影名称	搞笑镜头	拥抱镜头	打斗镜头	电影类型
1	功夫熊猫	39	0	31	喜剧片
2	叶问3	3	2	65	动作片
3	二次曝光	2	3	55	爱情片
4	代理情人	9	38	2	爱情片
5	新步步惊心	8	34	17	爱情片
6	谍影重重	5	2	57	动作片
7	美人鱼	21	17	5	喜剧片
8	宝贝当家	45	2	9	喜剧片
9	唐人街探案	23	3	17	?



• 数据简介

在数据集中一般：

- 一行数据我们称为一个样本
- 一列数据我们称为一个特征
- 有些数据有目标值（标签值），有些数据没有目标值（如上表中，电影类型就是这个数据集的目标值）

• 数据类型构成：

- 数据类型一：特征值+目标值（目标值是连续的和离散的）
- 数据类型二：只有特征值，没有目标值

• 数据分割：

- 机器学习一般的数据集会划分为两个部分：
 - 训练数据：用于训练，构建模型
 - 测试数据：在模型检验时使用，用于评估模型是否有效
- 划分比例：
 - 训练集：70% 80% 75%
 - 测试集：30% 20% 25%

2.2 数据基本处理

即对数据进行缺失值、去除异常值等处理

2.3 特征工程

2.3.1 什么是特征工程

特征工程是使用专业背景知识和技巧处理数据，使得特征能在机器学习算法上发挥更好的作用的过程。

- 意义：会直接影响机器学习的效果

2.3.2 为什么需要特征工程(Feature Engineering)

机器学习领域的大神Andrew Ng(吴恩达)老师说“Coming up with features is difficult, time-consuming, requires expert knowledge. “Applied machine learning” is basically feature engineering.”

注：业界广泛流传：数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限而已。

2.3.3 特征工程包含内容

- 特征提取
- 特征预处理
- 特征降维

2.3.4 各概念具体解释

- 特征提取
 - 将任意数据（如文本或图像）转换为可用于机器学习的数字特征

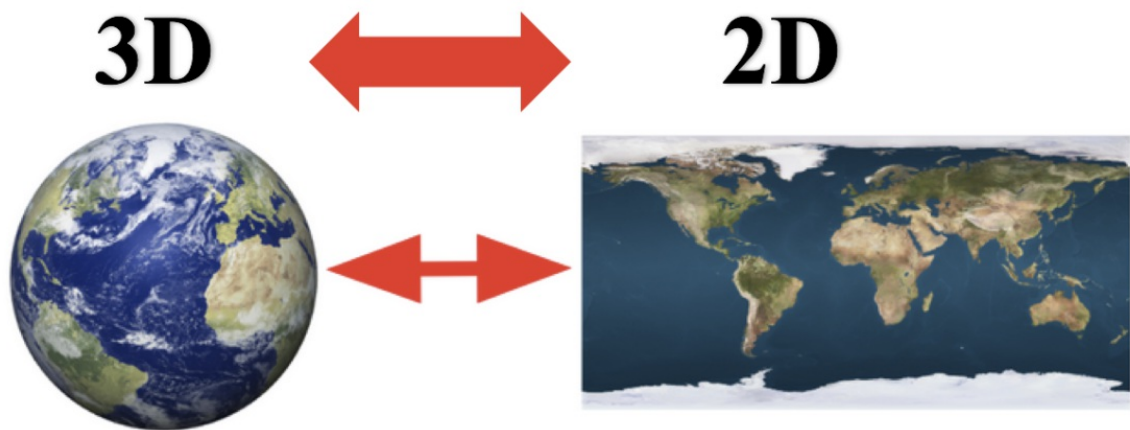
My father was a self-taught mandolin player. He was one of the best string instrument players in our town. He could not read music, but if he heard a tune a few times, he could play it. When he was younger, he was a member of a small country music band. They would play at local dances and on a few occasions would play for the local radio station. He often told us how he had auditioned and earned a position in a band that featured Patsy Cline as their lead singer. He told the family that after he was hired he never went back. Dad was a very religious man. He stated that there was a lot of drinking and cursing the day of his audition and he did not want to be around that type of environment.

```
array([[ 0.04950738,  0.19802951,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.09901475,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.09901475,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.09901475,  0.04950738,  0.04950738,  0.6435959 ,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.09901475,  0.04950738,  0.04950738,  0.04950738,  0.09901475,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.09901475,
         0.04950738,  0.04950738,  0.09901475,  0.04950738,  0.24753689,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.14852213,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.04950738,  0.19802951,
         0.19802951,  0.04950738,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.09901475,  0.04950738,  0.04950738,  0.04950738,
         0.04950738,  0.04950738,  0.04950738,  0.34655164,  0.04950738,
         0.04950738,  0.09901475,  0.04950738]])
```

- 特征预处理
 - 通过一些转换函数将特征数据转换成更加适合算法模型的特征数据过程

特征1	特征2	特征3	特征4		特征1	特征2	特征3	特征4
90	2	10	40	→	1.	0.	0.	0.
60	4	15	45		0.	1.	1.	0.83
75	3	13	46		0.5	0.5	0.6	1.

- 特征降维
 - 指在某些限定条件下，降低随机变量(特征)个数，得到一组“不相关”主变量的过程



2.4 机器学习

选择合适的算法对模型进行训练（具体内容见1.5）

2.5 模型评估

对训练好的模型进行评估（具体内容见1.6）

拓展阅读：

[完整机器学习项目的流程](#)

3 小结

- 机器学习定义【掌握】
 - 机器学习是从数据中自动分析获得模型，并利用模型对未知数据进行预测
- 机器学习工作流程总结【掌握】
 - 1.获取数据
 - 2.数据基本处理
 - 3.特征工程
 - 4.机器学习(模型训练)
 - 5.模型评估
 - 结果达到要求，上线服务
 - 没有达到要求，重新上面步骤
- 获取到的数据集介绍【掌握】
 - 数据集中一行数据一般称为一个样本，一列数据一般称为一个特征。
 - 数据集的构成：
 - 由特征值+目标值（部分数据集没有）构成
 - 为了模型的训练和测试，把数据集分为：
 - 训练数据（70%-80%）和测试数据（20%-30%）
- 特征工程包含内容【了解】
 - 特征提取
 - 特征预处理
 - 特征降维

1.5 机器学习算法分类

学习目标

- 了解机器学习常用算法的分类

根据数据集组成不同，可以把机器学习算法分为：

- 监督学习
- 无监督学习
- 半监督学习
- 强化学习

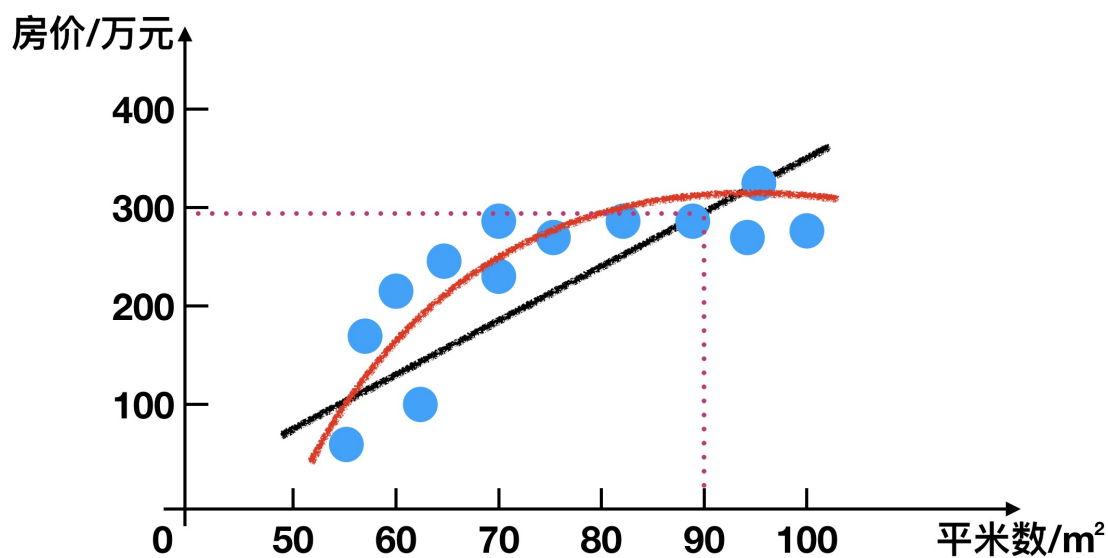
1 监督学习

- 定义：
 - 输入数据是由输入特征值和目标值所组成。
 - 函数的输出可以是一个连续的值(称为回归)，
 - 或是输出是有限个离散值（称作分类）。

1.1 回归问题

例如：预测房价，根据样本集拟合出一条连续曲线。

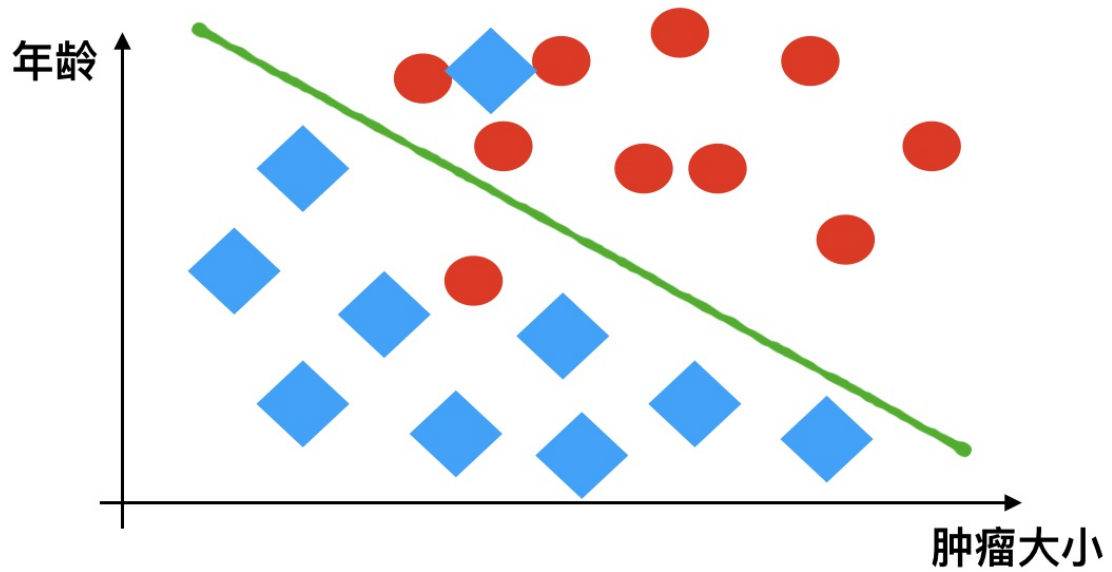
北京市六环外某小区房价预测模型



1.2 分类问题

例如：根据肿瘤特征判断良性还是恶性，得到的是结果是“良性”或者“恶性”，是离散的。

肿瘤良性\恶性预测



2 无监督学习

- 定义：
 - 输入数据是由输入特征值组成，没有目标值
 - 输入数据没有被标记，也没有确定的结果。样本数据类别未知；
 - 需要根据样本间的相似性对样本集进行类别划分。

举例：



- 有监督，无监督算法对比：

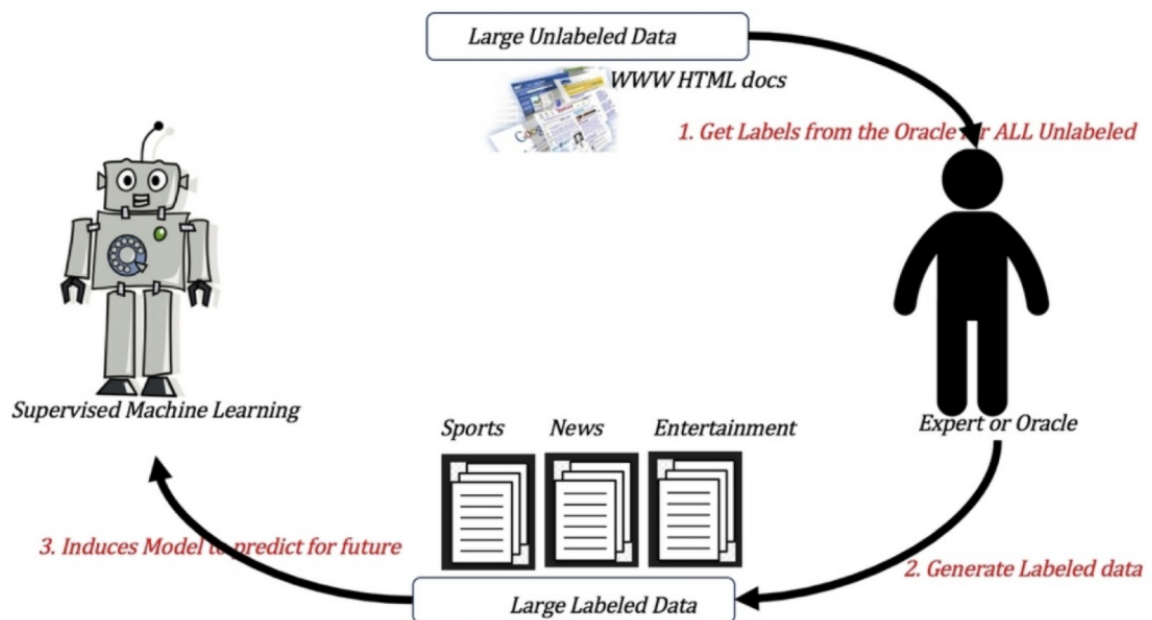


3 半监督学习

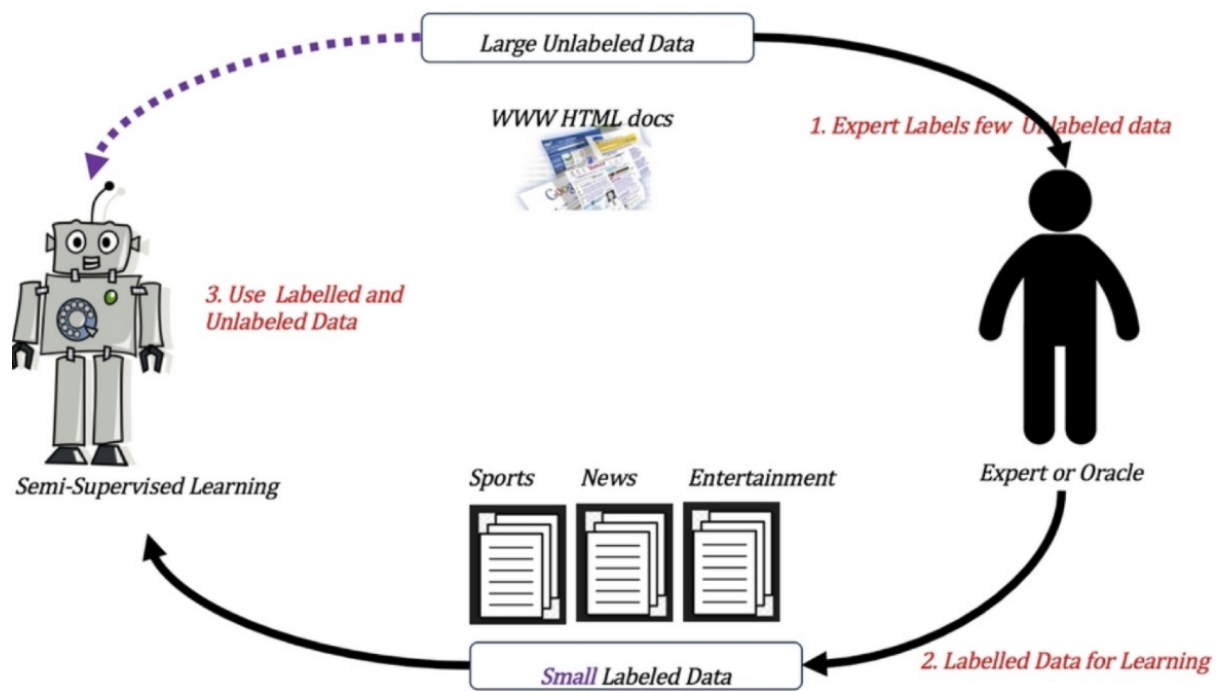
- 定义：
 - 训练集同时包含有标记样本数据和未标记样本数据。

举例：

- 监督学习训练方式：



- 半监督学习训练方式



4 强化学习

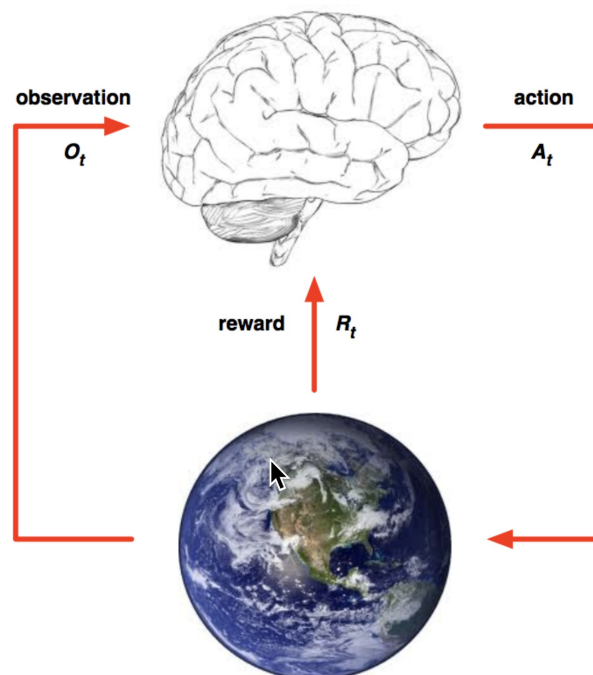
- 定义：
 - 实质是make decisions 问题，即自动进行决策，并且可以做连续决策。

举例：

小孩想要走路，但在这之前，他需要先站起来，站起来之后还要保持平衡，接下来还要先迈出一条腿，是左腿还是右腿，迈出一步后还要迈出下一步。

小孩就是 **agent**，他试图通过采取行动（即行走）来操纵环境（行走的表面），并且从一个状态转变到另一个状态（即他走的每一步），当他完成任务的子任务（即走了几步）时，孩子得到奖励（给巧克力吃），并且当他不能走路时，就不会给巧克力。

主要包含五个元素：agent, action, reward, environment, observation:



强化学习的目标就是获得最多的累计奖励。

监督学习和强化学习的对比

	监督学习	强化学习
反馈映射	输出的是之间的关系，可以告诉算法什么样的输入对应着什么样的输出。	输出的是给机器的反馈 reward function ，即用来判断这个行为是好还是坏。
反馈时间	做了比较坏的选择会立刻反馈给算法。	结果反馈有延时，有时候可能需要走了很多步以后才知道以前的某一步的选择是好还是坏。
输入特征	输入是独立同分布的。	面对输入总是在变化，每当算法做出一个行为，它影响下一次决策的输入。

拓展概念：什么是独立同分布：

独立同分布概念



拓展阅读：AlphaGo进化史 漫画告诉你Zero为什么这么牛：

<http://sports.sina.com.cn/chess/weiqi/2017-10-21/doc-ifmyyxw4023875.shtml>

5 小结

	In	Out	目的	案例
监督学习 (supervised learning)	有标签	有反馈	预测结果	猫狗分类 房价预测
无监督学习 (unsupervised learning)	无标签	无反馈	发现潜在结构	“物以类聚，人以群分”
半监督学习 (Semi-Supervised Learning)	部分有标签，部分无标签	有反馈	降低数据标记的难度	
强化学习 (reinforcement learning)	决策流程及激励系统	一系列行动	长期利益最大化	学下棋

1.6 模型评估

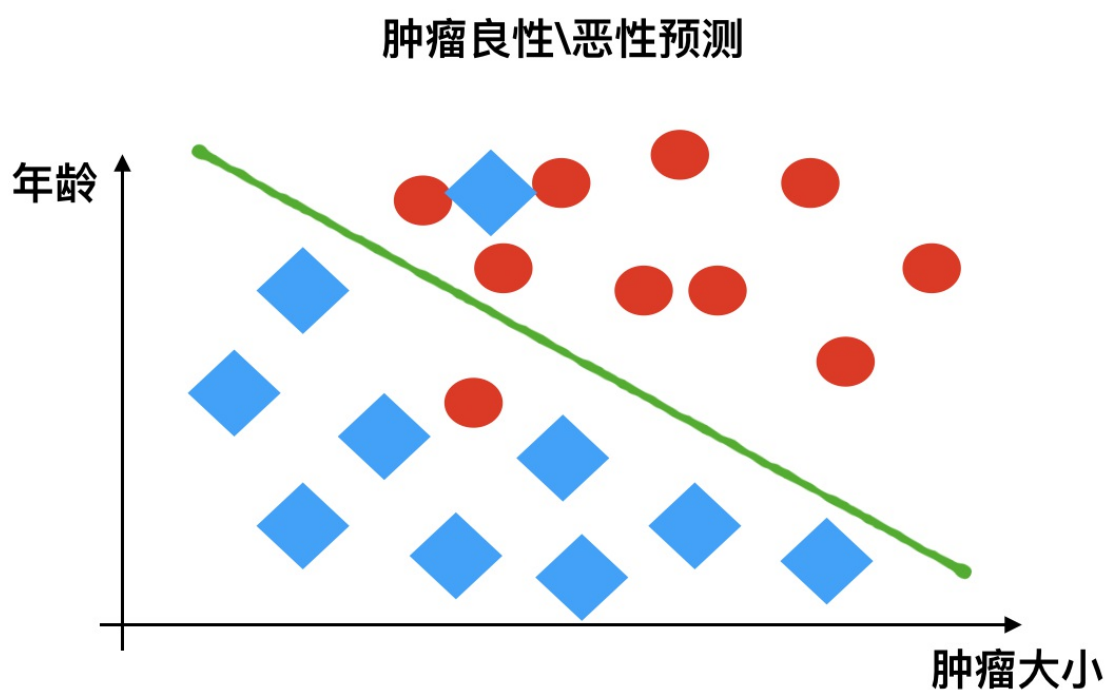
学习目标

- 目标
 - 了解机器学习中模型评估的方法
 - 知道过拟合、欠拟合发生情况

模型评估是模型开发过程不可或缺的一部分。它有助于发现表达数据的最佳模型和所选模型将来工作的性能如何。

按照数据集的目标值不同，可以把模型评估分为分类模型评估和回归模型评估。

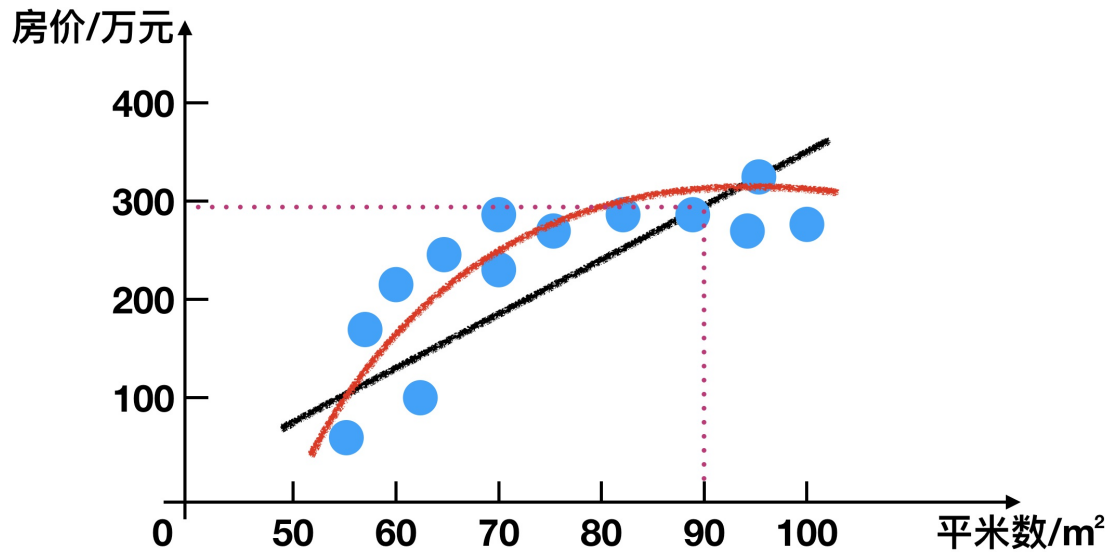
1 分类模型评估



- 准确率
 - 预测正确的数占样本总数的比例。
- 其他评价指标：精确率、召回率、**F1-score**、**AUC**指标等

2 回归模型评估

北京市六环外某小区房价预测模型



均方根误差 (Root Mean Squared Error, RMSE)

- RMSE是一个衡量回归模型误差率的常用公式。不过，它仅能比较误差是相同单位的模型。

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (p_i - a_i)^2}{n}}$$

a = actual target

p = predicted target

举例：

假设上面的房价预测，只有五个样本，对应的
真实值为：100, 120, 125, 230, 400
预测值为：105, 119, 120, 230, 410

那么使用均方根误差求得：

$$RMSE = \sqrt{\frac{[(100 - 105)^2 + (120 - 119)^2 + 5^2 + 0^2 + 10^2]}{5}} = 5.495$$

其他评价指标：相对平方误差 (Relative Squared Error, RSE)、平均绝对误差 (Mean Absolute Error, MAE)、相对绝对误差 (Relative Absolute Error, RAE)

3 拟合

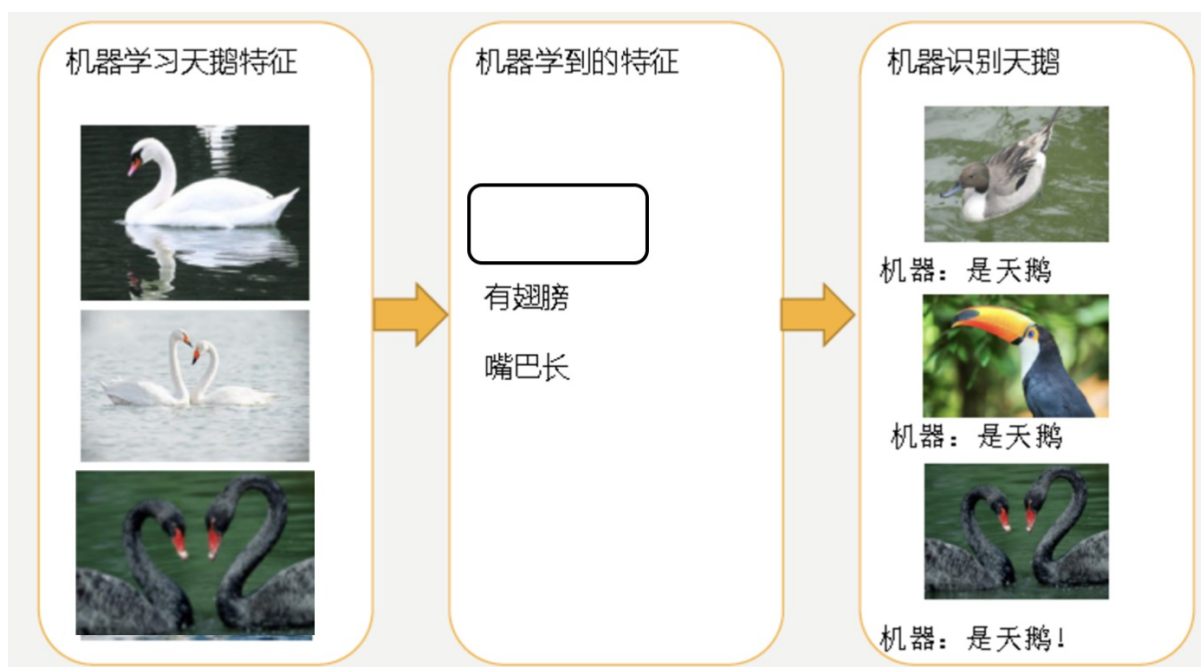
模型评估用于评价训练好的模型的表现效果，其表现效果大致可以分为两类：过拟合、欠拟合。

在训练过程中，你可能会遇到如下问题：

训练数据训练的很好啊，误差也不大，为什么在测试集上面有问题呢？

当算法在某个数据集当中出现这种情况，可能就出现了拟合问题。

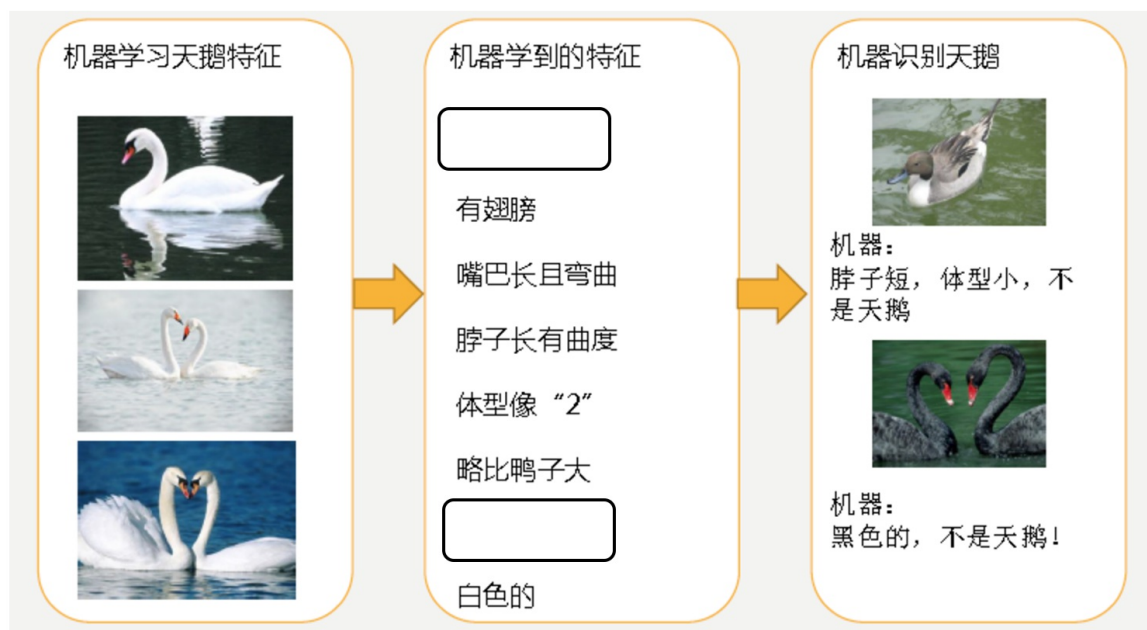
3.1 欠拟合



因为机器学习到的天鹅特征太少了，导致区分标准太粗糙，不能准确识别出天鹅。

欠拟合（under-fitting）：模型学习的太过粗糙，连训练集中的样本数据特征关系都没有学出来。

3.2 过拟合



机器已经基本能区别天鹅和其他动物了。然后，很不巧已有的天鹅图片全是白天鹅的，于是机器经过学习后，会认为天鹅的羽毛都是白的，以后看到羽毛是黑的天鹅就会认为那不是天鹅。

过拟合（over-fitting）：所建的机器学习模型或者是深度学习模型在训练样本中表现得过于优越，导致在测试数据集中表现不佳。

- 上问题解答：
 - 训练数据训练的很好啊，误差也不大，为什么在测试集上面有问题呢？

4 小结

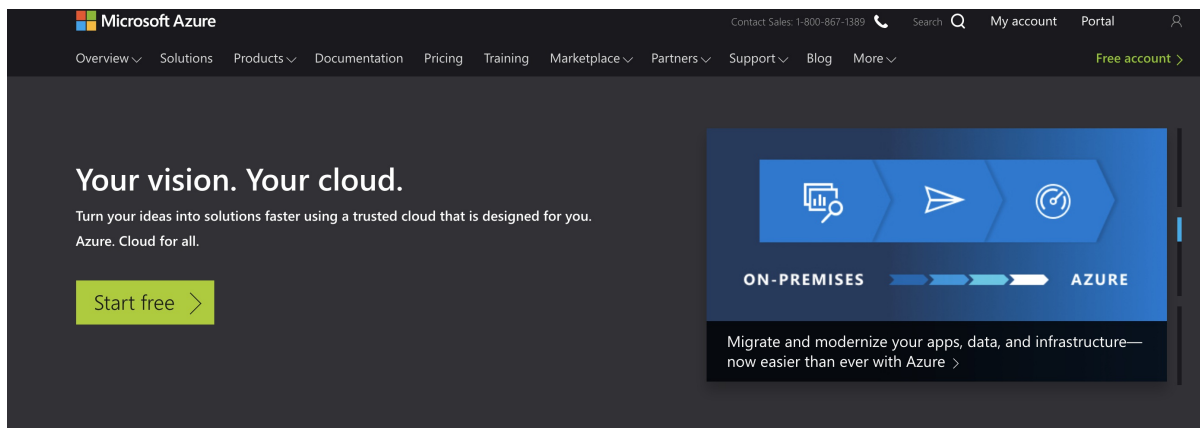
- 分类模型评估【了解】
 - 准确率
- 回归模型评估【了解】
 - RMSE -- 均方根误差
- 拟合【知道】
 - 举例 -- 判断是否是人
 - 欠拟合
 - 学习到的东西太少
 - 模型学习的太过粗糙
 - 过拟合
 - 学习到的东西太多
 - 学习到的特征多，不好泛化

1.7 Azure机器学习模型搭建实验

学习目标

- 目标
 - 了解Azure机器学习平台，知道机器学习流程

Azure平台简介



Azure Machine Learning（简称“AML”）是微软在其公有云Azure上推出的基于Web使用的一项机器学习服务，机器学习属人工智能的一个分支，它技术借助算法让电脑对大量流动数据集进行识别。这种方式能够通过历史数据来预测未来事件和行为，其实现方式明显优于传统的商业智能形式。

微软的目标是简化使用机器学习的过程，以便于开发人员、业务分析师和数据科学家进行广泛、便捷地应用。

这款服务的目的在于“将机器学习动力与云计算的简单性相结合”。

AML目前在微软的Global Azure云服务平台提供服务，用户可以通过站点：<https://studio.azureml.net/> 申请免费试用。



- Azure机器学习实验
 - 实验目的：了解机器学习从数据到建模并最终评估预测的整个流程。

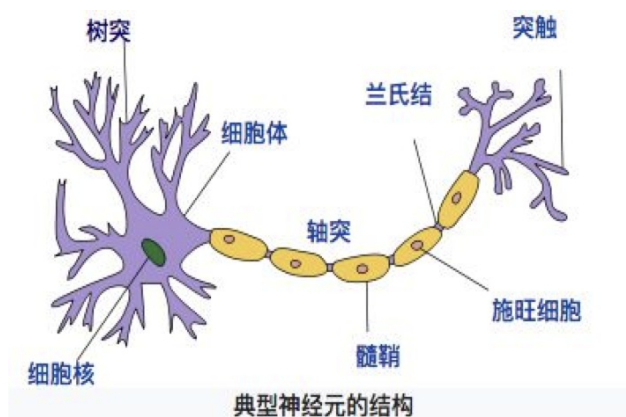
1.8 深度学习简介

学习目标

- 目标
 - 了解什么是深度学习

1 深度学习 —— 神经网络简介

深度学习（Deep Learning）（也称为深度结构学习【Deep Structured Learning】、层次学习【Hierarchical Learning】或者是深度机器学习【Deep Machine Learning】）是一类算法集合，是机器学习的一个分支。



深度学习方法近年来，在会话识别、图像识别和对象侦测等领域表现出了惊人的准确性。

但是，“深度学习”这个词语很古老，它在1986年由Dechter在机器学习领域提出，然后在2000年由Aizenberg等人引入到人工神经网络中。而现在，由于Alex Krizhevsky在2012年使用卷积网络结构赢得了ImageNet比赛之后受到大家的瞩目。

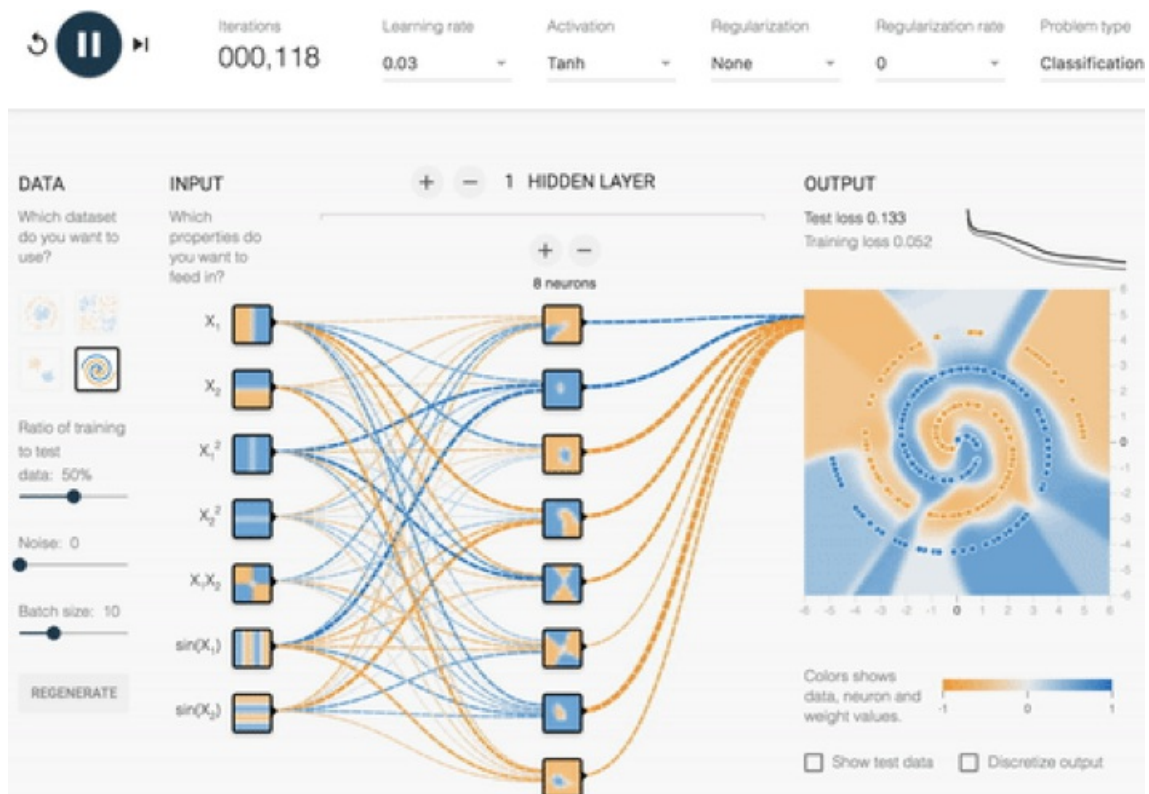
卷积网络之父：Yann LeCun



Yann LeCun
卷积网络之父

[PDF] Backpropagation Applied to Handwritten Zip Code Recognition
yann.lecun.com/exdb/pubs/ptfilecun-89e.pdf • 翻译此页
作者: Y. LeCun - 1989 - 被引用次数: 3168 - 相关文章
Backpropagation Applied to Handwritten Zip Code Recognition, Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, L. D. Jackel, AT&T Bell Laboratories Holmdel, NJ 07733 USA.
The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain.

- 深度学习演示
 - 链接:<http://playground.tensorflow.org>



2 深度学习各层负责内容

神经网络各层负责内容：

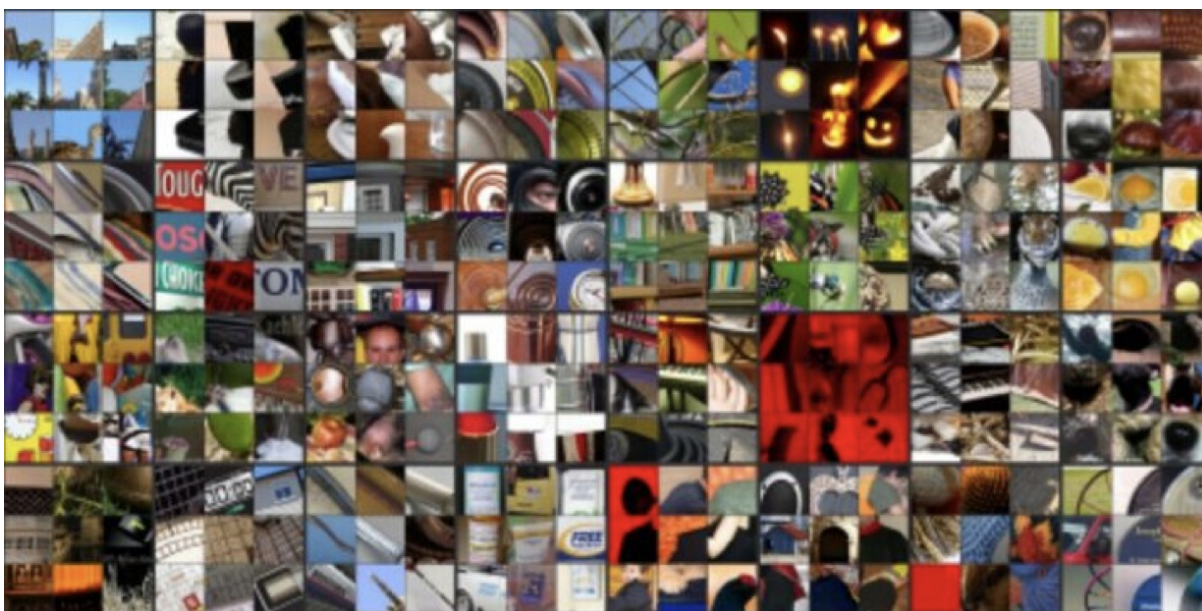
1层：负责识别颜色及简单纹理



2层：一些神经元可以识别更加细化的纹理，布纹，刻纹，叶纹等



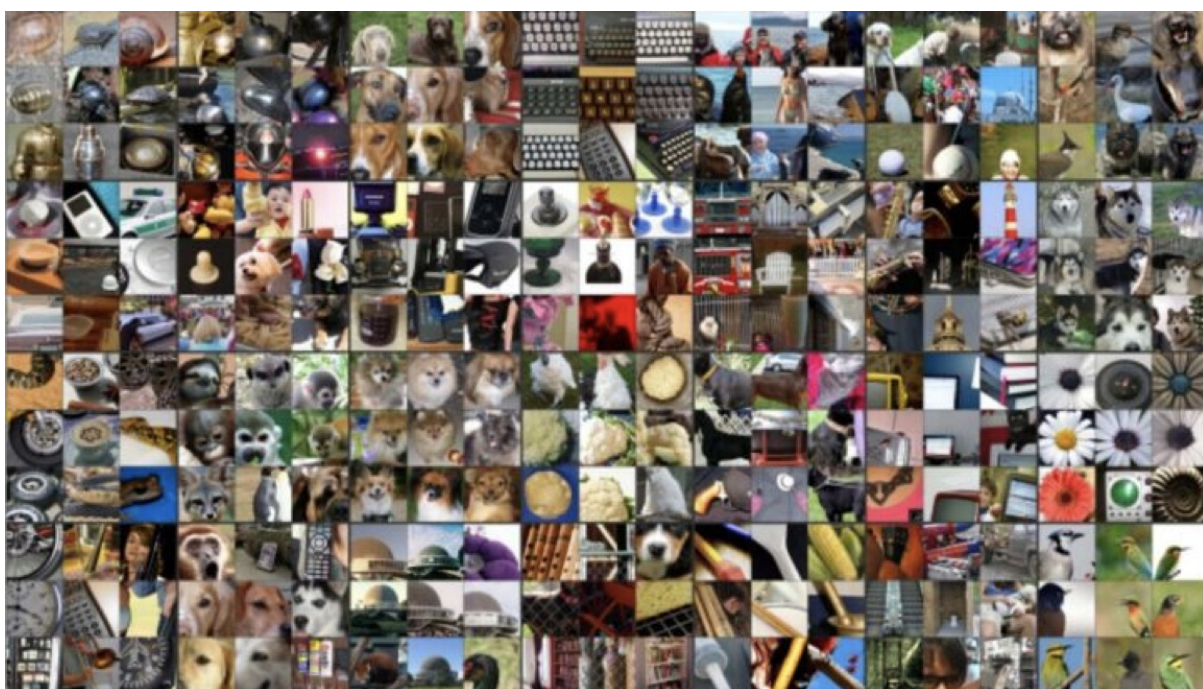
3层：一些神经元负责感受黑夜里的黄色烛光，高光，萤火，鸡蛋黄色等。



4层：一些神经元识别萌狗的脸，宠物形貌，圆柱体事物，七星瓢虫等的存在。



5层：一些神经元负责识别花，黑眼圈动物，鸟，键盘，原型屋顶等。



4 小结

- 深度学习的发展源头--神经网络【了解】
- 多层神经网络，在最初几层是识别简单内容，后面几层是识别一些复杂内容。【了解】

机器学习基础环境安装与使用

学习目标

- 完成机器学习基础阶段的环境安装
- 学会使用jupyter notebook平台完成代码编写运行

2.1 库的安装

学习目标

- 目标
 - 搭建好机器学习基础阶段的环境

整个机器学习基础阶段会用到Matplotlib、Numpy、Pandas等库，为了统一版本号在环境中使用，将所有的库及其版本放到了文件requirements.txt当中，然后统一安装

新建一个用于人工智能环境的虚拟环境

```
mkvirtualenv ai
```

```
matplotlib==2.2.2
numpy==1.14.2
pandas==0.20.3
tables==3.4.2
jupyter==1.0.0
```

注意：

- 每个包安装的过程中，尽量指定稳定版本进行安装

使用pip命令安装

```
pip3 install -r requirements.txt
```

小结

- 机器学习(科学计算库)阶段环境的搭建和基本库的安装
 - 注意：最好安装指定的稳定版本

2.2 Jupyter Notebook使用

学习目标

- 目标
 - 学会使用Jupyter Notebook

1 Jupyter Notebook介绍

Jupyter项目是一个非盈利的开源项目，源于2014年的ipython项目，因为它逐渐发展为支持跨所有编程语言的交互式数据科学和科学计算

- Jupyter Notebook，原名IPython Notebook，是IPython的加强网页版，一个开源Web应用程序
- 名字源自Julia、Python 和 R（数据科学的三种开源语言）
- 是一款程序员和科学工作者的编程/文档/笔记/展示软件
- .ipynb文件格式是用于计算型叙述的JSON文档格式的正式规范



Jupyter项目旨在开发跨几十种编程语言的开源软件，开放标准和用于交互式计算的服务。

2 为什么使用Jupyter Notebook?

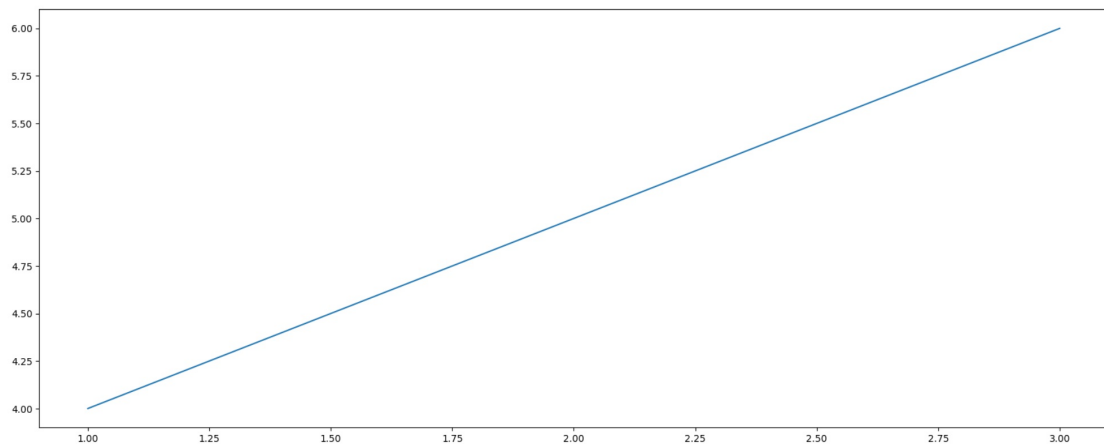
- 传统软件开发：工程 / 目标明确
 - 需求分析，设计架构，开发模块，测试
- 数据挖掘：艺术 / 目标不明确
 - 目的是具体的洞察目标，而不是机械的完成任务
 - 通过执行代码来理解问题
 - 迭代式地改进代码来改进解决方法

实时运行的代码、叙事性的文本和可视化被整合在一起，方便使用代码和数据来讲述故事

对比Jupyter Notebook和Pycharm

- 画图

```
In [1]: import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=(20, 8), dpi=100)
plt.plot([1,2,3], [4,5,6])
plt.show()
```



- 数据展示

```
In [2]: import pandas as pd
stock_day = pd.read_csv("./stock_day/stock_day.csv")
```

```
In [3]: stock_day
```

```
Out[3]:
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
2018-02-27	23.53	25.88	24.16	23.53	95578.03	0.63	2.68	22.942	22.142	22.875	53782.64	46738.65	55576.11	2.39
2018-02-26	22.80	23.78	23.53	22.80	60985.11	0.69	3.02	22.406	21.955	22.942	40827.52	42736.34	56007.50	1.53
2018-02-23	22.88	23.37	22.82	22.71	52914.01	0.54	2.42	21.938	21.929	23.022	35119.58	41871.97	56372.85	1.32
2018-02-22	22.25	22.76	22.28	22.02	36105.01	0.36	1.64	21.446	21.909	23.137	35397.58	39904.78	60149.60	0.90
2018-02-14	21.49	21.99	21.92	21.48	23331.04	0.44	2.05	21.366	21.923	23.253	33590.21	42935.74	61716.11	0.58
2018-02-13	21.40	21.90	21.48	21.31	30802.45	0.28	1.32	21.342	22.103	23.387	39694.65	45518.14	65161.68	0.77
2018-02-12	20.70	21.40	21.19	20.63	32445.39	0.82	4.03	21.504	22.338	23.533	44645.16	45679.94	68686.33	0.81
2018-02-09	21.20	21.46	20.36	20.19	54304.01	-1.50	-6.86	21.920	22.596	23.645	48624.36	48982.38	70552.47	1.36
2018-02-08	21.79	22.09	21.88	21.75	27068.16	0.09	0.41	22.372	23.009	23.839	44411.98	48612.16	73852.45	0.68
2018-02-07	22.69	23.11	21.80	21.29	53853.25	-0.50	-2.24	22.480	23.258	23.929	52281.28	56315.11	74925.33	1.35
2018-02-06	22.80	23.55	22.29	22.20	55555.00	-0.97	-4.17	22.864	23.607	24.029	51341.63	64413.58	75738.95	1.39
2018-02-05	22.45	23.39	23.27	22.25	52341.39	0.65	2.87	23.172	23.928	24.112	46714.72	69278.66	77070.00	1.31

- 总结: Jupyter Notebook 相比 Pycharm 在画图和数据展示方面更有优势。

3 Jupyter Notebook的使用-helloworld

3.1 界面启动、创建文件

- 3.1.1 界面启动

环境搭建好后, 本机输入jupyter notebook命令, 会自动弹出浏览器窗口打开Jupyter Notebook

```
# 进入虚拟环境
workon ai
# 输入命令
jupyter notebook
```

本地notebook的默认URL为: <http://localhost:8888>

想让notebook打开指定目录, 只要进入此目录后执行命令即可



• 3.1.2 新建notebook文档

- notebook的文档格式是 .ipynb



• 3.1.3 内容界面操作-helloworld

标题栏：点击标题（如Untitled）修改文档名

编辑栏：

```
In [1]: print("hello world")
```

```
hello world
```

```
In [2]: a = 1
        b = 2
        c = a + b
```

```
In [3]: a
```

```
Out[3]: 1
```

```
In [4]: b
```

```
Out[4]: 2
```

```
In [5]: c
```

```
Out[5]: 3
```

```
In [6]: import random
        random.random()
```

```
Out[6]: 0.9748235910569898
```

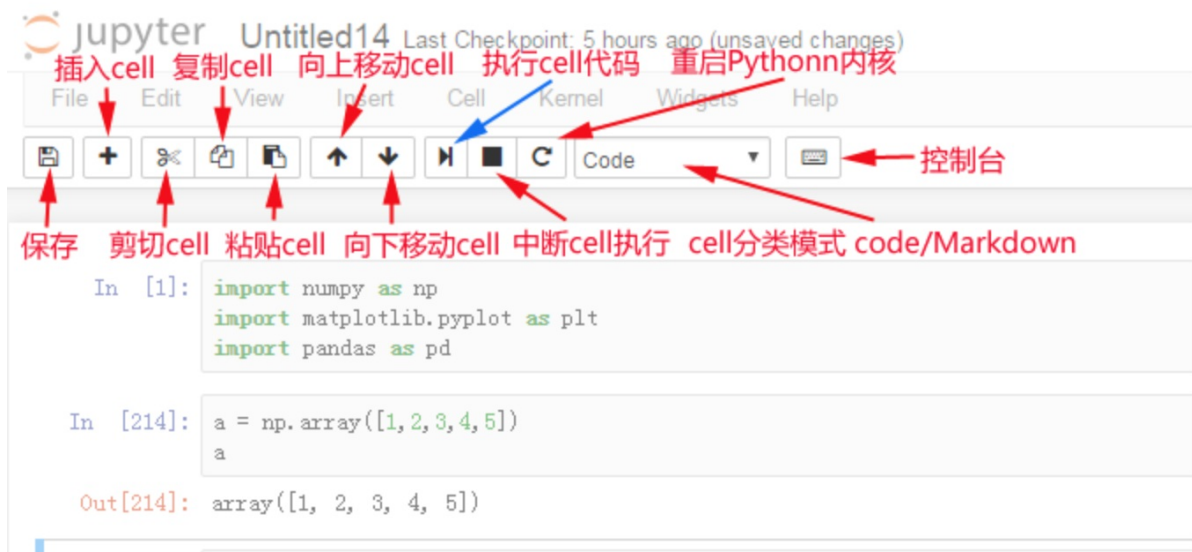
3.2 cell操作

- 什么是cell?
 - **cell**: 一对In Out会话被视作一个代码单元, 称为cell
 - cell行号前的*, 表示代码正在运行

Jupyter支持两种模式:

- 编辑模式 (Enter)
 - 命令模式下 回车Enter 或 鼠标双击 cell进入编辑模式
 - 可以操作cell内文本或代码, 剪切 / 复制 / 粘贴移动等操作
- 命令模式 (Esc)
 - 按 Esc 退出编辑, 进入命令模式
 - 可以操作cell单元本身进行剪切 / 复制 / 粘贴 / 移动等操作

3.2.1 鼠标操作



3.2.2 快捷键操作

- 两种模式通用快捷键
 - **Shift+Enter**, 执行本单元代码, 并跳转到下一单元
 - **Ctrl+Enter**, 执行本单元代码, 留在本单元
- 命令模式: 按ESC进入
 - **Y**, cell切换到Code模式
 - **M**, cell切换到Markdown模式
 - **A**, 在当前cell的上面添加cell
 - **B**, 在当前cell的下面添加cell
- 其他(了解)
 - 双击D: 删除当前cell
 - **Z**, 回退
 - **L**, 为当前cell加上行号 <!--
 - **Ctrl+Shift+P**, 对话框输入命令直接运行
 - 快速跳转到首个cell, **Ctrl+Home**
 - 快速跳转到最后一个cell, **Ctrl+End -->**
- 编辑模式: 按Enter进入

- 补全代码：变量、方法后跟 `Tab`键
- 为一行或多行代码添加/取消注释： `Ctrl+/` (Mac:CMD+/)
- 其他(了解):
 - 多光标操作： `Ctrl`键点击鼠标 (Mac:CMD+点击鼠标)
 - 回退： `Ctrl+Z` (Mac:CMD+Z)
 - 重做： `Ctrl+Y` (Mac:CMD+Y)

3.3 markdown演示

掌握标题和缩进即可



一级标题

二级标题

三级标题

四级标题

五级标题

- 缩进
 - 二级缩进
 - 三级缩进

4 Jupyter Notebook中自动补全代码等相关功能拓展【了解】

效果展示：

Contents 🔄 ⚙️

▼ 1 一级标题

1.1 二级标题

1 一级标题 📌

1.1 二级标题

```
In [2]: 1 print("hello")
        2 print("world")
        3 print("python")
```

4.1 安装jupyter_contrib_nbextensions库

安装该库的命令如下：

```
python -m pip install jupyter_contrib_nbextensions
```

然后执行：

```
jupyter contrib nbextension install --user --skip-running-check
```

在原来的基础上勾选：“Table of Contents”以及“Hinterland”

部分功能：

- | | | |
|--|---|--|
| ☐ (some) LaTeX environments for Jupyter | ☐ 2to3 Converter | ☒ AddBefore |
| ☒ Autopep8 pep8 | ☐ AutoSaveTime | ☒ Autoscroll |
| ☐ Cell Filter | ☒ Code Font Size 字体大小 | ☐ Code prettify |
| ☐ Codefolding | ☐ Codefolding in Editor | ☒ CodeMirror mode extensions |
| ☐ Collapsible Headings | ☐ Comment/Uncomment Hotkey | ☒ contrib_nbextensions_help_item |
| ☐ datestamper | ☐ Equation Auto Numbering | ☐ ExecuteTime |
| ☐ Execution Dependencies | ☐ Exercise | ☐ Exercise2 |
| ☐ Export Embedded HTML | ☐ Freeze | ☐ Gist-it |
| ☐ Help panel | ☐ Hide Header | ☐ Hide input |
| ☐ Hide input all | ☐ Highlight selected word | ☐ highlighter |
| ☒ Hinterland 代码自动补全 | ☐ Initialization cells | ☐ isort formatter |
| ☒ jupyter-js-widgets/extension | ☐ Keyboard shortcut editor | ☒ Launch QtConsole |
| ☐ Limit Output | ☐ Live Markdown Preview | ☐ Load TeX macros |
| ☐ Move selected cells | ☐ Navigation-Hotkeys | ☒ Nbextensions dashboard tab |
| ☒ Nbextensions edit menu item | ☐ nbTranslate | ☐ Notify |
| ☐ Printview | ☐ Python Markdown | ☐ Rubberband |
| ☐ Ruler | ☐ Ruler in Editor | ☐ Runtools |
| ☐ Scratchpad | ☐ ScrollDown | ☐ Select CodeMirror Keymap |
| ☐ SKILL Syntax | ☒ Skip-Traceback | ☐ Snippets |
| ☐ Snippets Menu | ☒ spellchecker 拼写检查 | ☐ Split Cells Notebook |
| ☒ Table of Contents (2) 目录索引 | ☐ table_beautifier | ☒ Toggle all line numbers 代码行号 |
| ☐ Tree Filter | ☐ Variable Inspector | ☐ zenmode |

5 小结

- 是什么
 - 是一个ipython的web加强版
- 为什么要使用jupyter
 - 用于数据探索过程
- 怎么用
 - 1.通过jupyter notebook 就可以使用
 - 2.保存文件是.ipynb

- 3.每个内容,都对应的是一个cell
- 快捷键
 - **Shift+Enter**, 执行本单元代码, 并跳转到下一单元
 - **Ctrl+Enter**, 执行本单元代码, 留在本单元

Matplotlib

学习目标

- 应用Matplotlib的基本功能实现图形显示
- 应用Matplotlib实现多图显示
- 应用Matplotlib实现不同画图种类

3.1 Matplotlib之HelloWorld

学习目标

- 目标
 - 了解什么是matplotlib
 - 为什么要学习matplotlib
 - matplotlib简单图形的绘制

1 什么是Matplotlib



- 是专门用于开发2D图表(包括3D图表)
- 以渐进、交互式方式实现数据可视化

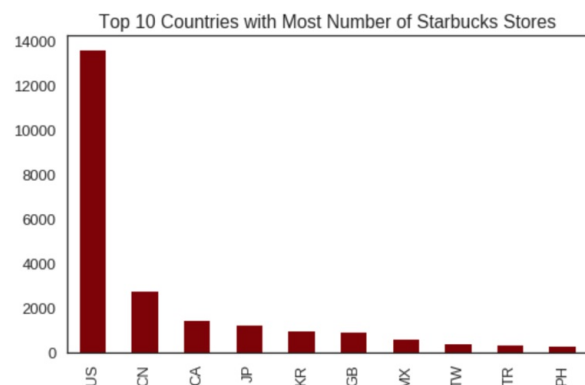
2 为什么要学习Matplotlib

可视化是在整个数据挖掘的关键辅助工具，可以清晰的理解数据，从而调整我们的分析方法。

- 能将数据进行可视化,更直观地呈现
- 使数据更加客观、更具说服力

例如下面两个图分别为数字展示和图形展示：

```
US      13608
CN       2734
CA       1468
JP       1237
KR        993
GB        901
MX        579
TW        394
TR        326
PH        298
Name: Country, dtype: int64
```



3 实现一个简单的Matplotlib画图 — 以折线图为例

3.1 matplotlib.pyplot模块

matplotlib.pyplot包含了一系列类似于matlab的画图函数。

```
import matplotlib.pyplot as plt
```

3.2 图形绘制流程:

- 1.创建画布 -- plt.figure()

```
plt.figure(figsize=(), dpi=)
figsize:指定图的长宽
dpi:图像的清晰度
返回fig对象
```

- 2.绘制图像 -- plt.plot(x, y)

以折线图为例

- 3.显示图像 -- plt.show()

3.3 折线图绘制与显示

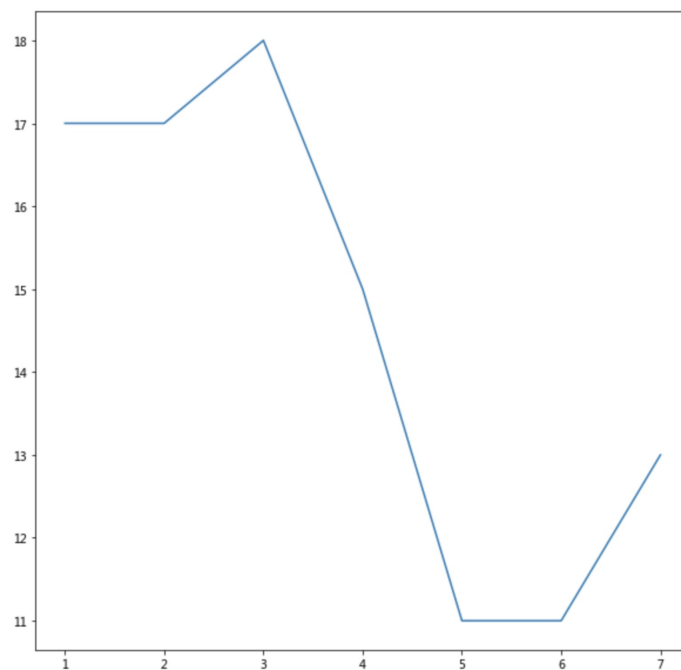
举例：展现上海一周的天气,比如从星期一到星期日的天气温度如下

```
import matplotlib.pyplot as plt

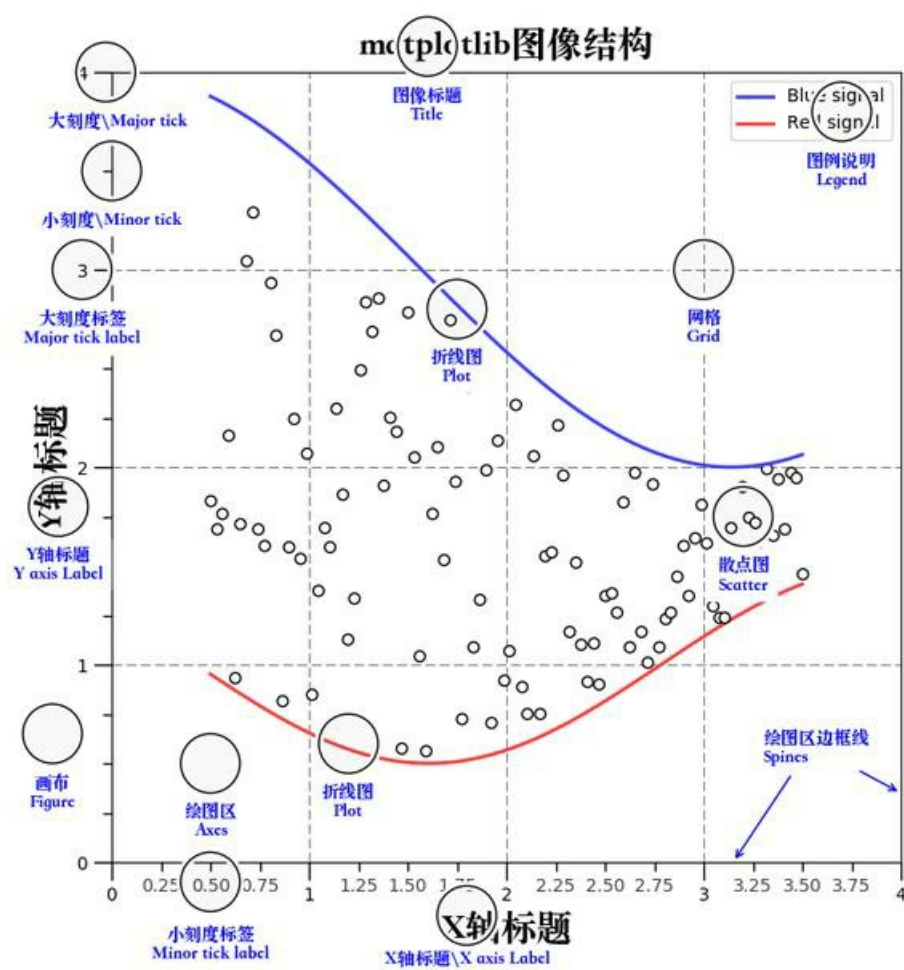
# 1. 创建画布
plt.figure(figsize=(10, 10), dpi=100)

# 2. 绘制折线图
plt.plot([1, 2, 3, 4, 5, 6, 7], [17, 17, 18, 15, 11, 11, 13])

# 3. 显示图像
plt.show()
```



4 认识Matplotlib图像结构(了解)



5 小结

- 什么是matplotlib【了解】
 - 是专门用于开发2D(3D)图表的包
- 绘制图像流程【掌握】
 - 1.创建画布 -- plt.figure(figsize=(20,8))
 - 2.绘制图像 -- plt.plot(x, y)
 - 3.显示图像 -- plt.show()

3.2 基础绘图功能 — 以折线图为例

学习目标

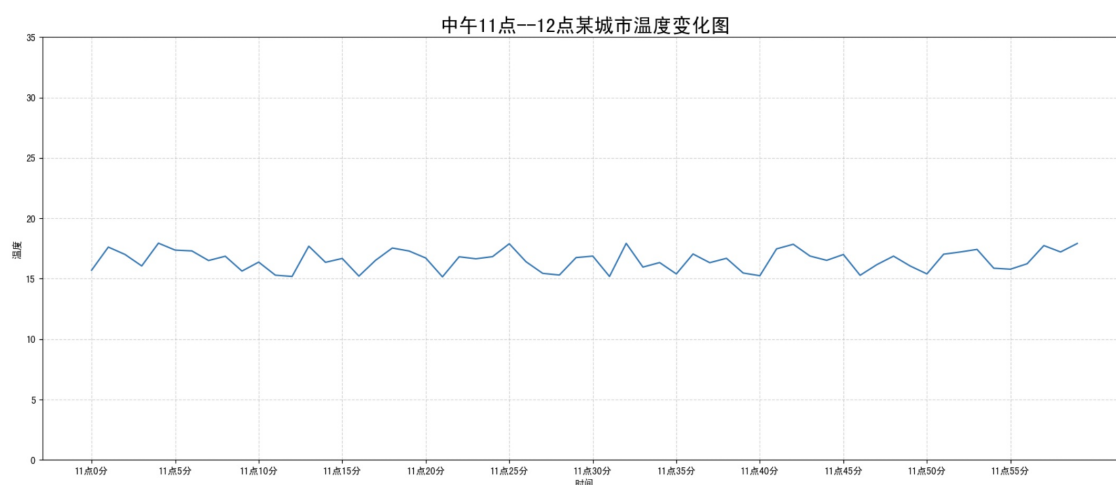
- 目标
 - 掌握给图形添加辅助功能(如: 标注、x,y轴名称、标题等)
 - 知道图形的保存
 - 知道如何多次plot绘制图形
 - 知道如何多个坐标系显示图形
 - 知道折线图的应用场景

1 完善原始折线图 — 给图形添加辅助功能

为了更好地理解所有基础绘图功能, 我们通过天气温度变化的绘图来融合所有的基础API使用

需求: 画出某城市11点到12点1小时内每分钟的温度变化折线图, 温度范围在15度~18度

效果:



1.1 准备数据并画出初始折线图

```
import matplotlib.pyplot as plt
import random

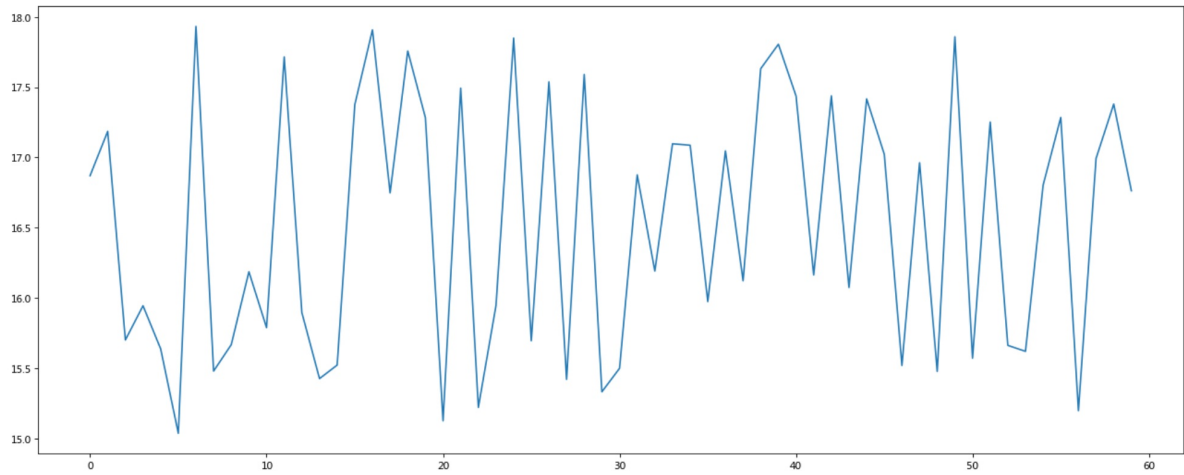
# 画出温度变化图

# 0. 准备x, y坐标的数据
x = range(60)
y_shanghai = [random.uniform(15, 18) for i in x]

# 1. 创建画布
plt.figure(figsize=(20, 8), dpi=80)

# 2. 绘制折线图
plt.plot(x, y_shanghai)

# 3. 显示图像
plt.show()
```



1.2 添加自定义x,y刻度

- `plt.xticks(x, **kwargs)`

x:要显示的刻度值

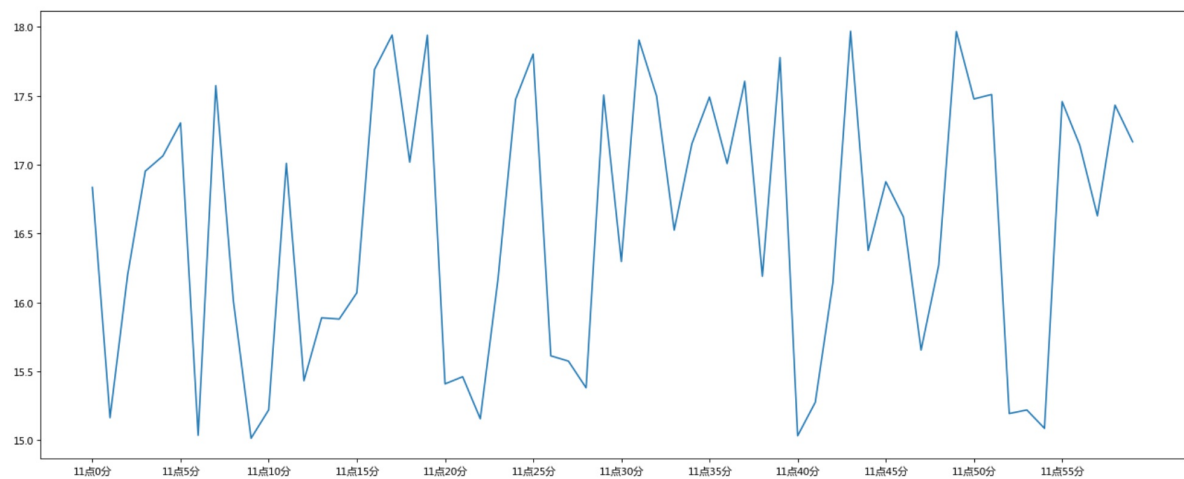
- `plt.yticks(y, **kwargs)`

y:要显示的刻度值

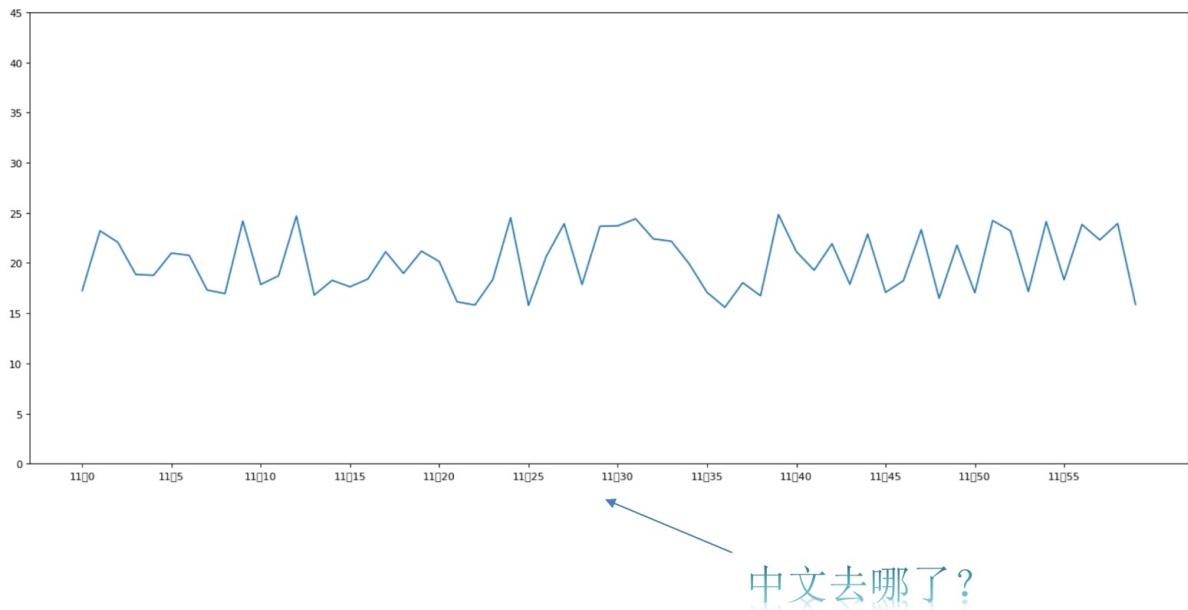
```
# 增加以下两行代码

# 构造x轴刻度标签
x_ticks_label = ["11点{}分".format(i) for i in x]
# 构造y轴刻度
y_ticks = range(40)

# 修改x,y轴坐标的刻度显示
plt.xticks(x[::5], x_ticks_label[::5])
plt.yticks(y_ticks[::5])
```



如果没有解决过中文问题的话，会显示这个样子：



1.3 中文显示问题解决

解决方案一：

下载中文字体（黑体，看准系统版本）

- 步骤一：下载 [SimHei](#) 字体（或者其他的支持中文显示的字体也行）
- 步骤二：安装字体
 - linux下：拷贝字体到 `/usr/share/fonts` 下：

```
sudo cp ~/SimHei.ttf /usr/share/fonts/SimHei.ttf
```

- windows和mac下：双击安装
- 步骤三：删除`~/matplotlib`中的缓存文件

```
cd ~/.matplotlib
rm -rf *
```

- 步骤四：修改配置文件`matplotlibrc`

```
vi ~/.matplotlib/matplotlibrc
```

将文件内容修改为：

```
font.family      : sans-serif
font.sans-serif  : SimHei
axes.unicode_minus : False
```

解决方案二：

在Python脚本中动态设置`matplotlibrc`，这样也可以避免由于更改配置文件而造成的麻烦，具体代码如下：

```
from pylab import mpl
# 设置显示中文字体
mpl.rcParams["font.sans-serif"] = ["SimHei"]
```

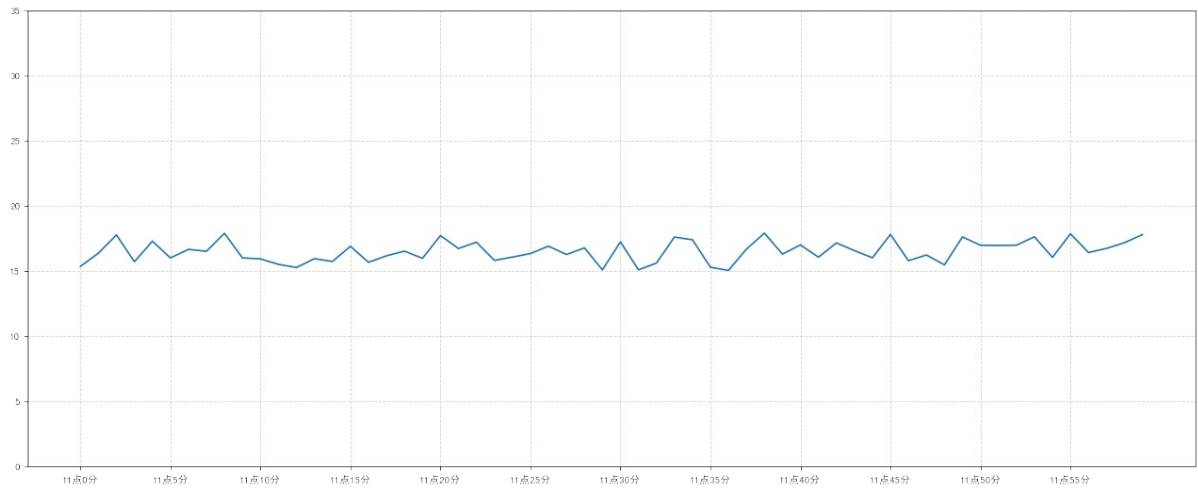
有时候，字体更改后，会导致坐标轴中的部分字符无法正常显示，此时需要更改`axes.unicode_minus`参数：

```
# 设置正常显示符号
mpl.rcParams["axes.unicode_minus"] = False
```

1.4 添加网格显示

为了更加清楚地观察图形对应的值

```
plt.grid(True, linestyle='--', alpha=0.5)
```

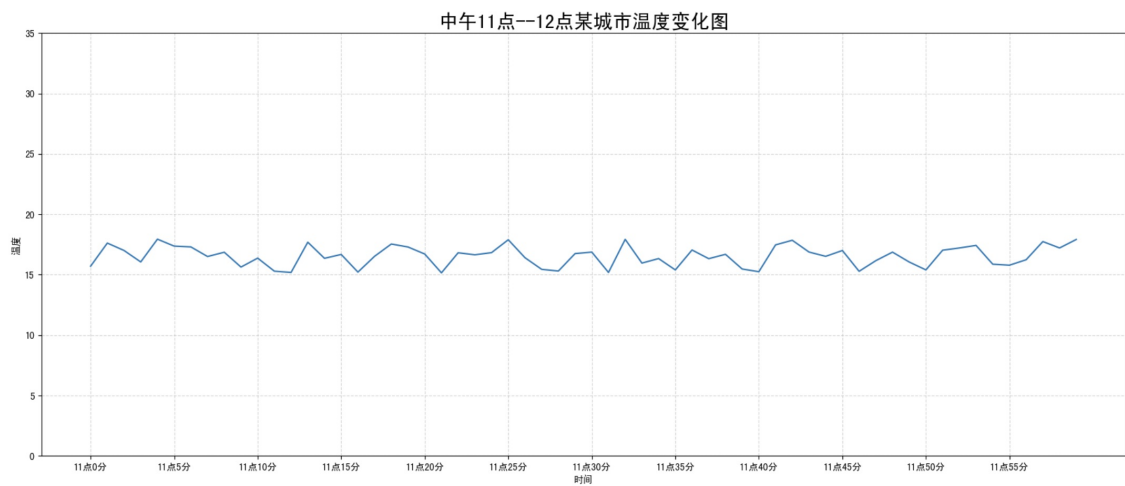


1.5 添加描述信息

添加x轴、y轴描述信息及标题

通过fontsize参数可以修改图像中字体大小

```
plt.xlabel("时间")
plt.ylabel("温度")
plt.title("中午11点0分到12点之间的温度变化图示", fontsize=20)
```



1.6 图像保存

```
# 保存图片到指定路径
plt.savefig("test.png")
```

- 注意: `plt.show()`会释放figure资源, 如果在显示图像之后保存图片将只能保存空图片。

完整代码:

```
import matplotlib.pyplot as plt
import random
from pylab import mpl
```

```

# 设置显示中文字体
mpl.rcParams["font.sans-serif"] = ["SimHei"]
# 设置正常显示符号
mpl.rcParams["axes.unicode_minus"] = False

# 0.准备数据
x = range(60)
y_shanghai = [random.uniform(15, 18) for i in x]

# 1.创建画布
plt.figure(figsize=(20, 8), dpi=100)

# 2.绘制图像
plt.plot(x, y_shanghai)

# 2.1 添加x,y轴刻度
# 构造x,y轴刻度标签
x_ticks_label = ["11点{}分".format(i) for i in x]
y_ticks = range(40)

# 刻度显示
plt.xticks(x[::5], x_ticks_label[::5])
plt.yticks(y_ticks[::5])

# 2.2 添加网格显示
plt.grid(True, linestyle="--", alpha=0.5)

# 2.3 添加描述信息
plt.xlabel("时间")
plt.ylabel("温度")
plt.title("中午11点--12点某城市温度变化图", fontsize=20)

# 2.4 图像保存
plt.savefig("./test.png")

# 3.图像显示
plt.show()

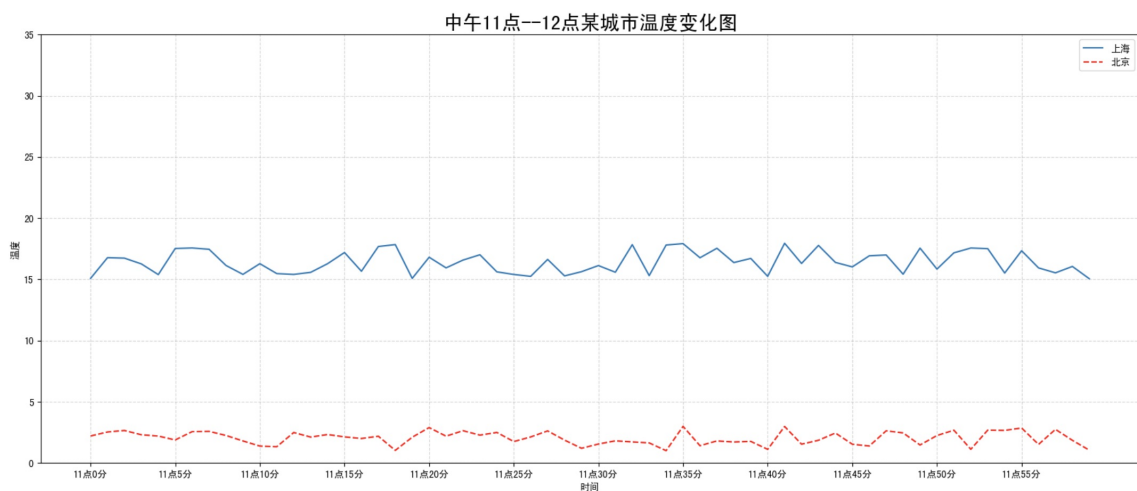
```

2 在一个坐标系中绘制多个图像

2.1 多次plot

需求：再添加一个城市的温度变化

收集到北京当天温度变化情况，温度在1度到3度。怎么去添加另一个在同一坐标系当中的不同图形，其实很简单只需要再次**plot**即可，但是需要区分线条，如下显示



```

# 增加北京的温度数据
y_beijing = [random.uniform(1, 3) for i in x]

# 绘制折线图
plt.plot(x, y_shanghai)
# 使用多次plot可以画多个折线
plt.plot(x, y_beijing, color='r', linestyle='--')

```

我们仔细观察，用到了两个新的地方，一个是对于不同的折线展示效果，一个是添加图例。

2.2 设置图形风格

颜色字符	风格字符
r 红色	- 实线
g 绿色	-- 虚线
b 蓝色	-. 点划线
w 白色	: 点虚线
c 青色	' ' 留空、空格
m 洋红	
y 黄色	
k 黑色	

2.3 显示图例

- 注意：如果只在plt.plot()中设置label还不能最终显示出图例，还需要通过plt.legend()将图例显示出来。

```
# 绘制折线图
plt.plot(x, y_shanghai, label="上海")
# 使用多次plot可以画多个折线
plt.plot(x, y_beijing, color='r', linestyle='--', label="北京")

# 显示图例
plt.legend(loc="best")
```

Location String	Location Code
'best'	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

完整代码：

```
# 0.准备数据
x = range(60)
y_shanghai = [random.uniform(15, 18) for i in x]
y_beijing = [random.uniform(1,3) for i in x]

# 1.创建画布
plt.figure(figsize=(20, 8), dpi=100)

# 2.绘制图像
plt.plot(x, y_shanghai, label="上海")
plt.plot(x, y_beijing, color="r", linestyle="--", label="北京")

# 2.1 添加x,y轴刻度
# 构造x,y轴刻度标签
x_ticks_label = ["11点{}分".format(i) for i in x]
y_ticks = range(40)

# 刻度显示
```

```
plt.xticks(x[::5], x_ticks_label[::5])
plt.yticks(y_ticks[::5])

# 2.2 添加网格显示
plt.grid(True, linestyle="--", alpha=0.5)

# 2.3 添加描述信息
plt.xlabel("时间")
plt.ylabel("温度")
plt.title("中午11点--12点某城市温度变化图", fontsize=20)

# 2.4 图像保存
plt.savefig("./test.png")

# 2.5 添加图例
plt.legend(loc=0)

# 3. 图像显示
plt.show()
```

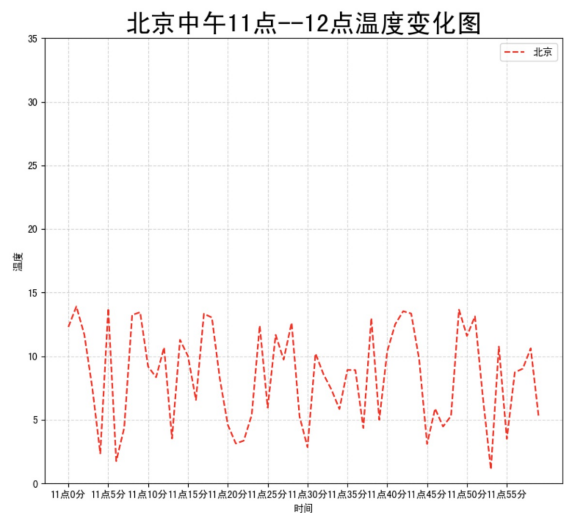
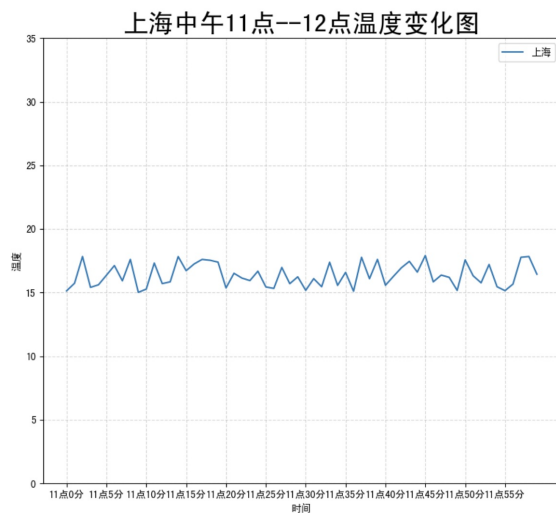
2.4 练一练

练习多次plot流程(从上面复制代码,到自己电脑,确保每人环境可以正常运行),

同时明确每个过程执行实现的具体效果

3 多个坐标系显示— plt.subplots(面向对象的画图方法)

如果我们想要将上海和北京的天气图显示在同一个图的不同坐标系当中, 效果如下:



可以通过subplots函数实现(旧的版本中有subplot, 使用起来不方便), 推荐subplots函数

- `matplotlib.pyplot.subplots(nrows=1, ncols=1, **fig_kw)` 创建一个带有多个axes(坐标系/绘图区)的图

Parameters:

`nrows, ncols` : 设置有几行几列坐标系
int, optional, default: 1, Number of rows/columns of the subplot grid.

Returns:

`fig` : 图对象
`axes` : 返回相应数量的坐标系

设置标题等方法不同:

```
set_xticks
set_yticks
set_xlabel
set_ylabel
```

关于axes子坐标系的更多方法: 参考https://matplotlib.org/api/axes_api.html#matplotlib.axes.Axes

- 注意: `plt.函数名()` 相当于面向过程的画图方法, `axes.set_方法名()` 相当于面向对象的画图方法。

```

# 0.准备数据
x = range(60)
y_shanghai = [random.uniform(15, 18) for i in x]
y_beijing = [random.uniform(1, 5) for i in x]

# 1.创建画布
# plt.figure(figsize=(20, 8), dpi=100)
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 8), dpi=100)

# 2.绘制图像
# plt.plot(x, y_shanghai, label="上海")
# plt.plot(x, y_beijing, color="r", linestyle="--", label="北京")
axes[0].plot(x, y_shanghai, label="上海")
axes[1].plot(x, y_beijing, color="r", linestyle="--", label="北京")

# 2.1 添加x,y轴刻度
# 构造x,y轴刻度标签
x_ticks_label = ["11点{}分".format(i) for i in x]
y_ticks = range(40)

# 刻度显示
# plt.xticks(x[::5], x_ticks_label[::5])
# plt.yticks(y_ticks[::5])
axes[0].set_xticks(x[::5])
axes[0].set_yticks(y_ticks[::5])
axes[0].set_xticklabels(x_ticks_label[::5])
axes[1].set_xticks(x[::5])
axes[1].set_yticks(y_ticks[::5])
axes[1].set_xticklabels(x_ticks_label[::5])

# 2.2 添加网格显示
# plt.grid(True, linestyle="--", alpha=0.5)
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[1].grid(True, linestyle="--", alpha=0.5)

# 2.3 添加描述信息
# plt.xlabel("时间")
# plt.ylabel("温度")
# plt.title("中午11点--12点某城市温度变化图", fontsize=20)
axes[0].set_xlabel("时间")
axes[0].set_ylabel("温度")
axes[0].set_title("中午11点--12点某城市温度变化图", fontsize=20)
axes[1].set_xlabel("时间")
axes[1].set_ylabel("温度")
axes[1].set_title("中午11点--12点某城市温度变化图", fontsize=20)

# # 2.4 图像保存
plt.savefig("./test.png")

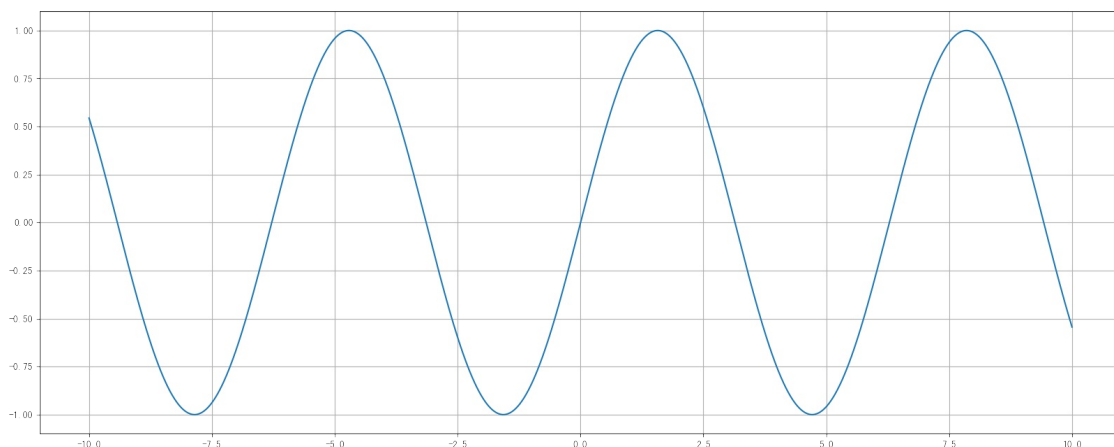
# # 2.5 添加图例
# plt.legend(loc=0)
axes[0].legend(loc=0)
axes[1].legend(loc=0)

# 3. 图像显示
plt.show()

```

4 折线图的应用场景

- 呈现公司产品(不同区域)每天活跃用户数
- 呈现app每天下载数量
- 呈现产品新功能上线后,用户点击次数随时间的变化
- 拓展：画各种数学函数图像
 - 注意：plt.plot()除了可以画折线图，也可以用于画各种数学函数图像



代码:

```
import numpy as np
# 0.准备数据
x = np.linspace(-10, 10, 1000)
y = np.sin(x)

# 1.创建画布
plt.figure(figsize=(20, 8), dpi=100)

# 2.绘制函数图像
plt.plot(x, y)
# 2.1 添加网格显示
plt.grid()

# 3.显示图像
plt.show()
```

5 小结

- 添加x,y轴刻度【知道】
 - `plt.xticks()`
 - `plt.yticks()`
 - 注意:在传递进去的第一个参数必须是数字,不能是字符串,如果是字符串吗,需要进行替换操作
- 添加网格显示【知道】
 - `plt.grid(linestyle="--", alpha=0.5)`
- 添加描述信息【知道】
 - `plt.xlabel()`
 - `plt.ylabel()`
 - `plt.title()`
- 图像保存【知道】
 - `plt.savefig("路径")`
- 多次plot【了解】
 - 直接进行添加就OK
- 显示图例【知道】
 - `plt.legend(loc="best")`
 - 注意:一定要在`plt.plot()`里面设置一个`label`,如果不设置,没法显示
- 多个坐标系显示【了解】
 - `plt.subplots(nrows=, ncols=)`
- 折线图的应用【知道】
 - 1.应用于观察数据的变化
 - 2.可是画出一些数学函数图像

3.3 常见图形绘制

学习目标

- 目标
 - 掌握常见统计图及其意义

Matplotlib能够绘制折线图、散点图、柱状图、直方图、饼图。

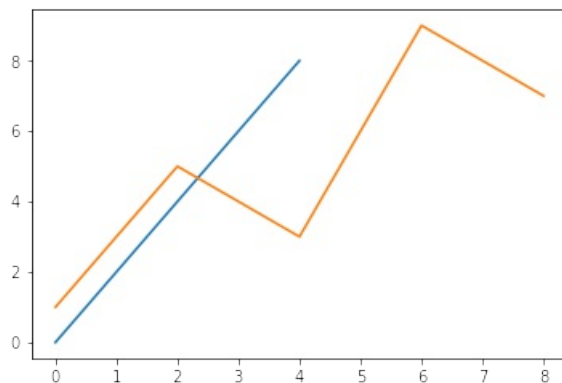
我们需要知道不同的统计图的意义，以此来决定选择哪种统计图来呈现我们的数据。

1 常见图形种类及意义

- 折线图：以折线的上升或下降来表示统计数量的增减变化的统计图

特点：能够显示数据的变化趋势，反映事物的变化情况。(变化)

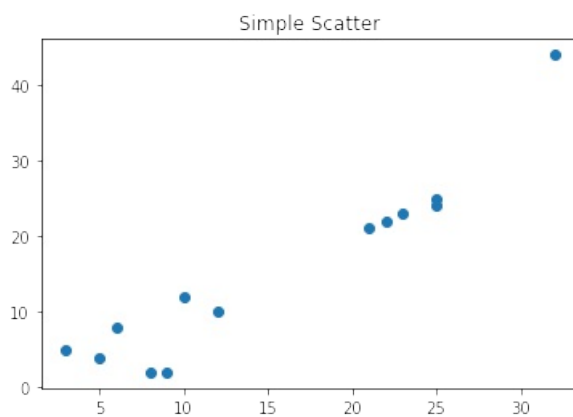
api: `plt.plot(x, y)`



- 散点图：用两组数据构成多个坐标点，考察坐标点的分布,判断两变量之间是否存在某种关联或总结坐标点的分布模式。

特点：判断变量之间是否存在数量关联趋势,展示离群点(分布规律)

api: `plt.scatter(x, y)`



- 柱状图：排列在工作表的列或行中的数据可以绘制到柱状图中。

特点：绘制离散的数据,能够一眼看出各个数据的大小,比较数据之间的差别。(统计/对比)

api: `plt.bar(x, width, align='center', **kwargs)`

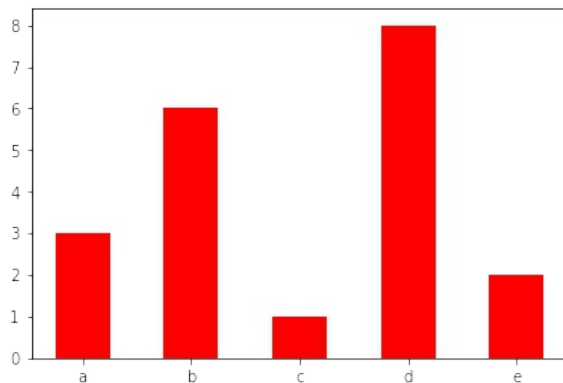
Parameters:
x : 需要传递的数据

width : 柱状图的宽度

align : 每个柱状图的位置对齐方式
{'center', 'edge'}, optional, default: 'center'

****kwargs** :

color: 选择柱状图的颜色

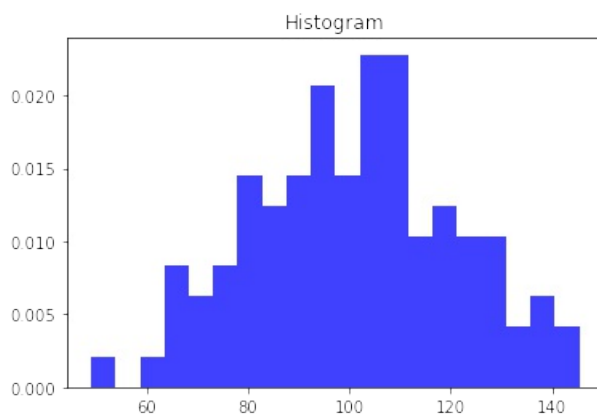


- 直方图：由一系列高度不等的纵向条纹或线段表示数据分布的情况。一般用横轴表示数据范围，纵轴表示分布情况。

特点：绘制连续性的数据展示一组或者多组数据的分布状况(统计)

api: `matplotlib.pyplot.hist(x, bins=None)`

Parameters:
x : 需要传递的数据
bins : 组距

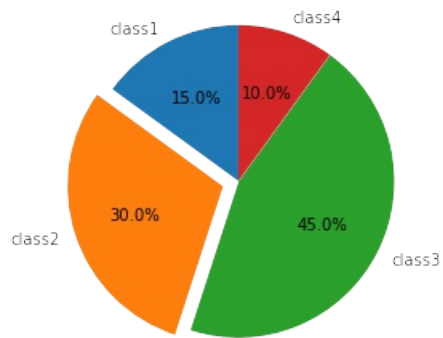


- 饼图：用于表示不同分类的占比情况，通过弧度大小来对比各种分类。

特点：分类数据的占比情况(占比)

api: `plt.pie(x, labels=, autopct=, colors)`

Parameters:
x: 数量，自动算百分比
labels: 每部分名称
autopct: 占比显示指定%1.2f%%
colors: 每部分颜色



2 散点图绘制

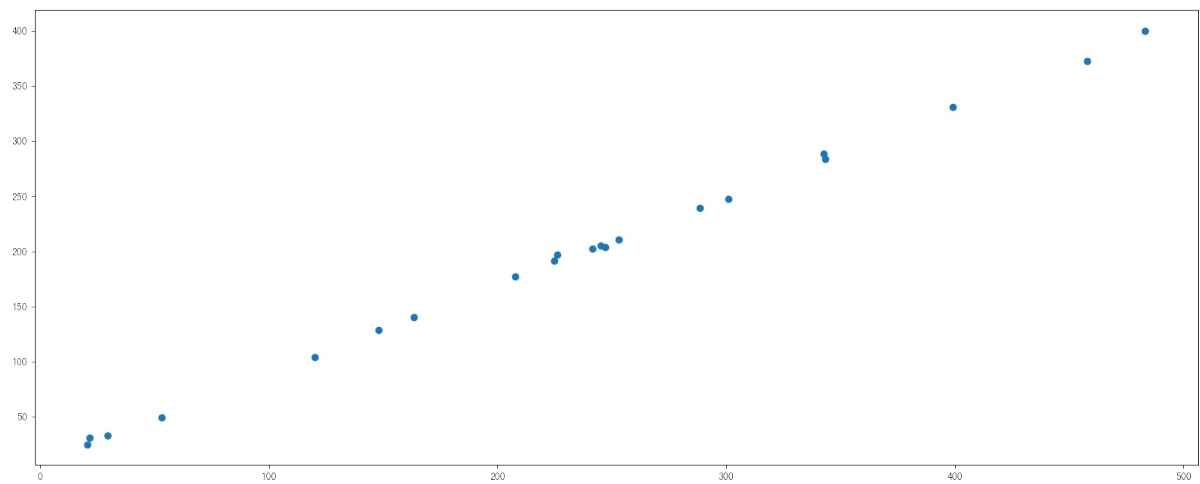
需求：探究房屋面积和房屋价格的关系

房屋面积数据：

```
x = [225.98, 247.07, 253.14, 457.85, 241.58, 301.01, 20.67, 288.64,
      163.56, 120.06, 207.83, 342.75, 147.9, 53.06, 224.72, 29.51,
      21.61, 483.21, 245.25, 399.25, 343.35]
```

房屋价格数据：

```
y = [196.63, 203.88, 210.75, 372.74, 202.41, 247.61, 24.9, 239.34,
      140.32, 104.15, 176.84, 288.23, 128.79, 49.64, 191.74, 33.1,
      30.74, 400.02, 205.35, 330.64, 283.45]
```



代码：

```
# 0.准备数据
x = [225.98, 247.07, 253.14, 457.85, 241.58, 301.01, 20.67, 288.64,
      163.56, 120.06, 207.83, 342.75, 147.9, 53.06, 224.72, 29.51,
      21.61, 483.21, 245.25, 399.25, 343.35]
y = [196.63, 203.88, 210.75, 372.74, 202.41, 247.61, 24.9, 239.34,
      140.32, 104.15, 176.84, 288.23, 128.79, 49.64, 191.74, 33.1,
      30.74, 400.02, 205.35, 330.64, 283.45]

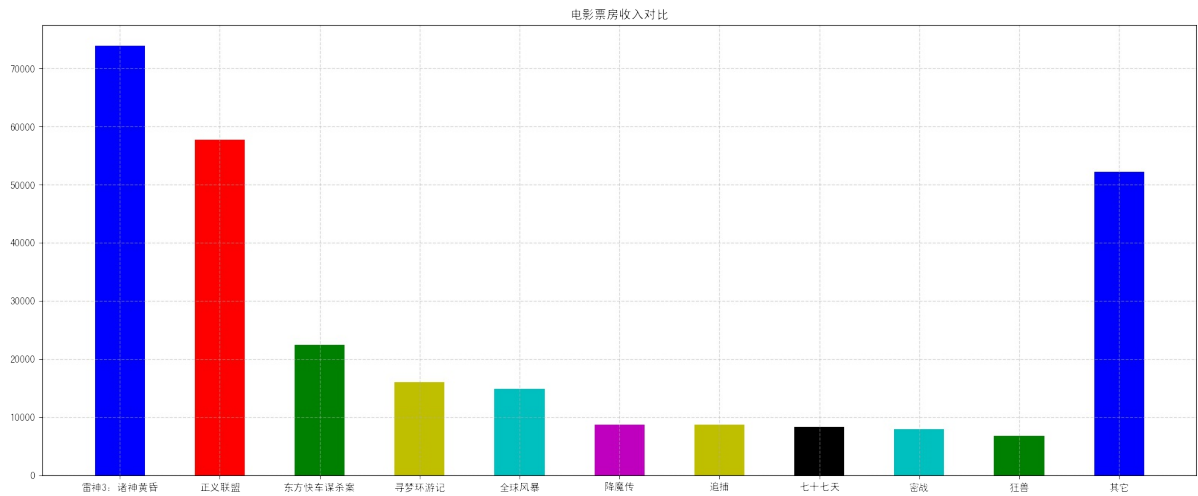
# 1.创建画布
plt.figure(figsize=(20, 8), dpi=100)

# 2.绘制散点图
plt.scatter(x, y)

# 3.显示图像
plt.show()
```

3 柱状图绘制

需求-对比每部电影的票房收入



电影数据如下图所示：

排名.影片名	单月票房(万)	月度占比	平均票价	场均人次	上映日期	口碑指数	月内天数
1.雷神3：诸神黄昏	73853	26.6%	35	15	2017-11-03	-	28
2.正义联盟	57767	20.8%	35	15	2017-11-17	-	14
3.东方快车谋杀案	22354	8.1%	31	12	2017-11-10	-	21
4.寻梦环游记	15969	5.8%	34	19	2017-11-24	-	7
5.全球风暴	14839	5.3%	34	9	2017-10-27	-	30
6.降魔传	8725	3.1%	34	7	2017-11-17	-	14
7.追捕	8716	3.1%	32	8	2017-11-24	-	7
8.七十七天	8318	3.0%	33	11	2017-11-03	-	28
9.密战	7916	2.9%	31	8	2017-11-03	-	28
10.狂兽	6764	2.4%	35	6	2017-11-10	-	21
其他	52222	18.8%	32	7	-	-	-

● 准备数据

```
['雷神3：诸神黄昏','正义联盟','东方快车谋杀案','寻梦环游记','全球风暴','降魔传','追捕','七十七天','密战','狂兽','其它']  
[73853,57767,22354,15969,14839,8725,8716,8318,7916,6764,52222]
```

● 绘制柱状图

代码：

```
# 0. 准备数据  
# 电影名字  
movie_name = ['雷神3：诸神黄昏','正义联盟','东方快车谋杀案','寻梦环游记','全球风暴','降魔传','追捕','七十七天','密战','狂兽','其它']
```

```

# 横坐标
x = range(len(movie_name))
# 票房数据
y = [73853, 57767, 22354, 15969, 14839, 8725, 8716, 8318, 7916, 6764, 52222]

# 1. 创建画布
plt.figure(figsize=(20, 8), dpi=100)

# 2. 绘制柱状图
plt.bar(x, y, width=0.5, color=['b', 'r', 'g', 'y', 'c', 'm', 'y', 'k', 'c', 'g', 'b'])

# 2.1b 修改x轴的刻度显示
plt.xticks(x, movie_name)

# 2.2 添加网格显示
plt.grid(linestyle="--", alpha=0.5)

# 2.3 添加标题
plt.title("电影票房收入对比")

# 3. 显示图像
plt.show()

```

参考链接:

<https://matplotlib.org/index.html>

4 小结

- 折线图【知道】
 - 能够显示数据的变化趋势, 反映事物的变化情况。(变化)
 - `plt.plot()`
- 散点图【知道】
 - 判断变量之间是否存在数量关联趋势, 展示离群点(分布规律)
 - `plt.scatter()`
- 柱状图【知道】
 - 绘制离散的数据, 能够一眼看出各个数据的大小, 比较数据之间的差别。(统计/对比)
 - `plt.bar(x, width, align="center")`
- 直方图【知道】
 - 绘制连续性的数据展示一组或者多组数据的分布状况(统计)
 - `plt.hist(x, bins)`
- 饼图【知道】
 - 用于表示不同分类的占比情况, 通过弧度大小来对比各种分类
 - `plt.pie(x, labels, autopct, colors)`

Numpy

学习目标

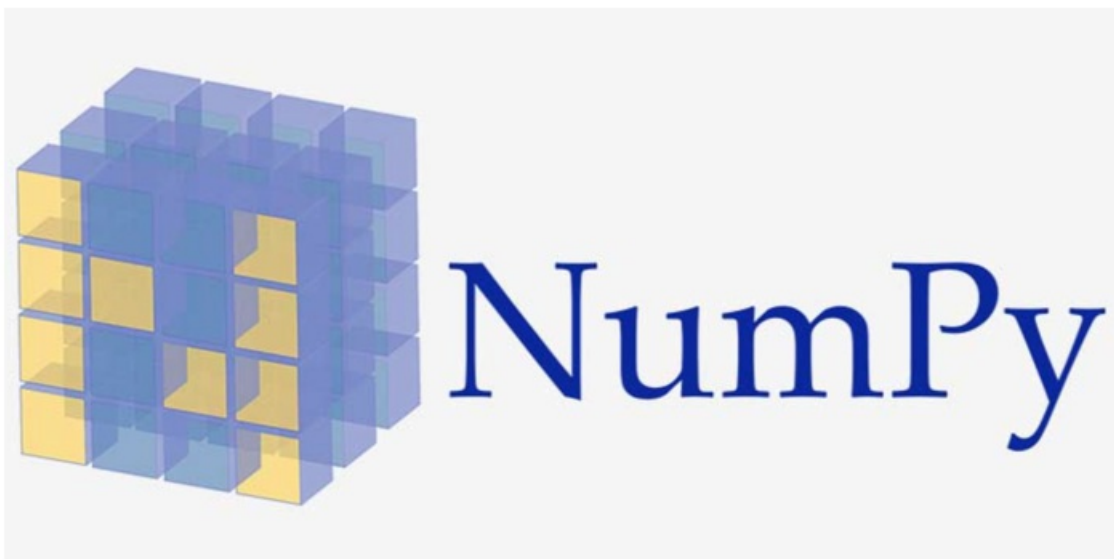
- 了解Numpy运算速度上的优势
- 知道数组的属性，形状、类型
- 应用Numpy实现数组的基本操作
- 应用随机数组的创建实现正态分布应用
- 应用Numpy实现数组的逻辑运算
- 应用Numpy实现数组的统计运算
- 应用Numpy实现数组之间的运算

4.1 Numpy优势

学习目标

- 目标
 - 了解Numpy运算速度上的优势
 - 知道Numpy的数组内存块风格
 - 知道Numpy的并行化运算

1 Numpy介绍



NumPy (Numerical Python) 是一个开源的Python科学计算库，用于快速处理任意维度的数组。

NumPy支持常见的数组和矩阵操作。对于同样的数值计算任务，使用NumPy比直接使用Python要简洁的多。

NumPy使用**ndarray**对象来处理多维数组，该对象是一个快速而灵活的大数据容器。

2 ndarray介绍

```
NumPy provides an N-dimensional array type, the ndarray,  
which describes a collection of "items" of the same type.
```

NumPy提供了一个**N**维数组类型**ndarray**，它描述了相同类型的“items”的集合。

语文	数学	英语	政治	体育
80	89	86	67	79
78	97	89	67	81
90	94	78	67	74
91	91	90	67	69
76	87	75	67	86
70	79	84	67	84
94	92	93	67	64
86	85	83	67	80

用ndarray进行存储:

```
import numpy as np

# 创建ndarray
score = np.array(
[[80, 89, 86, 67, 79],
 [78, 97, 89, 67, 81],
 [90, 94, 78, 67, 74],
 [91, 91, 90, 67, 69],
 [76, 87, 75, 67, 86],
 [70, 79, 84, 67, 84],
 [94, 92, 93, 67, 64],
 [86, 85, 83, 67, 80]])

score
```

返回结果:

```
array([[80, 89, 86, 67, 79],
       [78, 97, 89, 67, 81],
       [90, 94, 78, 67, 74],
       [91, 91, 90, 67, 69],
       [76, 87, 75, 67, 86],
       [70, 79, 84, 67, 84],
       [94, 92, 93, 67, 64],
       [86, 85, 83, 67, 80]])
```

提问:

使用Python列表可以存储一维数组,通过列表的嵌套可以实现多维数组,那么为什么还需要使用Numpy的ndarray呢?

3 ndarray与Python原生list运算效率对比

在这里我们通过一段代码运行来体会到ndarray的好处

```
import random
import time
import numpy as np
a = []
for i in range(100000000):
    a.append(random.random())

# 通过%time魔法方法,查看当前行的代码运行一次所花费的时间
%time sum1=sum(a)

b=np.array(a)

%time sum2=np.sum(b)
```

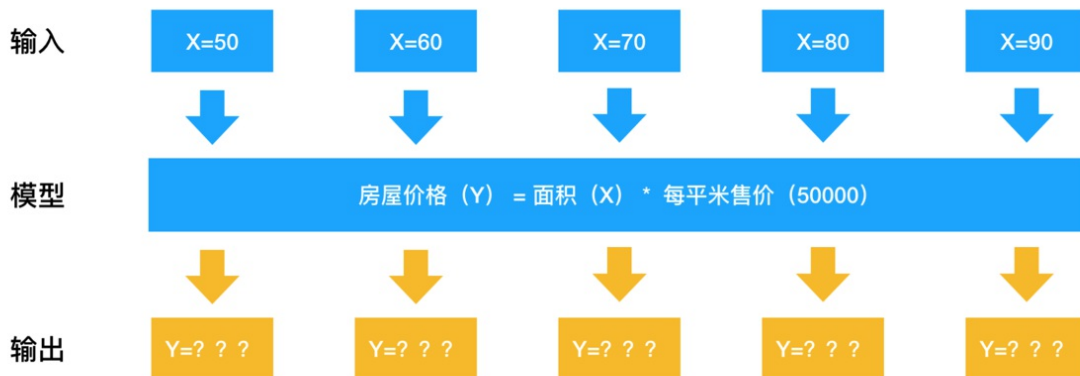
其中第一个时间显示的是使用原生Python计算时间,第二个内容是使用numpy计算时间:

```
CPU times: user 852 ms, sys: 262 ms, total: 1.11 s
Wall time: 1.13 s
```

```
CPU times: user 133 ms, sys: 653 μs, total: 133 ms
Wall time: 134 ms
```

从中我们看到ndarray的计算速度要快很多，节约了时间。

机器学习的最大特点就是大量的数据运算，那么如果没有一个快速的解决方案，那可能现在python也在机器学习领域达不到好的效果。



Numpy专门针对ndarray的操作和运算进行了设计，所以数组的存储效率和输入输出性能远优于Python中的嵌套列表，数组越大，Numpy的优势就越明显。

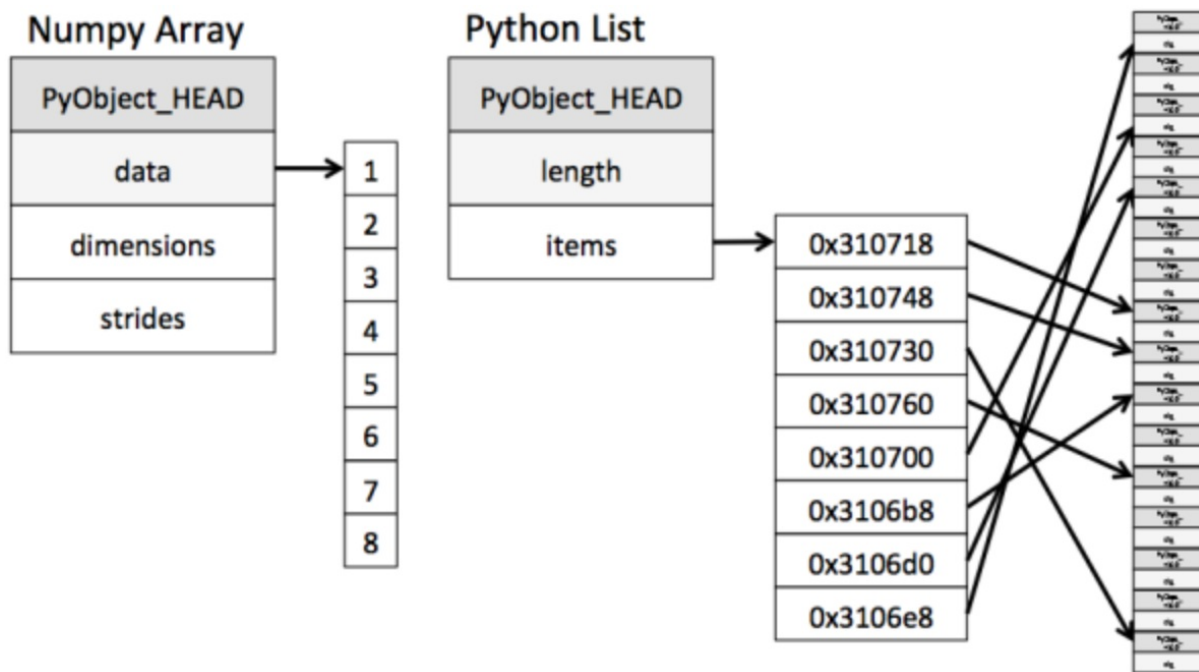
思考:

ndarray为什么可以这么快？

4 ndarray的优势

4.1 内存块风格

ndarray到底跟原生python列表有什么不同呢，请看一张图：



从图中我们可以看出ndarray在存储数据的时候，数据与数据的地址都是连续的，这样就使得批量操作数组元素时速度更快。

这是因为ndarray中的所有元素的类型都是相同的，而Python列表中的元素类型是任意的，所以ndarray在存储元素时内存可以连续，而python原生list就只能通过寻址方式找到下一个元素，这虽然也导致了在通用性能方面Numpy的ndarray不及Python原生list，但在科学计算中，Numpy的ndarray就可以省掉很多循环语句，代码使用方面比Python原生list简单的多。

4.2 ndarray支持并行化运算（向量化运算）

numpy内置了并行运算功能，当系统有多个核心时，做某种计算时，numpy会自动做并行计算

4.3 效率远高于纯Python代码

Numpy底层使用C语言编写，内部解除了GIL（全局解释器锁），其对数组的操作速度不受Python解释器的限制，所以，其效率远高于纯Python代码。

5 小结

- numpy介绍【了解】
 - 一个开源的Python科学计算库
 - 计算起来要比python简洁高效
 - Numpy使用ndarray对象来处理多维数组
- ndarray介绍【了解】
 - NumPy提供了一个N维数组类型ndarray，它描述了相同类型的“items”的集合。
 - 生成numpy对象:np.array()
- ndarray的优势【掌握】
 - 内存块风格
 - list -- 分离式存储,存储内容多样化
 - ndarray -- 一体式存储,存储类型必须一样
 - ndarray支持并行化运算（向量化运算）
 - ndarray底层是用C语言写的,效率更高,释放了GIL

4.2 N维数组-ndarray

学习目标

- 目标
 - 说明数组的属性，形状、类型

1 ndarray的属性

数组属性反映了数组本身固有的信息。

属性名字	属性解释
<code>ndarray.shape</code>	数组维度的元组
<code>ndarray.ndim</code>	数组维数
<code>ndarray.size</code>	数组中的元素数量
<code>ndarray.itemsize</code>	一个数组元素的长度（字节）
<code>ndarray.dtype</code>	数组元素的类型

2 ndarray的形状

首先创建一些数组。

```
# 创建不同形状的数组
>>> a = np.array([[1,2,3],[4,5,6]])
>>> b = np.array([1,2,3,4])
>>> c = np.array([[[1,2,3],[4,5,6]],[[1,2,3],[4,5,6]]])
```

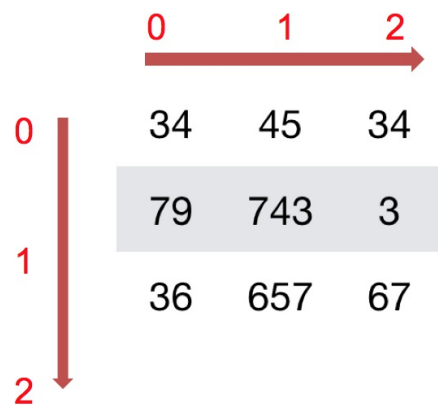
分别打印出形状

```
>>> a.shape
>>> b.shape
>>> c.shape

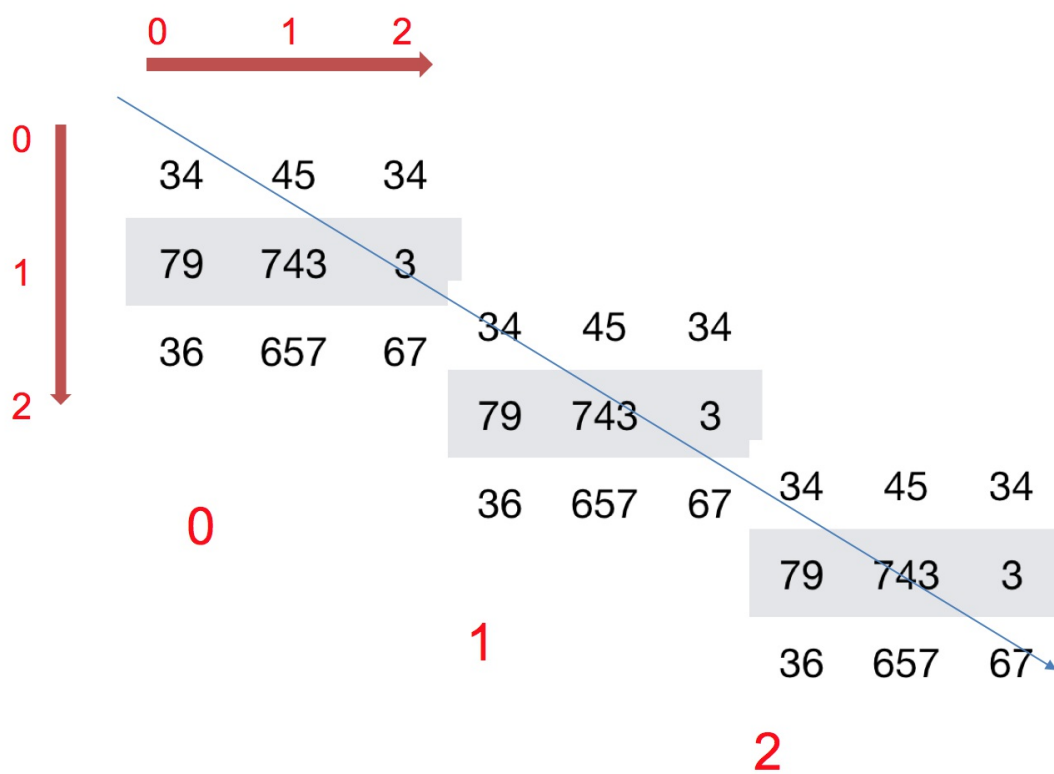
(2, 3) # 二维数组
(4,) # 一维数组
(2, 2, 3) # 三维数组
```

如何理解数组的形状？

二维数组：



三维数组:



3 ndarray的类型

```
>>> type(score.dtype)
<type 'numpy.dtype'>
```

dtype是numpy.dtype类型，先看看对于数组来说都有哪些类型

名称	描述	简写
np.bool	用一个字节存储的布尔类型（True或False）	'b'
np.int8	一个字节大小，-128 至 127	'i'
np.int16	整数，-32768 至 32767	'i2'
np.int32	整数，-2 ³¹ 至 2 ³² - 1	'i4'
np.int64	整数，-2 ⁶³ 至 2 ⁶³ - 1	'i8'
np.uint8	无符号整数，0 至 255	'u'
np.uint16	无符号整数，0 至 65535	'u2'
np.uint32	无符号整数，0 至 2 ³² - 1	'u4'
np.uint64	无符号整数，0 至 2 ⁶⁴ - 1	'u8'
np.float16	半精度浮点数：16位，正负号1位，指数5位，精度10位	'f2'
np.float32	单精度浮点数：32位，正负号1位，指数8位，精度23位	'f4'
np.float64	双精度浮点数：64位，正负号1位，指数11位，精度52位	'f8'
np.complex64	复数，分别用两个32位浮点数表示实部和虚部	'c8'
np.complex128	复数，分别用两个64位浮点数表示实部和虚部	'c16'
np.object_	python对象	'O'
np.string_	字符串	'S'
np.unicode_	unicode类型	'U'

创建数组的时候指定类型

```
>>> a = np.array([[1, 2, 3],[4, 5, 6]], dtype=np.float32)
>>> a.dtype
dtype('float32')

>>> arr = np.array(['python', 'tensorflow', 'scikit-learn', 'numpy'], dtype = np.string_)
>>> arr
array([b'python', b'tensorflow', b'scikit-learn', b'numpy'], dtype='<S12')

```

- 注意：若不指定，整数默认int64，小数默认float64

4 总结

数组的基本属性【知道】

属性名字	属性解释
ndarray.shape	数组维度的元组
ndarray.ndim	数组维数
ndarray.size	数组中的元素数量
ndarray.itemsize	一个数组元素的长度（字节）
ndarray.dtype	数组元素的类型

4.3 基本操作

学习目标

- 目标
 - 理解数组的各种生成方法
 - 应用数组的索引机制实现数组的切片获取
 - 应用维度变换实现数组的形状改变
 - 应用类型变换实现数组类型改变
 - 应用数组的转换

1 生成数组的方法

1.1 生成0和1的数组

- **np.ones(shape, dtype)**
- np.ones_like(a, dtype)
- **np.zeros(shape, dtype)**
- np.zeros_like(a, dtype)

```
ones = np.ones([4,8])
ones
```

返回结果:

```
array([[1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1.]])
```

```
np.zeros_like(ones)
```

返回结果:

```
array([[0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0.]])
```

1.2 从现有数组生成

1.2.1 生成方式

- **np.array(object, dtype)**
- **np.asarray(a, dtype)**

```
a = np.array([[1,2,3],[4,5,6]])
# 从现有的数组当中创建
a1 = np.array(a)
# 相当于索引的形式，并没有真正的创建一个新的
a2 = np.asarray(a)
```

1.2.2 关于array和asarray的不同

```
In [11]: a = np.array([[1, 2, 3], [4, 5, 6]])
```

```
In [12]: a
```

```
Out[12]: array([[1, 2, 3],
               [4, 5, 6]])
```

```
In [9]: a1 = np.array(a)
```

```
In [10]: a1
```

```
Out[10]: array([[1, 2, 3],
               [4, 5, 6]])
```

```
In [11]: a2 = np.asarray(a)
```

```
In [12]: a2
```

```
Out[12]: array([[1, 2, 3],
               [4, 5, 6]])
```

```
In [14]: a[0,0] = 100
```

```
In [15]: a
```

```
Out[15]: array([[100,  2,  3],
               [  4,  5,  6]])
```

```
In [16]: a1
```

```
Out[16]: array([[1, 2, 3],
               [4, 5, 6]])
```

```
In [17]: a2
```

```
Out[17]: array([[100,  2,  3],
               [  4,  5,  6]])
```

1.3 生成固定范围的数组

1.3.1 np.linspace(start, stop, num, endpoint)

- 创建等差数组 — 指定数量
- 参数:
 - start:序列的起始值
 - stop:序列的终止值
 - num:要生成的等间隔样例数量，默认为50
 - endpoint:序列中是否包含stop值，默认为ture

```
# 生成等间隔的数组
np.linspace(0, 100, 11)
```

返回结果:

```
array([ 0., 10., 20., 30., 40., 50., 60., 70., 80., 90., 100.])
```

1.3.2 np.arange(start,stop, step, dtype)

- 创建等差数组 — 指定步长
- 参数
 - step:步长,默认值为1

```
np.arange(10, 50, 2)
```

返回结果:

```
array([10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42,
       44, 46, 48])
```

1.3.3 np.logspace(start,stop, num)

- 创建等比数列
- 参数:
 - num:要生成的等比数列数量，默认为50

```
# 生成10^x
np.logspace(0, 2, 3)
```

返回结果:

```
array([ 1., 10., 100.])
```

1.4 生成随机数组

1.4.1 使用模块介绍

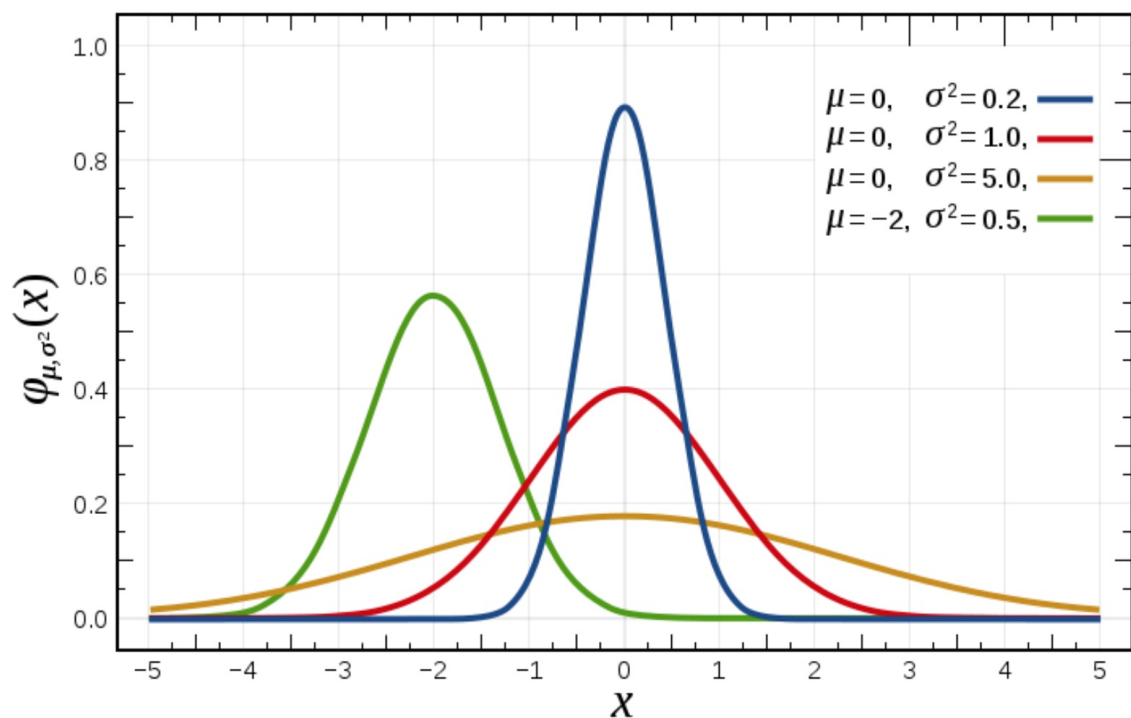
- np.random模块

1.4.2 正态分布

一、基础概念复习：正态分布（理解）

a. 什么是正态分布

正态分布是一种概率分布。正态分布是具有两个参数 μ 和 σ 的连续型随机变量的分布，第一参数 μ 是服从正态分布的随机变量的均值，第二个参数 σ 是此随机变量的标准差，所以正态分布记作 $N(\mu, \sigma)$ 。



b. 正态分布的应用

生活、生产与科学实验中很多随机变量的概率分布都可以近似地用正态分布来描述。

c. 正态分布特点

μ 决定了其位置，其标准差 σ 决定了分布的幅度。当 $\mu = 0, \sigma = 1$ 时的正态分布是标准正态分布。

标准差如何来？

- 方差

是在概率论和统计方差衡量一组数据时离散程度的度量

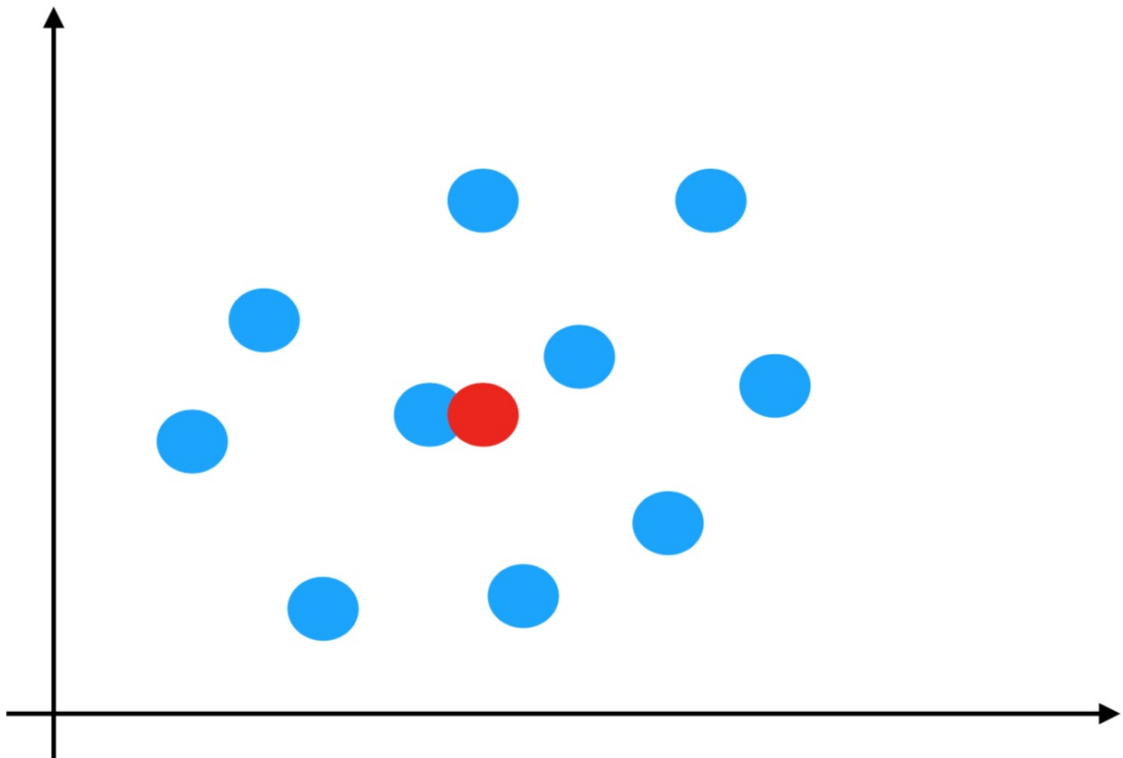
$$\sigma^2 = \frac{(x_1 - M)^2 + (x_2 - M)^2 + (x_3 - M)^2 + \dots + (x_n - M)^2}{n}$$

其中 M 为平均值， n 为数据总个数， σ 为标准差， σ^2 可以理解一个整体为方差

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

- 标准差与方差的意义

可以理解成数据的一个离散程度的衡量



二、正态分布创建方式

- `np.random.randn(d0, d1, ..., dn)`

功能：从标准正态分布中返回一个或多个样本值

- `np.random.normal(loc=0.0, scale=1.0, size=None)`

loc: float

此概率分布的均值（对应着整个分布的中心centre）

scale: float

此概率分布的标准差（对应于分布的宽度，scale越大越矮胖，scale越小，越瘦高）

size: int or tuple of ints

输出的shape，默认为None，只输出一个值

- `np.random.standard_normal(size=None)`

返回指定形状的标准正态分布的数组。

举例1：生成均值为**1.75**，标准差为**1**的正态分布数据，**100000000**个

```
x1 = np.random.normal(1.75, 1, 100000000)
```

返回结果：

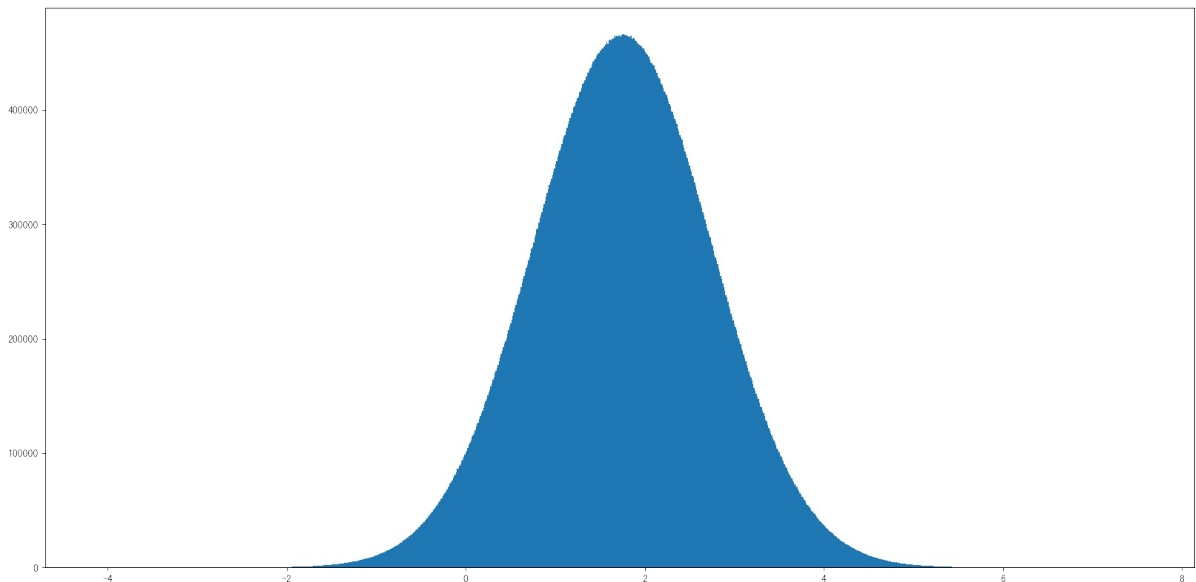
```
array([ 2.90646763,  1.46737886,  2.21799024, ...,  1.56047411,  1.87969135,
        0.9028096 ])
```

```
# 生成均匀分布的随机数
x1 = np.random.normal(1.75, 1, 100000000)

# 画图看分布状况
# 1) 创建画布
plt.figure(figsize=(20, 10), dpi=100)

# 2) 绘制直方图
plt.hist(x1, 1000)

# 3) 显示图像
plt.show()
```



例如：我们可以模拟生成一组股票的涨跌幅的数据

举例2：随机生成**4**支股票**1**周的交易日涨幅数据

4支股票，一周(**5**天)的涨跌幅数据，如何获取？

- 随机生成涨跌幅在某个正态分布内，比如均值0，方差1

股票涨跌幅数据的创建

```
# 创建符合正态分布的4只股票5天的涨跌幅数据
```

```
stock_change = np.random.normal(0, 1, (4, 5))
stock_change
```

返回结果:

```
array([[ 0.0476585 ,  0.32421568,  1.50062162,  0.48230497, -0.59998822],
       [-1.92160851,  2.20430374, -0.56996263, -1.44236548,  0.0165062 ],
       [-0.55710486, -0.18726488, -0.39972172,  0.08580347, -1.82842225],
       [-1.22384505, -0.33199305,  0.23308845, -1.20473702, -0.31753223]])
```

1.4.2 均匀分布

- `np.random.rand(d0, d1, ..., dn)`
 - 返回`[0.0, 1.0)`内的一组均匀分布的数。
- `np.random.uniform(low=0.0, high=1.0, size=None)`
 - 功能: 从一个均匀分布`[low,high)`中随机采样, 注意定义域是左闭右开, 即包含`low`, 不包含`high`。
 - 参数介绍:
 - `low`: 采样下界, `float`类型, 默认值为0;
 - `high`: 采样上界, `float`类型, 默认值为1;
 - `size`: 输出样本数目, 为`int`或元组(`tuple`)类型, 例如, `size=(m,n,k)`, 则输出`m*n*k`个样本, 缺省时输出1个值。
 - 返回值: `ndarray`类型, 其形状和参数`size`中描述一致。
- `np.random.randint(low, high=None, size=None, dtype='i')`
 - 从一个均匀分布中随机采样, 生成一个整数或N维整数数组,
 - 取数范围: 若`high`不为`None`时, 取`[low,high)`之间随机整数, 否则取值`[0,low)`之间随机整数。

```
# 生成均匀分布的随机数
x2 = np.random.uniform(-1, 1, 100000000)
```

返回结果:

```
array([ 0.22411206,  0.31414671,  0.85655613, ..., -0.92972446,
        0.95985223,  0.23197723])
```

画图看分布状况:

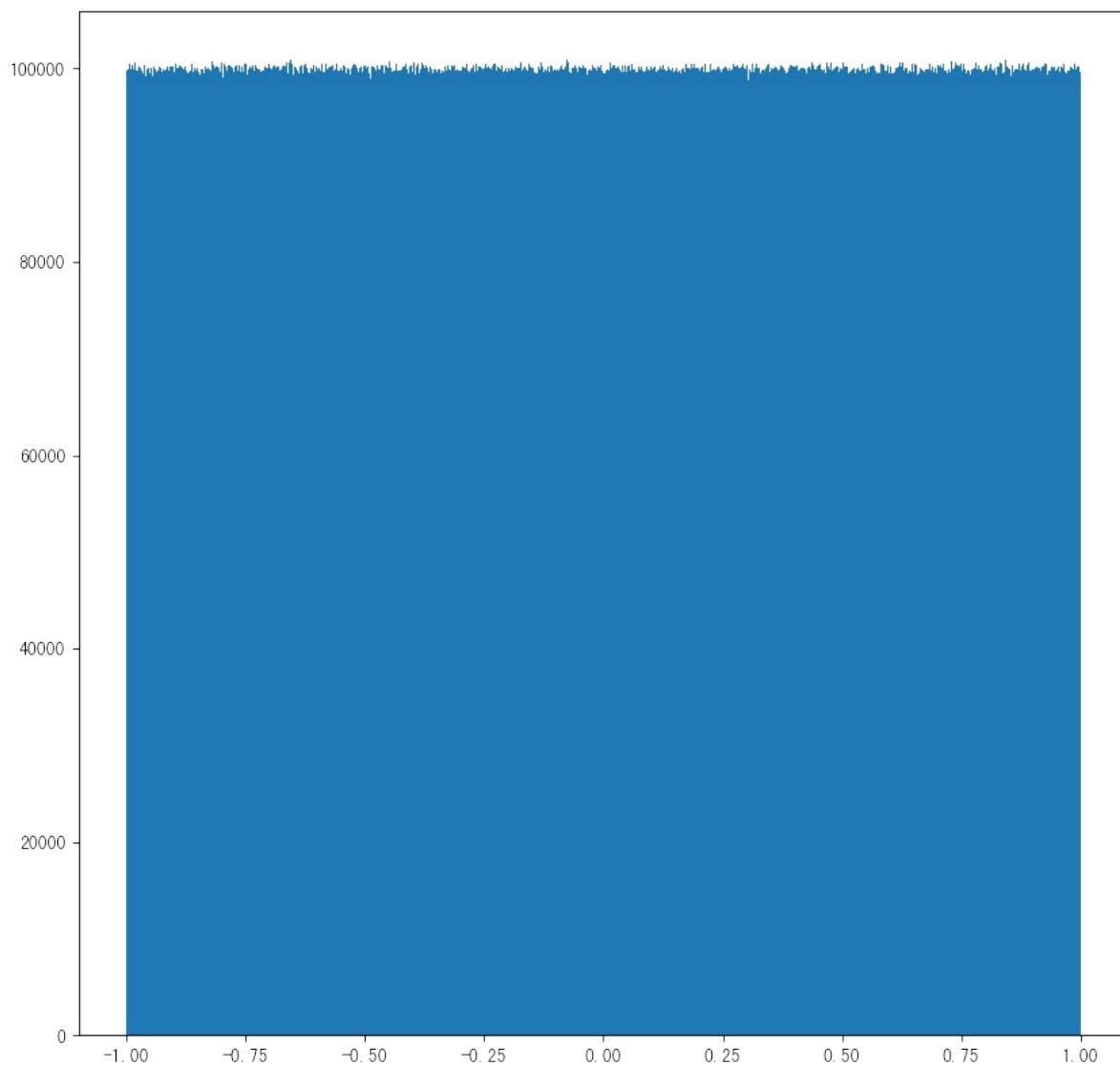
```
import matplotlib.pyplot as plt

# 生成均匀分布的随机数
x2 = np.random.uniform(-1, 1, 100000000)

# 画图看分布状况
# 1) 创建画布
plt.figure(figsize=(10, 10), dpi=100)

# 2) 绘制直方图
plt.hist(x=x2, bins=1000) # x代表要使用的数据, bins表示要划分区间数

# 3) 显示图像
plt.show()
```



2 数组的索引、切片

一维、二维、三维的数组如何索引？

- 直接进行索引,切片
- 对象[:, :] -- 先行后列

二维数组索引方式:

- 举例: 获取第一个股票的前3个交易日的涨跌幅数据

```
# 二维的数组, 两个维度
stock_change[0, 0:3]
```

返回结果:

```
array([-0.03862668, -1.46128096, -0.75596237])
```

- 三维数组索引方式:

```
# 三维
a1 = np.array([ [1,2,3],[4,5,6]], [[12,3,34],[5,6,7]])
# 返回结果
array([[[ 1,  2,  3],
        [ 4,  5,  6]],
```

```

[[12, 3, 34],
 [ 5, 6, 7]])
# 索引、切片
>>> a1[0, 0, 1] # 输出: 2

```

3 形状修改

3.1 ndarray.reshape(shape, order)

- 返回一个具有相同数据域，但shape不一样的视图
- 行、列不进行互换

```

# 在转换形状的时候，一定要注意数组的元素匹配
stock_change.reshape([5, 4])
stock_change.reshape([-1,10]) # 数组的形状被修改为: (2, 10), -1: 表示通过待计算

```

3.2 ndarray.resize(new_shape)

- 修改数组本身的形状（需要保持元素个数前后相同）
- 行、列不进行互换

```

stock_change.resize([5, 4])

# 查看修改后结果
stock_change.shape
(5, 4)

```

3.3 ndarray.T

- 数组的转置
- 将数组的行、列进行互换

```

stock_change.T.shape
(4, 5)

```

4 类型修改

4.1 ndarray.astype(type)

- 返回修改了类型之后的数组

```

stock_change.astype(np.int32)

```

4.2 ndarray.tostring([order])或者ndarray.tobytes([order])

- 构造包含数组中原始数据字节的Python字节

```

arr = np.array([[[[1, 2, 3], [4, 5, 6]], [[12, 3, 34], [5, 6, 7]]])
arr.tostring()

```

4.3 jupyter输出太大可能导致崩溃问题【了解】

如果遇到

```

IOPub data rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub_data_rate_limit`.

```

这个问题是在jupyter当中对输出的字节数有限制，需要去修改配置文件

创建配置文件

```
jupyter notebook --generate-config
vi ~/.jupyter/jupyter_notebook_config.py
```

取消注释,多增加

```
## (bytes/sec) Maximum rate at which messages can be sent on iopub before they
# are limited.
c.NotebookApp.iopub_data_rate_limit = 10000000
```

但是不建议这样去修改, **jupyter** 输出太大会崩溃

5 数组的去重

5.1 np.unique()

```
temp = np.array([[1, 2, 3, 4], [3, 4, 5, 6]])
>>> np.unique(temp)
array([1, 2, 3, 4, 5, 6])
```

6 小结

- 创建数组【掌握】
 - 生成0和1的数组
 - np.ones()
 - np.ones_like()
 - 从现有数组中生成
 - np.array -- 深拷贝
 - np.asarray -- 浅拷贝
 - 生成固定范围数组
 - np.linspace()
 - num -- 生成等间隔的多少个
 - np.arange()
 - step -- 每间隔多少生成数据
 - np.logspace()
 - 生成以10的N次幂的数据
 - 生层随机数组
 - 正态分布
 - 里面需要关注的参数:均值:u, 标准差:σ
 - u -- 决定了这个图形的左右位置
 - σ -- 决定了这个图形是瘦高还是矮胖
 - np.random.randn()
 - np.random.normal(0, 1, 100)
 - 均匀
 - np.random.rand()
 - np.random.uniform(0, 1, 100)
 - np.random.randint(0, 10, 10)
- 数组索引【知道】
 - 直接进行索引,切片
 - 对象[:, :] -- 先行后列
- 数组形状改变【掌握】
 - 对象.reshape()
 - 没有进行行列互换,新产生一个ndarray
 - 对象.resize()
 - 没有进行行列互换,修改原来的ndarray
 - 对象.T

- 进行了行列互换
- 数组去重【知道】
 - np.unique(对象)

4.4 ndarray运算

学习目标

- 目标
 - 应用数组的通用判断函数
 - 应用np.where实现数组的三元运算

问题

如果想要操作符合某一条件的数据，应该怎么做？

1 逻辑运算

```
# 生成10名同学，5门功课的数据
>>> score = np.random.randint(40, 100, (10, 5))

# 取出最后4名同学的成绩，用于逻辑判断
>>> test_score = score[6:, 0:5]

# 逻辑判断，如果成绩大于60就标记为True 否则为False
>>> test_score > 60
array([[ True,  True,  True, False,  True],
       [ True,  True,  True, False,  True],
       [ True,  True, False, False,  True],
       [False,  True,  True,  True,  True]])

# BOOL赋值，将满足条件的设置为指定的值-布尔索引
>>> test_score[test_score > 60] = 1
>>> test_score
array([[ 1,  1,  1, 52,  1],
       [ 1,  1,  1, 59,  1],
       [ 1,  1, 44, 44,  1],
       [59,  1,  1,  1,  1]])
```

2 通用判断函数

- np.all()

```
# 判断前两名同学的成绩[0:2, :]是否全及格
>>> np.all(score[0:2, :] > 60)
False
```

- np.any()

```
# 判断前两名同学的成绩[0:2, :]是否有大于90分的
>>> np.any(score[0:2, :] > 80)
True
```

3 np.where（三元运算符）

通过使用np.where能够进行更加复杂的运算

- np.where()

```
# 判断前四名同学，前四门课程中，成绩中大于60的置为1，否则为0
temp = score[:4, :4]
np.where(temp > 60, 1, 0)
```

- 复合逻辑需要结合np.logical_and和np.logical_or使用

```
# 判断前四名学生,前四门课程中,成绩中大于60且小于90的换为1,否则为0
np.where(np.logical_and(temp > 60, temp < 90), 1, 0)

# 判断前四名学生,前四门课程中,成绩中大于90或小于60的换为1,否则为0
np.where(np.logical_or(temp > 90, temp < 60), 1, 0)
```

4 统计运算

如果想要知道学生成绩最大的分数,或者做小分数应该怎么做?

4.1 统计指标

在数据挖掘/机器学习领域,统计指标的值也是我们分析问题的一种方式。常用的指标如下:

- `min(a, axis)`
 - Return the minimum of an array or minimum along an axis.
- `max(a, axis)`
 - Return the maximum of an array or maximum along an axis.
- `median(a, axis)`
 - Compute the median along the specified axis.
- `mean(a, axis, dtype)`
 - Compute the arithmetic mean along the specified axis.
- `std(a, axis, dtype)`
 - Compute the standard deviation along the specified axis.
- `var(a, axis, dtype)`
 - Compute the variance along the specified axis.

4.2 案例: 学生成绩统计运算

进行统计的时候, **axis** 轴的取值并不一定, **Numpy**中不同的**API**轴的值都不一样, 在这里, **axis 0**代表列, **axis 1**代表行去进行统计

```
# 接下来对于前四名学生,进行一些统计运算
# 指定列 去统计
temp = score[:, 4, 0:5]
print("前四名学生,各科成绩的最大分: {}".format(np.max(temp, axis=0)))
print("前四名学生,各科成绩的最小分: {}".format(np.min(temp, axis=0)))
print("前四名学生,各科成绩波动情况: {}".format(np.std(temp, axis=0)))
print("前四名学生,各科成绩的平均分: {}".format(np.mean(temp, axis=0)))
```

结果:

```
前四名学生,各科成绩的最大分: [96 97 72 98 89]
前四名学生,各科成绩的最小分: [55 57 45 76 77]
前四名学生,各科成绩波动情况: [16.25576821 14.92271758 10.40432602 8.0311892 4.32290412]
前四名学生,各科成绩的平均分: [78.5 75.75 62.5 85. 82.25]
```

如果需要统计出某科最高分对应的是哪个同学?

- `np.argmax(temp, axis=)`
- `np.argmin(temp, axis=)`

```
print("前四名学生,各科成绩最高分对应的学生下标: {}".format(np.argmax(temp, axis=0)))
```

结果:

```
前四名学生,各科成绩最高分对应的学生下标: [0 2 0 0 1]
```

5 小结

- 逻辑运算【知道】
 - 直接进行大于,小于的判断
 - 合适之后,可以直接进行赋值
- 通用判断函数【知道】

- np.all()
- np.any()
- 统计运算【掌握】
 - np.max()
 - np.min()
 - np.median()
 - np.mean()
 - np.std()
 - np.var()
 - np.argmax(axis=) — 最大元素对应的下标
 - np.argmin(axis=) — 最小元素对应的下标

4.5 数组间运算

学习目标

- 目标
 - 知道数组与数之间的运算
 - 知道数组与数组之间的运算
 - 说明数组间运算的广播机制

1 数组与数的运算

```
arr = np.array([[1, 2, 3, 2, 1, 4], [5, 6, 1, 2, 3, 1]])
arr + 1
arr / 2

# 可以对比python列表的运算，看出区别
a = [1, 2, 3, 4, 5]
a * 3
```

2 数组与数组的运算

```
arr1 = np.array([[1, 2, 3, 2, 1, 4], [5, 6, 1, 2, 3, 1]])
arr2 = np.array([[1, 2, 3, 4], [3, 4, 5, 6]])
```

上面这个能进行运算吗，结果是不行的！

2.1 广播机制

数组在进行矢量化运算时，要求数组的形状是相等的。当形状不相等的数组执行算术运算的时候，就会出现广播机制，该机制会对数组进行扩展，使数组的`shape`属性值一样，这样，就可以进行矢量化运算了。下面通过一个例子进行说明：

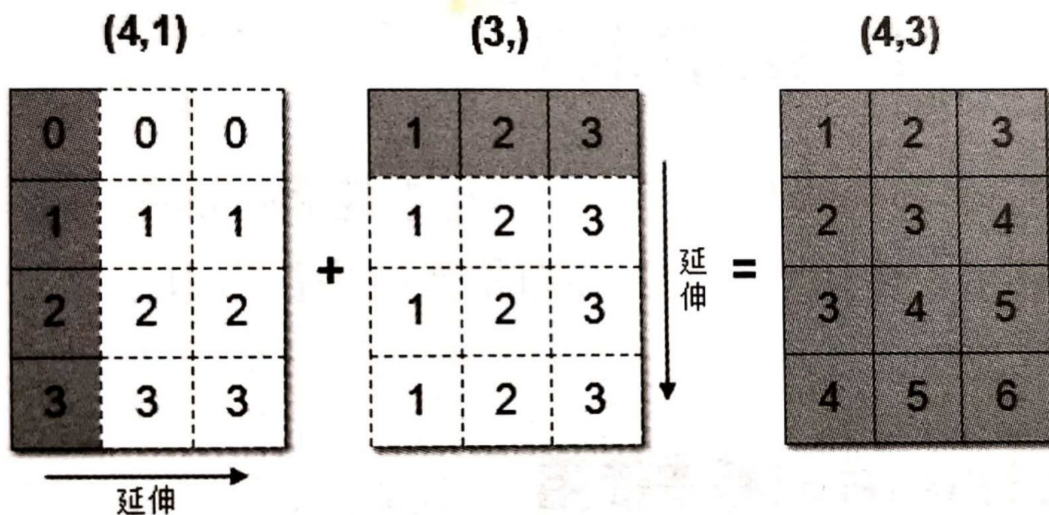
```
arr1 = np.array([[0],[1],[2],[3]])
arr1.shape
# (4, 1)

arr2 = np.array([1,2,3])
arr2.shape
# (3, )

arr1+arr2
# 结果是：
array([[1, 2, 3],
       [2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])
```

上述代码中，数组`arr1`是4行1列，`arr2`是1行3列。这两个数组要进行相加，按照广播机制会对数组`arr1`和`arr2`都进行扩展，使得数组`arr1`和`arr2`都变成4行3列。

下面通过一张图来描述广播机制扩展数组的过程：



这句话乃是理解广播的核心。广播主要发生在两种情况，一种是两个数组的维数不相等，但是它们的后缘维度的轴长相符，另外一种是有的一方的长度为1。

广播机制实现了两个或两个以上数组的运算，即使这些数组的shape不是完全相同的，只需要满足如下任意一个条件即可。

- 如果两个数组的后缘维度（**trailing dimension**，即从末尾开始算起的维度）的轴长度相符，
- 或其中的一方的长度为1。

广播会在缺失和（或）长度为1的维度上进行。

广播机制需要扩展维度小的数组，使得它与维度最大的数组的shape值相同，以便使用元素级函数或者运算符进行运算。

如果是下面这样，则不匹配：

```
A (1d array): 10
B (1d array): 12
A (2d array): 2 x 1
B (3d array): 8 x 4 x 3
```

思考：下面两个ndarray是否能够进行运算？

```
arr1 = np.array([[1, 2, 3, 2, 1, 4], [5, 6, 1, 2, 3, 1]])
arr2 = np.array([[1], [3]])
```

3 小结

- 数组运算,满足广播机制,就OK【知道】
 - 1.维度相等
 - 2.shape(其中对应的地方为1,也是可以的)

4.6 数学：矩阵

学习目标

- 目标
 - 知道什么是矩阵和向量
 - 知道矩阵的加法,乘法
 - 知道矩阵的逆和转置
 - 应用`np.matmul`、`np.dot`实现矩阵运算
-

1 矩阵和向量

1.1 矩阵

矩阵，英文**matrix**，和**array**的区别矩阵必须是**2**维的，但是**array**可以是多维的。

如图:这个是 **3×2** 矩阵，即 3 行 2 列，如 m 为行，n 为列，那么 m×n 即 **3×2**

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

矩阵的维数即行数×列数

矩阵元素(矩阵项):

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

A_{ij} 指第 i 行，第 j 列的元素。

1.2 向量

向量是一种特殊的矩阵，讲义中的向量一般都是列向量，下面展示的就是三维列 向量(**3×1**)。)

$$A = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

2 加法和标量乘法

矩阵的加法:行列数相等的可以加。

例:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \\ 10 & 12 \end{bmatrix}$$

矩阵的乘法:每个元素都要乘。

例:

$$3 * \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \\ 15 & 18 \end{bmatrix}$$

组合算法也类似。

3 矩阵向量乘法

矩阵和向量的乘法如图：**m×n** 的矩阵乘以 **n×1** 的向量，得到的是 **m×1** 的向量

例:

$$\begin{bmatrix} 1 & 3 \\ 4 & 0 \\ 2 & 1 \end{bmatrix} * \begin{bmatrix} 1 \\ 5 \end{bmatrix} = \begin{bmatrix} 16 \\ 4 \\ 7 \end{bmatrix}$$

```
1*1+3*5 = 16
4*1+0*5 = 4
2*1+1*5 = 7
```

矩阵乘法遵循准则：

$$(M \text{行}, N \text{列}) * (N \text{行}, L \text{列}) = (M \text{行}, L \text{列})$$

4 矩阵乘法

矩阵乘法：

$m \times n$ 矩阵乘以 $n \times o$ 矩阵，变成 $m \times o$ 矩阵。

举例：比如说现在有两个矩阵 A 和 B，那么它们的乘积就可以表示为图中所示的形式。

$$\begin{matrix} C & = & A & \times & B \\ \begin{pmatrix} C0 & C1 \\ C2 & C3 \end{pmatrix} & = & \begin{pmatrix} A0 & A1 \\ A2 & A3 \end{pmatrix} & \times & \begin{pmatrix} B0 & B1 \\ B2 & B3 \end{pmatrix} \\ C0 = A0 \times B0 + A1 \times B2 \\ C1 = A0 \times B1 + A1 \times B3 \\ C2 = A2 \times B0 + A3 \times B2 \\ C3 = A2 \times B1 + A3 \times B3 \end{matrix}$$

• 练一练

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix} B = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix}$$

求矩阵AB的结果

答案：

$$B = \begin{bmatrix} 1*1+2*1+3*2 & 1*2+2*1+3*1 & 1*1+2*2+3*1 \\ 4*1+5*1+6*2 & 4*2+5*1+6*1 & 4*1+5*2+6*1 \\ 7*1+8*1+0*2 & 7*2+8*1+0*1 & 7*1+8*2+0*1 \end{bmatrix} = \begin{bmatrix} 9 & 7 & 8 \\ 21 & 19 & 20 \\ 15 & 22 & 23 \end{bmatrix}$$

5 矩阵乘法的性质

矩阵的乘法不满足交换律： $A \times B \neq B \times A$

矩阵的乘法满足结合律。即： $A \times (B \times C) = (A \times B) \times C$

单位矩阵：在矩阵的乘法中，有一种矩阵起着特殊的作用，如同数的乘法中的 1，我们称这种矩阵为单位矩阵。它是个方阵，一般用 I 或者 E 表示，从左上角到右下角的对角线（称为主对角线）上的元素均为 1 以外全都为 0。如：

$$\begin{matrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ 2 \times 2 \end{matrix}, \quad \begin{matrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ 3 \times 3 \end{matrix}, \quad \begin{matrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ 4 \times 4 \end{matrix}$$

6 逆、转置

矩阵的逆：如矩阵 A 是一个 $m \times m$ 矩阵（方阵），如果有逆矩阵，则：

$$AA^{-1} = A^{-1}A = I$$

低阶矩阵求逆的方法:

1.待定系数法

2.初等变换

矩阵的转置: 设 A 为 $m \times n$ 阶矩阵 (即 m 行 n 列), 第 i 行 j 列的元素是 $a(i,j)$, 即:

$$A = a(i,j)$$

定义 A 的转置为这样一个 $n \times m$ 阶矩阵 B , 满足 $B = a(j,i)$, 即 $b(i,j) = a(j,i)$ (B 的第 i 行第 j 列元素是 A 的第 j 行第 i 列元素), 记 $A^T = B$ 。

直观来看, 将 A 的所有元素绕着一条从第 1 行第 1 列元素出发的右下方 45 度的射线作 镜面反转, 即得到 A 的转置。

例:

$$\begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix}^T = \begin{bmatrix} a & c & e \\ b & d & f \end{bmatrix}$$

7 矩阵运算

学生	平时成绩	期末成绩		最终成绩
1	80	86	?	84.2
2	82	80		80.6
3	85	78		80.1
4	90	90		90
5	86	82		83.2
6	82	90		87.6
7	78	80		79.4
8	92	94		93.4

$$\begin{bmatrix} 80 & 86 \\ 82 & 80 \\ 85 & 78 \\ 90 & 90 \\ 86 & 82 \\ 82 & 90 \\ 78 & 80 \\ 92 & 94 \end{bmatrix} * \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix} = \begin{bmatrix} 84.2 \\ 80.6 \\ 80.1 \\ 90 \\ 83.2 \\ 87.6 \\ 79.4 \\ 93.4 \end{bmatrix}$$

7.1 矩阵乘法api:

- `np.matmul`
- `np.dot`

```
>>> a = np.array([[80, 86],
[82, 80],
[85, 78],
[90, 90],
[86, 82],
[82, 90],
[78, 80],
[92, 94]])
>>> b = np.array([[0.7], [0.3]])
```

```
>>> np.matmul(a, b)
array([[81.8],
[81.4],
[82.9],
[90. ],
[84.8],
[84.4],
[78.6],
[92.6]])
```

```
>>> np.dot(a,b)
array([[81.8],
[81.4],
[82.9],
[90. ],
[84.8],
[84.4],
[78.6],
[92.6]])
```

np.matmul和**np.dot**的区别:

二者都是矩阵乘法。**np.matmul**中禁止矩阵与标量的乘法。在矢量乘矢量的内积运算中，**np.matmul**与**np.dot**没有区别。

7 小结

- 1.矩阵和向量【知道】
 - 矩阵就是特殊的二维数组
 - 向量就是一行或者一列的数据
- 2.矩阵加法和标量乘法【知道】
 - 矩阵的加法:行列数相等的可以加。
 - 矩阵的乘法:每个元素都要乘。
- 3.矩阵和矩阵(向量)相乘 【知道】
 - $(M\text{行}, N\text{列}) * (N\text{行}, L\text{列}) = (M\text{行}, L\text{列})$
- 4.矩阵性质【知道】
 - 矩阵不满足交换率,满足结合律
- 5.单位矩阵【知道】
 - 对角线都是1的矩阵,其他位置都为0
- 6.矩阵运算【掌握】
 - **np.matmul**
 - **np.dot**
 - 注意:二者都是矩阵乘法。**np.matmul**中禁止矩阵与标量的乘法。在矢量乘矢量的内积运算中，**np.matmul**与**np.dot**没有区别。

Pandas

学习目标

- 了解Numpy与Pandas的不同
- 说明Pandas的Series与Dataframe两种结构的区别
- 了解Pandas的MultiIndex与panel结构
- 应用Pandas实现基本数据操作
- 应用Pandas实现数据的合并
- 应用crosstab和pivot_table实现交叉表与透视表
- 应用groupby和聚合函数实现数据的分组与聚合
- 了解Pandas的plot画图功能
- 应用Pandas实现数据的读取和存储

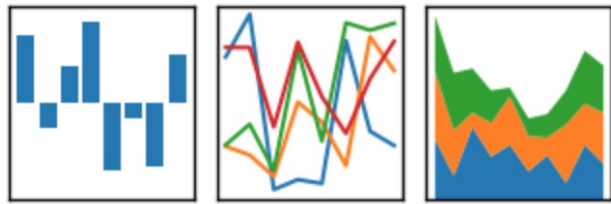
5.1Pandas介绍

学习目标

- 目标
 - 了解什么是pandas
 - 了解Numpy与Pandas的不同
 - 知道使用pandas的优势

1 Pandas介绍

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$


- 2008年WesMcKinney开发出的库
- 专门用于数据挖掘的开源python库
- 以Numpy为基础，借力Numpy模块在计算方面性能高的优势
- 基于matplotlib，能够简便的画图
- 独特的数据结构

2 为什么使用Pandas

Numpy已经能够帮助我们处理数据，能够结合matplotlib解决部分数据展示等问题，那么pandas学习的目的在什么地方呢？

- 增强图表可读性
 - 回忆我们在numpy当中创建学生成绩表样式：
 - 返回结果：

```
array([[92, 55, 78, 50, 50],
       [71, 76, 50, 48, 96],
       [45, 84, 78, 51, 68],
       [81, 91, 56, 54, 76],
       [86, 66, 77, 67, 95],
       [46, 86, 56, 61, 99],
       [46, 95, 44, 46, 56],
       [80, 50, 45, 65, 57],
       [41, 93, 90, 41, 97],
       [65, 83, 57, 57, 40]])
```

如果数据展示为这样，可读性就会更友好：

```
array([[92, 55, 78, 50, 50],
       [71, 76, 50, 48, 96],
       [45, 84, 78, 51, 68],
       [81, 91, 56, 54, 76],
       [86, 66, 77, 67, 95],
       [46, 86, 56, 61, 99],
       [46, 95, 44, 46, 56],
       [80, 50, 45, 65, 57],
       [41, 93, 90, 41, 97],
       [65, 83, 57, 57, 40]])
```

	语文	数学	英语	政治	体育
同学0	92	55	78	50	50
同学1	71	76	50	48	96
同学2	45	84	78	51	68
同学3	81	91	56	54	76
同学4	86	66	77	67	95
同学5	46	86	56	61	99
同学6	46	95	44	46	56
同学7	80	50	45	65	57
同学8	41	93	90	41	97
同学9	65	83	57	57	40

- 便捷的数据处理能力

表格一:

23	24	Trolls	Animation,Adventure,Comedy	Bergens invade Troll Village, Poppy...	Walt Dohrn	Anna Kendrick, Justin Timberlake,Zoey Deschan...	2016	92	6.5	38552	153.69	56.0
24	25	Independence Day: Resurgence	Action,Adventure,Sci-Fi	Two decades after the first Independence Day l...	Roland Emmerich	Liam Hemsworth, Jeff Goldblum, Bill Pullman, Ma...	2016	120	5.3	127553	103.14	32.0
25	26	Paris pieds nus	Comedy	Fiona visits Paris for the first time to assis...	Dominique Abel	Fiona Gordon, Dominique Abel,Emmanuelle Riva, ...	2016	83	6.8	222	NaN	NaN
26	27	Bahubali: The Beginning	Action,Adventure,Drama	In ancient India, an adventurous and daring ma...	S.S. Rajamouli	Prabhas, Rana Daggubati, Anushka Shetty,Tamann...	2015	159	8.3	76193	6.50	NaN
27	28	Dead Awake	Horror,Thriller	A young woman must save herself and her friend...	Phillip Guzman	Jocelin Donahue, Jesse Bradford, Jesse Borrego...	2016	99	4.7	523	0.01	NaN

表格二:

23	24	Trolls	Animation,Adventure,Comedy	Bergens invade Troll Village, Poppy...	Walt Dohrn	Anna Kendrick, Justin Timberlake,Zoey Deschan...	2016	92	6.5	38552	153.690000	56.000000
24	25	Independence Day: Resurgence	Action,Adventure,Sci-Fi	Two decades after the first Independence Day l...	Roland Emmerich	Liam Hemsworth, Jeff Goldblum, Bill Pullman, Ma...	2016	120	5.3	127553	103.140000	32.000000
25	26	Paris pieds nus	Comedy	Fiona visits Paris for the first time to assis...	Dominique Abel	Fiona Gordon, Dominique Abel,Emmanuelle Riva, ...	2016	83	6.8	222	82.956376	58.985043
26	27	Bahubali: The Beginning	Action,Adventure,Drama	In ancient India, an adventurous and daring ma...	S.S. Rajamouli	Prabhas, Rana Daggubati, Anushka Shetty,Tamann...	2015	159	8.3	76193	6.500000	58.985043
27	28	Dead Awake	Horror,Thriller	A young woman must save herself	Phillip Guzman	Jocelin Donahue, Jesse Bradford, Jesse Borrego...	2016	99	4.7	523	0.010000	58.985043

- 读取文件方便
- 封装了Matplotlib、Numpy的画图和计算

3 小结

- pandas的优势【了解】
 - 增强图表可读性
 - 便捷的数据处理能力
 - 读取文件方便
 - 封装了Matplotlib、Numpy的画图和计算

5.2 Pandas数据结构

学习目标

- 目标
 - 知道Pandas的Series结构
 - 掌握Pandas的Dataframe结构
 - 了解Pandas的MultiIndex与panel结构

Pandas中一共有三种数据结构，分别为：Series、DataFrame和MultiIndex（老版本中叫Panel）。

其中Series是一维数据结构，DataFrame是二维的表格型数据结构，MultiIndex是三维的数据结构。

1.Series

Series是一个类似于一维数组的数据结构，它能够保存任何类型的数据，比如整数、字符串、浮点数等，主要由一组数据和与之相关的索引两部分构成。

Series	
index	value
0	12
1	4
2	7
3	9

1.1 Series的创建

```
# 导入pandas
import pandas as pd

pd.Series(data=None, index=None, dtype=None)
```

- 参数：
 - data: 传入的数据，可以是ndarray、list等
 - index: 索引，必须是唯一的，且与数据的长度相等。如果没有传入索引参数，则默认会自动创建一个从0-N的整数索引。
 - dtype: 数据的类型

通过已有数据创建

- 指定内容，默认索引

```
pd.Series(np.arange(10))
```

```
# 运行结果
0    0
1    1
2    2
3    3
4    4
5    5
6    6
7    7
8    8
9    9
dtype: int64
```

- 指定索引

```
pd.Series([6.7,5.6,3,10,2], index=[1,2,3,4,5])
```

```
# 运行结果
1    6.7
2    5.6
3    3.0
4   10.0
5    2.0
dtype: float64
```

- 通过字典数据创建

```
color_count = pd.Series({'red':100, 'blue':200, 'green': 500, 'yellow':1000})
color_count
```

```
# 运行结果
blue      200
green     500
red       100
yellow   1000
dtype: int64
```

1.2 Series的属性

为了更方便地操作Series对象中的索引和数据，**Series**中提供了两个属性**index**和**values**

- index

```
color_count.index

# 结果
Index(['blue', 'green', 'red', 'yellow'], dtype='object')
```

- values

```
color_count.values

# 结果
array([ 200,  500,  100, 1000])
```

也可以使用索引来获取数据：

```
color_count[2]

# 结果
100
```

2.DataFrame

DataFrame是一个类似于二维数组或表格(如excel)的对象，既有行索引，又有列索引

- 行索引，表明不同行，纵向索引，叫index，0轴，axis=0

- 列索引，表名不同列，纵向索引，叫columns，1轴，axis=1

DataFrame			
index	writer	title	price
0	mark	cookbook	23.56
1	barket	HTML5	50.70
2	tom	Python	12.30
3	job	Numpy	28.00

2.1 DataFrame的创建

```
# 导入pandas
import pandas as pd

pd.DataFrame(data=None, index=None, columns=None)
```

- 参数：
 - index: 行标签。如果没有传入索引参数，则默认会自动创建一个从0-N的整数索引。
 - columns: 列标签。如果没有传入索引参数，则默认会自动创建一个从0-N的整数索引。
- 通过已有数据创建

举例一：

```
pd.DataFrame(np.random.randn(2,3))
```

	0	1	2
0	-0.834694	-0.147477	0.364357
1	-0.703061	-0.171343	-0.346670

回忆咱们在前面直接使用np创建的数组显示方式，比较两者的区别。

举例二：创建学生成绩表

```
# 生成10名同学，5门功课的数据
score = np.random.randint(40, 100, (10, 5))

# 结果
array([[92, 55, 78, 50, 50],
       [71, 76, 50, 48, 96],
       [45, 84, 78, 51, 68],
       [81, 91, 56, 54, 76],
       [86, 66, 77, 67, 95],
       [46, 86, 56, 61, 99],
       [46, 95, 44, 46, 56],
       [80, 50, 45, 65, 57],
       [41, 93, 90, 41, 97],
       [65, 83, 57, 57, 40]])
```

但是这样的数据形式很难看到存储的是什么样的数据，可读性比较差！！

问题：如何让数据更有意义的显示？

```
# 使用Pandas中的数据结构
score_df = pd.DataFrame(score)
```

	0	1	2	3	4
0	92	55	78	50	50
1	71	76	50	48	96
2	45	84	78	51	68
3	81	91	56	54	76
4	86	66	77	67	95
5	46	86	56	61	99
6	46	95	44	46	56
7	80	50	45	65	57
8	41	93	90	41	97
9	65	83	57	57	40

给分数数据增加行列索引,显示效果更佳

效果:

	语文	数学	英语	政治	体育
同学0	92	55	78	50	50
同学1	71	76	50	48	96
同学2	45	84	78	51	68
同学3	81	91	56	54	76
同学4	86	66	77	67	95
同学5	46	86	56	61	99
同学6	46	95	44	46	56
同学7	80	50	45	65	57
同学8	41	93	90	41	97
同学9	65	83	57	57	40

- 增加行、列索引

```
# 构造行索引序列
subjects = ["语文", "数学", "英语", "政治", "体育"]

# 构造列索引序列
stu = ['同学' + str(i) for i in range(score_df.shape[0])]

# 添加行索引
data = pd.DataFrame(score, columns=subjects, index=stu)
```

2.2 DataFrame的属性

- shape

```
data.shape

# 结果
(10, 5)
```

- index

DataFrame的行索引列表

```
data.index

# 结果
```

```
Index(['同学0', '同学1', '同学2', '同学3', '同学4', '同学5', '同学6', '同学7', '同学8', '同学9'], dtype='object')
```

- **columns**

DataFrame的列索引列表

```
data.columns  
  
# 结果  
Index(['语文', '数学', '英语', '政治', '体育'], dtype='object')
```

- **values**

直接获取其中array的值

```
data.values  
  
array([[92, 55, 78, 50, 50],  
       [71, 76, 50, 48, 96],  
       [45, 84, 78, 51, 68],  
       [81, 91, 56, 54, 76],  
       [86, 66, 77, 67, 95],  
       [46, 86, 56, 61, 99],  
       [46, 95, 44, 46, 56],  
       [80, 50, 45, 65, 57],  
       [41, 93, 90, 41, 97],  
       [65, 83, 57, 57, 40]])
```

- **T**

转置

```
data.T
```

结果

	同学0	同学1	同学2	同学3	同学4	同学5	同学6	同学7	同学8	同学9
语文	92	71	45	81	86	46	46	80	41	65
数学	55	76	84	91	66	86	95	50	93	83
英语	78	50	78	56	77	56	44	45	90	57
政治	50	48	51	54	67	61	46	65	41	57
体育	50	96	68	76	95	99	56	57	97	40

- **head(5)**: 显示前5行内容

如果不补充参数，默认5行。填入参数N则显示前N行

```
data.head(5)
```

	语文	数学	英语	政治	体育
同学0	92	55	78	50	50
同学1	71	76	50	48	96
同学2	45	84	78	51	68
同学3	81	91	56	54	76
同学4	86	66	77	67	95

- **tail(5)**: 显示后5行内容

如果不补充参数，默认5行。填入参数N则显示后N行

```
data.tail(5)
```

2.3 DataFrame索引的设置

需求：

	语文	数学	英语	政治	体育
学生_0	92	55	78	50	50
学生_1	71	76	50	48	96
学生_2	45	84	78	51	68
学生_3	81	91	56	54	76
学生_4	86	66	77	67	95
学生_5	46	86	56	61	99
学生_6	46	95	44	46	56
学生_7	80	50	45	65	57
学生_8	41	93	90	41	97
学生_9	65	83	57	57	40

2.3.1 修改行列索引值

```
stu = ["学生_" + str(i) for i in range(score_df.shape[0])]

# 必须整体全部修改
data.index = stu
```

注意：以下修改方式是错误的

```
# 错误修改方式
data.index[3] = '学生_3'
```

2.3.2 重设索引

- `reset_index(drop=False)`
 - 设置新的下标索引
 - `drop`:默认为`False`，不删除原来索引，如果为`True`,删除原来的索引值

```
# 重置索引,drop=False
data.reset_index()
```

	index	语文	数学	英语	政治	体育
0	学生_0	92	55	78	50	50
1	学生_1	71	76	50	48	96
2	学生_2	45	84	78	51	68
3	学生_3	81	91	56	54	76
4	学生_4	86	66	77	67	95
5	学生_5	46	86	56	61	99
6	学生_6	46	95	44	46	56
7	学生_7	80	50	45	65	57
8	学生_8	41	93	90	41	97
9	学生_9	65	83	57	57	40

```
# 重置索引,drop=True
data.reset_index(drop=True)
```

2.3.3 以某列值设置为新的索引

- `set_index(keys, drop=True)`
 - **keys**: 列索引名成或者列索引名称的列表
 - **drop**: boolean, default True.当做新的索引，删除原来的列

设置新索引案例

1、创建

```
df = pd.DataFrame({'month': [1, 4, 7, 10],
                    'year': [2012, 2014, 2013, 2014],
                    'sale': [55, 40, 84, 31]})
```

	month	sale	year
0	1	55	2012
1	4	40	2014
2	7	84	2013
3	10	31	2014

2、以月份设置新的索引

```
df.set_index('month')
```

	sale	year
month		
1	55	2012
4	40	2014
7	84	2013
10	31	2014

3、设置多个索引，以年和月份

```
df = df.set_index(['year', 'month'])
df
```

		sale
year	month	
2012	1	55
2014	4	40
2013	7	84
2014	10	31

注：通过刚才的设置，这样DataFrame就变成了一个具有MultiIndex的DataFrame。

3.MultiIndex与Panel

3.1 MultiIndex

MultiIndex是三维的数据结构;

多级索引（也称层次化索引）是pandas的重要功能，可以在Series、DataFrame对象上拥有2个以及2个以上的索引。

3.1.1 multiIndex的特性

打印刚才的df的行索引结果

```
df.index

MultiIndex(levels=[[2012, 2013, 2014], [1, 4, 7, 10]],
            labels=[[0, 2, 1, 2], [0, 1, 2, 3]],
            names=['year', 'month'])
```

多级或分层索引对象。

- index属性
 - names:levels的名称
 - levels: 每个level的元组值

```
df.index.names
# FrozenList(['year', 'month'])

df.index.levels
# FrozenList([[1, 2], [1, 4, 7, 10]])
```

3.1.2 multiIndex的创建

```
arrays = [[1, 1, 2, 2], ['red', 'blue', 'red', 'blue']]
pd.MultiIndex.from_arrays(arrays, names=('number', 'color'))

# 结果
MultiIndex(levels=[[1, 2], ['blue', 'red']],
            codes=[[0, 0, 1, 1], [1, 0, 1, 0]],
            names=['number', 'color'])
```

3.2 Panel

3.2.1 panel的创建

- class pandas.Panel (data=None, items=None, major_axis=None, minor_axis=None)
 - 作用：存储3维数组的Panel结构
 - 参数：
 - data : ndarray或者dataframe
 - items : 索引或类似数组的对象，axis=0
 - major_axis : 索引或类似数组的对象，axis=1
 - minor_axis : 索引或类似数组的对象，axis=2

```
p = pd.Panel(data=np.arange(24).reshape(4,3,2),
             items=list('ABCD'),
             major_axis=pd.date_range('20130101', periods=3),
             minor_axis=['first', 'second'])

# 结果
<class 'pandas.core.panel.Panel'>
Dimensions: 4 (items) x 3 (major_axis) x 2 (minor_axis)
Items axis: A to D
Major_axis axis: 2013-01-01 00:00:00 to 2013-01-03 00:00:00
Minor_axis axis: first to second
```

3.2.2 查看panel数据

```
p[:, :, "first"]
p["B", :, :]
```

注：Pandas从版本0.20.0开始弃用：推荐的用于表示3D数据的方法是通过DataFrame上的MultiIndex方法

4 小结

- pandas的优势【了解】
 - 增强图表可读性
 - 便捷的数据处理能力
 - 读取文件方便
 - 封装了Matplotlib、Numpy的画图和计算
- series【知道】
 - 创建
 - `pd.Series([], index=[])`
 - `pd.Series({})`
 - 属性
 - 对象.index
 - 对象.values
- DataFrame【掌握】
 - 创建
 - `pd.DataFrame(data=None, index=None, columns=None)`
 - 属性
 - `shape` -- 形状
 - `index` -- 行索引
 - `columns` -- 列索引
 - `values` -- 查看值
 - `T` -- 转置
 - `head()` -- 查看头部内容
 - `tail()` -- 查看尾部内容
 - DataFrame索引
 - 修改的时候,需要进行全局修改
 - 对象.reset_index()
 - 对象.set_index(keys)
- MultiIndex与Panel【了解】
 - multiIndex:
 - 类似ndarray中的三维数组
 - 创建:
 - `pd.MultiIndex.from_arrays()`
 - 属性:
 - 对象.index
 - panel:
 - `pd.Panel(data, items, major_axis, minor_axis)`
 - panel数据要是想看到,则需要进行索引到dataframe或者series才可以

5.3 基本数据操作

学习目标

- 目标
 - 记忆DataFrame的形状、行列索引名称获取等基本属性
 - 应用Series和DataFrame的索引进行切片获取
 - 应用sort_index和sort_values实现索引和值的排序

为了更好的理解这些基本操作，我们将读取一个真实的股票数据。关于文件操作，后面在介绍，这里只先用一下API

```
# 读取文件
data = pd.read_csv("../data/stock_day.csv")

# 删除一些列，让数据更简单些，再去后面操作
data = data.drop(["ma5", "ma10", "ma20", "v_ma5", "v_ma10", "v_ma20"], axis=1)
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
2018-02-23	22.88	23.37	22.82	22.71	52914.01	0.54	2.42	21.938	21.929	23.022	35119.58	41871.97	56372.85	1.32
2018-02-22	22.25	22.76	22.28	22.02	36105.01	0.36	1.64	21.446	21.909	23.137	35397.58	39904.78	60149.60	0.90
2018-02-14	21.49	21.99	21.92	21.48	23331.04	0.44	2.05	21.366	21.923	23.253	33590.21	42935.74	61716.11	0.58
2018-02-13	21.40	21.90	21.48	21.31	30802.45	0.28	1.32	21.342	22.103	23.387	39694.65	45518.14	65161.68	0.77
2018-02-12	20.70	21.40	21.19	20.63	32445.39	0.82	4.03	21.504	22.338	23.533	44645.16	45679.94	68686.33	0.81
2018-02-09	21.20	21.46	20.36	20.19	54304.01	-1.50	-6.86	21.920	22.596	23.645	48624.36	48982.38	70552.47	1.36
2018-02-08	21.79	22.09	21.88	21.75	27068.16	0.09	0.41	22.372	23.009	23.839	44411.98	48612.16	73852.45	0.68
2018-02-07	22.69	23.11	21.80	21.29	53853.25	-0.50	-2.24	22.480	23.258	23.929	52281.28	56315.11	74925.33	1.35
2018-02-06	22.80	23.55	22.29	22.20	55555.00	-0.97	-4.17	22.864	23.607	24.029	51341.63	64413.58	75738.95	1.39
2018-02-05	22.45	23.39	23.27	22.25	52341.39	0.65	2.87	23.172	23.928	24.112	46714.72	69278.66	77070.00	1.31
2018-02-02	22.40	22.70	22.62	21.53	33242.11	0.20	0.89	23.272	24.114	24.184	49340.40	70873.73	79929.71	0.83
2018-02-01	23.71	23.86	22.42	22.22	66414.64	-1.30	-5.48	23.646	24.365	24.279	52812.35	80394.43	88480.92	1.66
2018-01-31	23.85	23.98	23.72	23.31	49155.02	-0.11	-0.46	24.036	24.583	24.411	60348.94	80496.48	91666.75	1.23
2018-01-30	23.71	24.08	23.83	23.70	32420.43	0.05	0.21	24.350	24.671	24.365	77485.53	84805.23	92943.35	0.81
2018-01-29	24.40	24.63	23.77	23.72	65469.81	-0.73	-2.98	24.684	24.728	24.294	91842.60	91692.73	93456.22	1.64
2018-01-26	24.27	24.74	24.49	24.22	50601.83	0.11	0.45	24.956	24.694	24.221	92407.05	92122.56	91980.51	1.27

1 索引操作

Numpy当中我们已经讲过使用索引选取序列和切片选择，pandas也支持类似的操作，也可以使用列名、行名称，甚至组合使用。

1.1 直接使用行列索引(先列后行)

获取'2018-02-27'这天的'close'的结果

```
# 直接使用行列索引名字的方式（先列后行）
data['open']['2018-02-27']
23.53

# 不支持的操作
# 错误
data['2018-02-27']['open']
# 错误
data[:, :2]
```

1.2 结合loc或者iloc使用索引

获取从'2018-02-27':'2018-02-22', 'open'的结果

```
# 使用loc:只能指定行列索引的名字
data.loc['2018-02-27':'2018-02-22', 'open']

2018-02-27    23.53
2018-02-26    22.80
2018-02-23    22.88
Name: open, dtype: float64

# 使用iloc可以通过索引的下标去获取
# 获取前3天数据,前5列的结果
data.iloc[:3, :5]
```

	open	high	close	low
2018-02-27	23.53	25.88	24.16	23.53
2018-02-26	22.80	23.78	23.53	22.80
2018-02-23	22.88	23.37	22.82	22.71

1.3 使用ix组合索引

Warning:Starting in 0.20.0, the `.ix` indexer is deprecated, in favor of the more strict `.iloc` and `.loc` indexers.

获取行第1天到第4天, ['open', 'close', 'high', 'low']这个四个指标的结果

```
# 使用ix进行下表和名称组合做引
data.ix[0:4, ['open', 'close', 'high', 'low']]

# 推荐使用loc和iloc来获取的方式
data.loc[data.index[0:4], ['open', 'close', 'high', 'low']]
data.iloc[0:4, data.columns.get_indexer(['open', 'close', 'high', 'low'])]
```

	open	close	high	low
2018-02-27	23.53	24.16	25.88	23.53
2018-02-26	22.80	23.53	23.78	22.80
2018-02-23	22.88	22.82	23.37	22.71
2018-02-22	22.25	22.28	22.76	22.02

2 赋值操作

对DataFrame当中的close列进行重新赋值为1

```
# 直接修改原来的值
data['close'] = 1
# 或者
data.close = 1
```

3 排序

排序有两种形式, 一种对于索引进行排序, 一种对于内容进行排序

3.1 DataFrame排序

- 使用`df.sort_values(by=, ascending=)`
 - 单个键或者多个键进行排序,
 - 参数:
 - `by`: 指定排序参考的键
 - `ascending`:默认升序
 - `ascending=False`:降序
 - `ascending=True`:升序

```
# 按照开盘价大小进行排序 , 使用ascending指定按照大小排序
data.sort_values(by="open", ascending=True).head()
```

	open	high	close	low	volume	price_change	p_change	turnover
2015-03-02	12.25	12.67	12.52	12.20	1000	0.32	2.62	3.30
2015-09-02	12.30	14.11	12.36	12.30	1000	-1.10	-8.17	2.40
2015-03-03	12.52	13.06	12.70	12.52	1000	0.18	1.44	4.76
2015-03-04	12.80	12.92	12.90	12.61	1000	0.20	1.57	2.30
2015-03-05	12.88	13.45	13.16	12.87	1000	0.26	2.02	3.19

```
# 按照多个键进行排序
data.sort_values(by=['open', 'high'])
```

	open	high	close	low	volume	price_change	p_change	turnover
2015-03-02	12.25	12.67	12.52	12.20	1000	0.32	2.62	3.30
2015-09-02	12.30	14.11	12.36	12.30	1000	-1.10	-8.17	2.40
2015-03-03	12.52	13.06	12.70	12.52	1000	0.18	1.44	4.76
2015-03-04	12.80	12.92	12.90	12.61	1000	0.20	1.57	2.30
2015-03-05	12.88	13.45	13.16	12.87	1000	0.26	2.02	3.19

- 使用df.sort_index给索引进行排序

这个股票的日期索引原来是从大到小，现在重新排序，从小到大

```
# 对索引进行排序
data.sort_index()
```

	open	high	close	low	volume	price_change	p_change	turnover
2015-03-02	12.25	12.67	12.52	12.20	1000	0.32	2.62	3.30
2015-03-03	12.52	13.06	12.70	12.52	1000	0.18	1.44	4.76
2015-03-04	12.80	12.92	12.90	12.61	1000	0.20	1.57	2.30
2015-03-05	12.88	13.45	13.16	12.87	1000	0.26	2.02	3.19
2015-03-06	13.17	14.48	14.28	13.13	1000	1.12	8.51	6.16

3.2 Series排序

- 使用series.sort_values(ascending=True)进行排序

series排序时，只有一列，不需要参数

```
data['p_change'].sort_values(ascending=True).head()

2015-09-01    -10.03
2015-09-14    -10.02
2016-01-11    -10.02
2015-07-15    -10.02
2015-08-26    -10.01
Name: p_change, dtype: float64
```

- 使用series.sort_index()进行排序

与df一致

```
# 对索引进行排序
data['p_change'].sort_index().head()

2015-03-02    2.62
2015-03-03    1.44
2015-03-04    1.57
2015-03-05    2.02
2015-03-06    8.51
Name: p_change, dtype: float64
```

4 总结

- 1.索引【掌握】
 - 直接索引 -- 先列后行,是需要通过索引的字符串进行获取
 - loc -- 先行后列,是需要通过索引的字符串进行获取
 - iloc -- 先行后列,是通过下标进行索引
 - ix -- 先行后列,可以用上面两种方法混合进行索引
- 2.赋值【知道】
 - `data[""] = **`
 - `data. =`
- 3.排序【知道】
 - dataframe
 - 对象.sort_values()
 - 对象.sort_index()
 - series
 - 对象.sort_values()
 - 对象.sort_index()

5.4 DataFrame运算

学习目标

- 目标
 - 应用add等实现数据间的加、减法运算
 - 应用逻辑运算符实现数据的逻辑筛选
 - 应用isin, query实现数据的筛选
 - 使用describe完成综合统计
 - 使用max, min, mean, std完成统计计算
 - 使用idxmin、idxmax完成最大值最小值的索引
 - 使用cumsum等实现累计分析
 - 应用apply函数实现数据的自定义处理

1 算术运算

- add(other)

比如进行数学运算加上具体的一个数字

```
data['open'].add(1)

2018-02-27    24.53
2018-02-26    23.80
2018-02-23    23.88
2018-02-22    23.25
2018-02-14    22.49
```

- sub(other)

2 逻辑运算

2.1 逻辑运算符

- 例如筛选data["open"] > 23的日期数据
 - data["open"] > 23返回逻辑结果

```
data["open"] > 23

2018-02-27    True
2018-02-26   False
2018-02-23   False
2018-02-22   False
2018-02-14   False
```

```
# 逻辑判断的结果可以作为筛选的依据
data[data["open"] > 23].head()
```

	open	high	close	low	volume	price_change	p_change	turnover
2018-02-27	23.53	25.88	24.16	23.53	95578.03	0.63	2.68	2.39
2018-02-01	23.71	23.86	22.42	22.22	66414.64	-1.30	-5.48	1.66
2018-01-31	23.85	23.98	23.72	23.31	49155.02	-0.11	-0.46	1.23
2018-01-30	23.71	24.08	23.83	23.70	32420.43	0.05	0.21	0.81
2018-01-29	24.40	24.63	23.77	23.72	65469.81	-0.73	-2.98	1.64

- 完成多个逻辑判断，

```
data[(data["open"] > 23) & (data["open"] < 24)].head()
```

	open	high	close	low	volume	price_change	p_change	turnover
2018-02-27	23.53	25.88	24.16	23.53	95578.03	0.63	2.68	2.39
2018-02-01	23.71	23.86	22.42	22.22	66414.64	-1.30	-5.48	1.66
2018-01-31	23.85	23.98	23.72	23.31	49155.02	-0.11	-0.46	1.23
2018-01-30	23.71	24.08	23.83	23.70	32420.43	0.05	0.21	0.81
2018-01-16	23.40	24.60	24.40	23.30	101295.42	0.96	4.10	2.54

2.2 逻辑运算函数

- query(expr)
 - expr: 查询字符串

通过query使得刚才的过程更加方便简单

```
data.query("open<24 & open>23").head()
```

- isin(values)

例如判断'open'是否为23.53和23.85

```
# 可以指定值进行一个判断，从而进行筛选操作
data[data["open"].isin([23.53, 23.85])]
```

	open	high	close	low	volume	price_change	p_change	turnover
2018-02-27	23.53	25.88	24.16	23.53	95578.03	0.63	2.68	2.39
2018-01-31	23.85	23.98	23.72	23.31	49155.02	-0.11	-0.46	1.23
2017-07-26	23.53	23.92	23.40	22.85	110276.48	-0.30	-1.27	2.76
2015-12-18	23.53	24.66	23.99	23.43	109230.05	0.65	2.79	3.74
2015-11-26	23.85	24.08	23.53	23.50	51446.29	-0.31	-1.30	1.76

3 统计运算

3.1 describe

综合分析: 能够直接得出很多统计结果, count, mean, std, min, max 等

```
# 计算平均值、标准差、最大值、最小值
data.describe()
```

	open	high	close	low	volume	price_change	p_change	turnover	my_price_change
count	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000
mean	21.272706	21.900513	21.336267	20.771835	99905.519114	0.018802	0.190280	2.936190	0.063561
std	3.930973	4.077578	3.942806	3.791968	73879.119354	0.898476	4.079698	2.079375	0.800565
min	12.250000	12.670000	12.360000	12.200000	1158.120000	-3.520000	-10.030000	0.040000	-3.300000
25%	19.000000	19.500000	19.045000	18.525000	48533.210000	-0.390000	-1.850000	1.360000	-0.300000
50%	21.440000	21.970000	21.450000	20.980000	83175.930000	0.050000	0.260000	2.500000	0.080000
75%	23.400000	24.065000	23.415000	22.850000	127580.055000	0.455000	2.305000	3.915000	0.450000
max	34.990000	36.350000	35.210000	34.010000	501915.410000	3.030000	10.030000	12.560000	3.410000

3.2 统计函数

Numpy当中已经详细介绍，在这里我们演示min(最小值), max(最大值), mean(平均值), median(中位数), var(方差), std(标准差),mode(众数)结果:

count	Number of non-NA observations
sum	Sum of values
mean	Mean of values
median	Arithmetic median of values
min	Minimum
max	Maximum
mode	Mode
abs	Absolute Value
prod	Product of values
std	Bessel-corrected sample standard deviation
var	Unbiased variance
idxmax	compute the index labels with the maximum
idxmin	compute the index labels with the minimum

对于单个函数去进行统计的时候，坐标轴还是按照默认列“columns” (axis=0, default)， 如果要对行“index” 需要指定(axis=1)

- max(), min()

```
# 使用统计函数: 0 代表列求结果, 1 代表行求统计结果
data.max(0)

open          34.99
high          36.35
close         35.21
low           34.01
volume        501915.41
price_change   3.03
p_change      10.03
turnover      12.56
my_price_change 3.41
dtype: float64
```

- std(), var()

```
# 方差
data.var(0)

open          1.545255e+01
high          1.662665e+01
close         1.554572e+01
low           1.437902e+01
volume        5.458124e+09
price_change   8.072595e-01
p_change      1.664394e+01
turnover      4.323800e+00
my_price_change 6.409037e-01
dtype: float64

# 标准差
data.std(0)

open          3.930973
high          4.077578
close         3.942806
low           3.791968
volume        73879.119354
price_change   0.898476
p_change      4.079698
turnover      2.079375
my_price_change 0.800565
dtype: float64
```

- median(): 中位数

中位数为将数据从小到大排列，在最中间的那个数为中位数。如果没有中间数，取中间两个数的平均值。

```
df = pd.DataFrame({'COL1' : [2,3,4,5,4,2],
                   'COL2' : [0,1,2,3,4,2]})

df.median()

COL1    3.5
COL2    2.0
dtype: float64
```

- `idxmax()`、`idxmin()`

```
# 求出最大值的位置
data.idxmax(axis=0)

open          2015-06-15
high          2015-06-10
close         2015-06-12
low           2015-06-12
volume        2017-10-26
price_change  2015-06-09
p_change      2015-08-28
turnover      2017-10-26
my_price_change 2015-07-10
dtype: object
```

```
# 求出最小值的位置
data.idxmin(axis=0)

open          2015-03-02
high          2015-03-02
close         2015-09-02
low           2015-03-02
volume        2016-07-06
price_change  2015-06-15
p_change      2015-09-01
turnover      2016-07-06
my_price_change 2015-06-15
dtype: object
```

3.3 累计统计函数

函数	作用
<code>cumsum</code>	计算前1/2/3/.../n个数的和
<code>cummax</code>	计算前1/2/3/.../n个数的最大值
<code>cummin</code>	计算前1/2/3/.../n个数的最小值
<code>cumprod</code>	计算前1/2/3/.../n个数的积

那么这些累计统计函数怎么用？

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
2015-02-25	12.22	12.36	12.09	12.07	33474.90	-0.13	-1.06	12.090	12.090	12.090	33474.90	33474.90	33474.90	1.15
2015-02-26	12.09	12.18	12.13	12.00	25990.64	0.04	0.33	12.110	12.110	12.110	29732.77	29732.77	29732.77	0.89
2015-02-27	12.06	12.22	12.20	12.06	35393.05	0.07	0.58	12.140	12.140	12.140	31619.53	31619.53	31619.53	1.21
2015-03-02	12.25	12.67	12.52	12.20	96291.73	0.32	2.62	12.235	12.235	12.235	47787.58	47787.58	47787.58	3.30
2015-03-03	12.52	13.06	12.70	12.52	139071.61	0.18	1.44	12.328	12.328	12.328	66044.39	66044.39	66044.39	4.76
2015-03-04	12.80	12.92	12.90	12.61	67075.44	0.20	1.57	12.490	12.423	12.423	72764.49	66216.23	66216.23	2.30
2015-03-05	12.88	13.45	13.16	12.87	93180.39	0.26	2.02	12.696	12.529	12.529	86202.44	70068.25	70068.25	3.19
2015-03-06	13.17	14.48	14.28	13.13	179831.72	1.12	8.51	13.112	12.748	12.748	115090.18	83788.69	83788.69	6.16
2015-03-09	14.14	14.85	14.31	13.80	144945.66	0.03	0.21	13.470	12.921	12.921	124820.96	90583.90	90583.90	4.96
2015-03-10	14.20	14.80	14.65	14.01	101213.51	0.34	2.38	13.860	13.094	13.094	117249.34	91646.87	91646.87	3.46
2015-03-11	14.80	15.08	14.30	14.14	119708.43	-0.35	-2.39	14.140	13.315	13.204	127775.94	100270.22	94197.92	4.10
2015-03-12	14.11	14.80	14.11	13.95	84978.37	-0.19	-1.33	14.330	13.513	13.279	126135.54	106168.99	93429.62	2.91
2015-03-13	14.13	14.50	14.47	14.08	61342.22	0.36	2.55	14.368	13.740	13.371	102437.64	108763.91	90961.36	2.10
2015-03-16	14.52	15.05	14.87	14.51	94468.30	0.40	2.76	14.480	13.975	13.478	92342.17	108581.56	91211.85	3.23
2015-03-17	14.90	15.44	15.18	14.63	158770.77	0.31	2.08	14.586	14.223	13.591	103853.62	110551.48	95715.78	5.43
2015-03-18	15.18	15.66	15.31	15.02	121538.71	0.13	0.86	14.788	14.464	13.699	104219.67	115997.81	97329.72	4.16
2015-03-19	15.20	15.64	15.38	15.11	93644.19	0.07	0.46	15.042	14.686	13.798	105952.84	116044.19	97112.92	3.21
2015-03-20	15.38	15.48	15.34	15.18	76800.13	-0.04	-0.26	15.216	14.792	13.883	109044.42	105741.03	95984.43	2.63
2015-03-23	15.34	15.56	15.38	15.25	89461.32	0.04	0.26	15.318	14.899	13.962	108043.02	100192.60	95641.11	3.06
2015-03-24	15.38	16.16	15.68	15.28	153390.08	0.30	1.95	15.418	15.002	14.048	106966.89	105410.25	98528.56	5.25
2015-03-25	15.97	15.97	15.30	15.18	97174.40	-0.38	-2.42	15.416	15.102	14.209	102094.02	103156.85	101713.53	3.33

以上这些函数可以对series和dataframe操作

这里我们按照时间的从前往后来进行累计

- 排序

```
# 排序之后，进行累计求和
data = data.sort_index()
```

- 对p_change进行求和

```
stock_rise = data['p_change']
# plot方法集成了前面直方图、条形图、饼图、折线图
stock_rise.cumsum()
```

```
2015-03-02    2.62
2015-03-03    4.06
2015-03-04    5.63
2015-03-05    7.65
2015-03-06   16.16
2015-03-09   16.37
2015-03-10   18.75
2015-03-11   16.36
2015-03-12   15.03
2015-03-13   17.58
2015-03-16   20.34
2015-03-17   22.42
2015-03-18   23.28
2015-03-19   23.74
2015-03-20   23.48
2015-03-23   23.74
```

那么如何让这个连续求和的结果更好的显示呢？



如果要使用plot函数，需要导入matplotlib.

```
import matplotlib.pyplot as plt
# plot显示图形
stock_rise.cumsum().plot()
# 需要调用show，才能显示出结果
plt.show()
```

关于plot，稍后会介绍API的选择

4 自定义运算

- apply(func, axis=0)
 - func:自定义函数
 - axis=0:默认是列，axis=1为行进行运算
- 定义一个对列，最大值-最小值的函数

```
data[['open', 'close']].apply(lambda x: x.max() - x.min(), axis=0)

open      22.74
close     22.85
dtype: float64
```

5 小结

- 算术运算【知道】
- 逻辑运算【知道】
 - 1.逻辑运算符号
 - 2.逻辑运算函数
 - 对象.query()
 - 对象.isin()
- 统计运算【知道】
 - 1.对象.describe()
 - 2.统计函数
 - 3.累积统计函数
- 自定义运算【知道】

- `apply(func, axis=0)`

5.5 Pandas画图

学习目标

- 目标
 - 了解DataFrame的画图函数
 - 了解Series的画图函数
-

1 pandas.DataFrame.plot

- `DataFrame.plot (kind='line')`
- `kind`: str, 需要绘制图形的种类
 - 'line': line plot (default)
 - 'bar': vertical bar plot
 - 'barh': horizontal bar plot
 - 关于“barh”的解释:
 - <http://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.barh.html>
 - 'hist': histogram
 - 'pie': pie plot
 - 'scatter': scatter plot

更多细节: <https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.plot.html?highlight=plot#pandas.DataFrame.plot>

2 pandas.Series.plot

更多细节: <https://pandas.pydata.org/pandas-docs/stable/generated/pandas.Series.plot.html?highlight=plot#pandas.Series.plot>

5.6 文件读取与存储

学习目标

- 目标
 - 了解Pandas的几种文件读取存储操作
 - 应用CSV方式、HDF方式和json方式实现文件的读取和存储

我们的数据大部分存在于文件当中，所以pandas会支持复杂的IO操作，pandas的API支持众多的文件格式，如CSV、SQL、XLS、JSON、HDF5。

注：最常用的HDF5和CSV文件

Format Type	Data Description	Reader	Writer
text	CSV	read_csv	to_csv
text	JSON	read_json	to_json
text	HTML	read_html	to_html
text	Local clipboard	read_clipboard	to_clipboard
binary	MS Excel	read_excel	to_excel
binary	HDF5 Format	read_hdf	to_hdf
binary	Feather Format	read_feather	to_feather
binary	Parquet Format	read_parquet	to_parquet
binary	Msgpack	read_msgpack	to_msgpack
binary	Stata	read_stata	to_stata
binary	SAS	read_sas	
binary	Python Pickle Format	read_pickle	to_pickle
SQL	SQL	read_sql	to_sql
SQL	Google Big Query	read_gbq	to_gbq

1 CSV

1.1 read_csv

- `pandas.read_csv(filepath_or_buffer, sep=',', usecols=)`
 - `filepath_or_buffer`: 文件路径
 - `sep`: 分隔符，默认用"," 隔开
 - `usecols`: 指定读取的列名，列表形式
- 举例：读取之前的股票的数据

```
# 读取文件,并且指定只获取'open', 'close'指标
data = pd.read_csv("./data/stock_day.csv", usecols=['open', 'close'])
```

	open	close
2018-02-27	23.53	24.16
2018-02-26	22.80	23.53
2018-02-23	22.88	22.82
2018-02-22	22.25	22.28
2018-02-14	21.49	21.92

1.2 to_csv

- `DataFrame.to_csv(path_or_buf=None, sep=',', columns=None, header=True, index=True, mode='w', encoding=None)`
 - `path_or_buf`: 文件路径
 - `sep`: 分隔符，默认用"," 隔开
 - `columns`: 选择需要的列索引
 - `header`: boolean or list of string, default True, 是否写进列索引值

- `index`:是否写进行索引
- `mode:'w'`: 重写, 'a' 追加
- 举例: 保存读取出来的股票数据
 - 保存'open'列的数据, 然后读取查看结果

```
# 选取10行数据保存, 便于观察数据
data[:10].to_csv("./data/test.csv", columns=['open'])
```

```
# 读取, 查看结果
pd.read_csv("./data/test.csv")
```

```

      Unnamed: 0    open
0    2018-02-27    23.53
1    2018-02-26    22.80
2    2018-02-23    22.88
3    2018-02-22    22.25
4    2018-02-14    21.49
5    2018-02-13    21.40
6    2018-02-12    20.70
7    2018-02-09    21.20
8    2018-02-08    21.79
9    2018-02-07    22.69
```

会发现将索引存入到文件当中, 变成单独的一列数据。如果需要删除, 可以指定`index`参数, 删除原来的文件, 重新保存一次。

```
# index: 存储不会讲索引值变成一列数据
data[:10].to_csv("./data/test.csv", columns=['open'], index=False)
```

2 HDF5

2.1 read_hdf与to_hdf

HDF5文件的读取和存储需要指定一个键, 值为要存储的**DataFrame**

- `pandas.read_hdf(path_or_buf, key=None, **kwargs)`
 - 从h5文件当中读取数据
 - `path_or_buffer`: 文件路径
 - `key`: 读取的键
 - `return`: Theselected object
- `DataFrame.to_hdf(path_or_buf, key, *kwargs*)`

2.2 案例

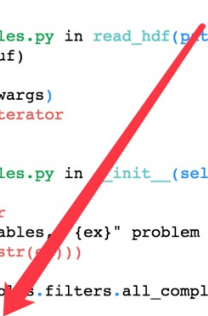
- 读取文件

```
day_close = pd.read_hdf("./data/day_close.h5")
```

如果读取的时候出现以下错误

```
pd.read_hdf("./data/stock_plot/day_high.h5")
```

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
~/virtualenv/ml3/lib/python3.6/site-packages/pandas/io/pytables.py in __init__(self, path, mode, complevel, complib,  
    fletcher32, **kwargs)  
    446         try:  
--> 447             import tables # noqa  
    448         except ImportError as ex: # pragma: no cover  
  
ModuleNotFoundError: No module named 'tables'  
  
During handling of the above exception, another exception occurred:  
  
ImportError                                Traceback (most recent call last)  
<ipython-input-39-01a9490c6153> in <module>()  
----> 1 pd.read_hdf("./data/stock_plot/day_high.h5")  
  
~/virtualenv/ml3/lib/python3.6/site-packages/pandas/io/pytables.py in read_hdf(path_or_buf, key, mode, **kwargs)  
    338         'File %s does not exist' % path_or_buf)  
    339  
--> 340     store = HDFStore(path_or_buf, mode=mode, **kwargs)  
    341     # can't auto open/close if we are using an iterator  
    342     # so delegate to the iterator  
  
~/virtualenv/ml3/lib/python3.6/site-packages/pandas/io/pytables.py in __init__(self, path, mode, complevel, complib,  
    fletcher32, **kwargs)  
    448     except ImportError as ex: # pragma: no cover  
    449         raise ImportError('HDFStore requires PyTables, {ex}" problem '  
--> 450             'importing'.format(ex=str(ex)))  
    451  
    452     if complib is not None and complib not in tables.filters.all_complibs:  
  
ImportError: HDFStore requires PyTables, "No module named 'tables'" problem importing
```



缺少这个模块

需要安装安装tables模块避免不能读取HDF5文件

```
pip install tables
```

```
six (1.10.0)  
smart-open (1.5.3)  
tables (3.4.2)  
tensorflow (1.0.1)
```

- 存储文件

```
day_close.to_hdf("./data/test.h5", key="day_close")
```

再次读取的时候, 需要指定键的名字

```
new_close = pd.read_hdf("./data/test.h5", key="day_close")
```

注意: 优先选择使用HDF5文件存储

- HDF5在存储的时候支持压缩, 使用的方式是**blosc**, 这个是速度最快的也是pandas默认支持的
- 使用压缩可以提高磁盘利用率, 节省空间
- HDF5还是跨平台的, 可以轻松迁移到hadoop上面

3 JSON

JSON是我们常用的一种数据交换格式, 前面在前后端的交互经常用到, 也会在存储的时候选择这种格式。所以我们需要知道Pandas如何进行读取和存储JSON格式。

3.1 read_json

- pandas.read_json(path_or_buf=None, orient=None, typ='frame', lines=False)

- 将JSON格式转换成默认的Pandas DataFrame格式
- orient : string, Indication of expected JSON string format.
 - 'split' : dict like {index -> [index], columns -> [columns], data -> [values]}
 - split 将索引总结到索引，列名到列名，数据到数据。将三部分都分开了
 - 'records' : list like [{column -> value}, ... , {column -> value}]
 - records 以 columns: values 的形式输出
 - 'index' : dict like {index -> {column -> value}}
 - index 以 index: {columns: values}... 的形式输出
 - 'columns' : dict like {column -> {index -> value}}, 默认该格式
 - columns 以 columns:{index:values} 的形式输出
 - 'values' : just the values array
 - values 直接输出值
- lines : boolean, default False
 - 按照每行读取json对象
- typ : default 'frame', 指定转换成的对象类型series或者dataframe

3.2 read_json 案例

- 数据介绍

这里使用一个新闻标题讽刺数据集，格式为json。 `is_sarcastic` : 1讽刺的，否则为0； `headline` : 新闻报道的标题； `article_link` : 链接到原始新闻文章。存储格式为：

```
{
  "article_link": "https://www.huffingtonpost.com/entry/versace-black-code_us_5861fbefe4b0de3a08f600d5",
  "headline": "former versace store clerk sue s over secret 'black code' for minority shoppers",
  "is_sarcastic": 0
}
{"article_link": "https://www.huffingtonpost.com/entry/roseanne-revival-review_us_5ab3a497e4b054d118e04365",
  "headline": "the 'roseanne' revival catches up to our thorny political mood, for better and worse",
  "is_sarcastic": 0
}
```

- 读取

orient指定存储的json格式，lines指定按照行去变成一个样本

```
json_read = pd.read_json("./data/Sarcasm_Headlines_Dataset.json", orient="records", lines=True)
```

结果为：

	article_link	headline	is_sarcastic
0	https://www.huffingtonpost.com/entry/versace-b...	former versace store clerk sues over secret 'b...	0
1	https://www.huffingtonpost.com/entry/roseanne-...	the 'roseanne' revival catches up to our thorn...	0
2	https://local.theonion.com/mom-starting-to-fea...	mom starting to fear son's web series closest ...	1
3	https://politics.theonion.com/boehner-just-wan...	boehner just wants wife to listen, not come up...	1
4	https://www.huffingtonpost.com/entry/jk-rowlin...	j.k. rowling wishes snape happy birthday in th...	0
5	https://www.huffingtonpost.com/entry/advancing...	advancing the world's women	0
6	https://www.huffingtonpost.com/entry/how-meat-...	the fascinating case for eating lab-grown meat	0
7	https://www.huffingtonpost.com/entry/boxed-col...	this ceo will send your kids to school, if you...	0
8	https://politics.theonion.com/top-snake-handle...	top snake handler leaves sinking huckabee camp...	1
9	https://www.huffingtonpost.com/entry/fridays-m...	friday's morning email: inside trump's presser...	0
10	https://www.huffingtonpost.com/entry/airline-p...	airline passengers tackle man who rushes cockp...	0

3.3 to_json

- DataFrame.to_json(path_or_buf=None, orient=None, lines=False)
 - 将Pandas 对象存储为json格式
 - path_or_buf=None: 文件地址
 - orient:存储的json形式，{'split','records','index','columns','values'}
 - lines:一个对象存储为一行

3.4 案例

- 存储文件

```
json_read.to_json("./data/test.json", orient='records')
```

结果

```
[{"article_link": "https://www.huffingtonpost.com/entry/versace-black-code_us_5861fbefe4b0de3a08f600d5", "headline": "former versace store clerk sues over secret 'black code' for minority shoppers", "is_sarcastic": 0}, {"article_link": "https://www.huffingtonpost.com/entry/roseanne-revival-review_us_5ab3a497e4b054d118e04365", "headline": "the 'roseanne' revival catches up to our thorny political mood, for better and worse", "is_sarcastic": 0}, {"article_link": "https://local.theonion.com/mom-starting-to-fear-son-s-web-series-closest-thing-she-1819576697", "headline": "mom starting to fear son's web series closest thing she will have to grandchild", "is_sarcastic": 1}, {"article_link": "https://politics.theonion.com/boehner-just-wants-wife-to-listen-not-come-up-with-alt-1819574302", "headline": "boehner just wants wife to listen, not come up with alternative debt-reduction ideas", "is_sarcastic": 1}, {"article_link": "https://www.huffingtonpost.com/entry/jk-rowling-wishes-snape-happy-birthday_us_569117c4e4b0cad15e64fdcb", "headline": "j.k. rowling wishes snape happy birthday in the most magical way", "is_sarcastic": 0}, {"article_link": "https://www.huffingtonpost.com/entry/advancing-the-worlds-women_b_6810038.html", "headline": "advancing the world's women", "is_sarcastic": 0}, ...]
```

- 修改lines参数为True

```
json_read.to_json("./data/test.json", orient='records', lines=True)
```

结果

```
{"article_link": "https://www.huffingtonpost.com/entry/versace-black-code_us_5861fbefe4b0de3a08f600d5", "headline": "former versace store clerk sues over secret 'black code' for minority shoppers", "is_sarcastic": 0}
{"article_link": "https://www.huffingtonpost.com/entry/roseanne-revival-review_us_5ab3a497e4b054d118e04365", "headline": "the 'roseanne' revival catches up to our thorny political mood, for better and worse", "is_sarcastic": 0}
{"article_link": "https://local.theonion.com/mom-starting-to-fear-son-s-web-series-closest-thing-she-1819576697", "headline": "mom starting to fear son's web series closest thing she will have to grandchild", "is_sarcastic": 1}
{"article_link": "https://politics.theonion.com/boehner-just-wants-wife-to-listen-not-come-up-with-alt-1819574302", "headline": "boehner just wants wife to listen, not come up with alternative debt-reduction ideas", "is_sarcastic": 1}
{"article_link": "https://www.huffingtonpost.com/entry/jk-rowling-wishes-snape-happy-birthday_us_569117c4e4b0cad15e64fdcb", "headline": "j.k. rowling wishes snape happy birthday in the most magical way", "is_sarcastic": 0}...
```

4 小结

- pandas的CSV、HDF5、JSON文件的读取【知道】
 - 对象.read_**()
 - 对象.to_**()

5.7 高级处理-缺失值处理

学习目标

- 目标
 - 应用isnull判断是否有缺失数据NaN
 - 应用fillna实现缺失值的填充
 - 应用dropna实现缺失值的删除
 - 应用replace实现数据的替换

83	6.8	222	NaN	NaN
159	8.3	76193	6.50	NaN
99	4.7	523	0.01	NaN

1 如何处理nan

- 获取缺失值的标记方式(NaN或者其他标记方式)
- 如果缺失值的标记方式是NaN
 - 判断数据中是否包含NaN:
 - pd.isnull(df),
 - pd.notnull(df)
 - 存在缺失值nan:
 - 1、删除存在缺失值的:dropna(axis='rows')
 - 注: 不会修改原数据, 需要接受返回值
 - 2、替换缺失值:fillna(value, inplace=True)
 - value:替换成的值
 - inplace:True:会修改原数据, False:不替换修改原数据, 生成新的对象
- 如果缺失值没有使用NaN标记, 比如使用"? "
 - 先替换'?'为np.nan, 然后继续处理

2 电影数据的缺失值处理

- 电影数据文件获取

```
# 读取电影数据
movie = pd.read_csv("./data/IMDB-Movie-Data.csv")
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0

2.1 判断缺失值是否存在

- pd.notnull()

```
pd.notnull(movie)
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	True	True	True	True	True	True	True	True	True	True	True	True
1	True	True	True	True	True	True	True	True	True	True	True	True
2	True	True	True	True	True	True	True	True	True	True	True	True
3	True	True	True	True	True	True	True	True	True	True	True	True
4	True	True	True	True	True	True	True	True	True	True	True	True
5	True	True	True	True	True	True	True	True	True	True	True	True
6	True	True	True	True	True	True	True	True	True	True	True	True
7	True	True	True	True	True	True	True	True	True	False	True	True

```
np.all(pd.notnull(movie))
```

- pd.isnull()

2.2 存在缺失值nan,并且是np.nan

- 1、删除

pandas删除缺失值，使用dropna的前提是，缺失值的类型必须是np.nan

```
# 不修改原数据
movie.dropna()

# 可以定义新的变量接受或者用原来的变量名
data = movie.dropna()
```

- 2、替换缺失值

```
# 替换存在缺失值的样本的两列
# 替换填充平均值，中位数
# movie['Revenue (Millions)'].fillna(movie['Revenue (Millions)'].mean(), inplace=True)
```

替换所有缺失值：

```
for i in movie.columns:
    if np.all(pd.notnull(movie[i])) == False:
        print(i)
        movie[i].fillna(movie[i].mean(), inplace=True)
```

2.3 不是缺失值nan，有默认标记的

数据是这样的：

17	1050670	10	7	7	6	4	10	4	1	2	4
18	1050718	6	1	1	1	2	1	3	1	1	2
19	1054590	7	3	2	10	5	10	5	4	4	4
20	1054593	10	5	5	3	6	7	7	10	1	4
21	1056784	3	1	1	1	2	1	2	1	1	2
22	1057013	8	4	5	1	2	?	7	3	1	4
23	1059552	1	1	1	1	2	1	3	1	1	2
24	1065726	5	2	3	4	2	7	3	6	1	4
25	1066373	3	2	1	1	1	1	2	1	1	2
26	1066979	5	1	1	1	2	1	2	1	1	2
27	1067444	2	1	1	1	2	1	2	1	1	2

```
wis = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/breast-cancer-wisconsin.data")
```

以上数据在读取时，可能会报如下错误：

```
URLError: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed (_ssl.c:833)>
```

解决办法：

```
# 全局取消证书验证
import ssl
ssl._create_default_https_context = ssl._create_unverified_context
```

处理思路分析：

- 1、先替换‘?’为np.nan
 - df.replace(to_replace=, value=)
 - to_replace:替换前的值
 - value:替换后的值

```
# 把一些其它值标记的缺失值，替换成np.nan
wis = wis.replace(to_replace='?', value=np.nan)
```

- 2、在进行缺失值的处理

```
# 删除
wis = wis.dropna()
```

3 小结

- `isnull`、`notnull`判断是否存在缺失值【知道】
 - `np.any(pd.isnull(movie))` # 里面如果有一个缺失值,就返回True
 - `np.all(pd.notnull(movie))` # 里面如果有一个缺失值,就返回False
- `dropna`删除`np.nan`标记的缺失值【知道】
 - `movie.dropna()`
- `fillna`填充缺失值【知道】
 - `movie[i].fillna(value=movie[i].mean(), inplace=True)`
- `replace`替换具体某些值【知道】
 - `wis.replace(to_replace="?", value=np.NaN)`

5.8 高级处理-数据离散化

学习目标

- 目标
 - 应用cut、qcut实现数据的区间分组
 - 应用get_dummies实现数据的one-hot编码

1 为什么要离散化

连续属性离散化的目的是为了简化数据结构，数据离散化技术可以用来减少给定连续属性值的个数。离散化方法经常作为数据挖掘的工具。

2 什么是数据的离散化

连续属性的离散化就是在连续属性的值域上，将值域划分为若干个离散的区间，最后用不同的符号或整数 值代表落在每个子区间中的属性值。

离散化有很多种方法，这使用一种最简单的方式去操作

- 原始人的身高数据：165, 174, 160, 180, 159, 163, 192, 184
- 假设按照身高分几个区间段：150~165, 165~180, 180~195

这样我们将数据分到了三个区间段，我可以对应的标记为矮、中、高三个类别，最终要处理成一个“哑变量”矩阵

3 股票的涨跌幅离散化

我们对股票每日的“p_change”进行离散化

	rise_(-100, -7]	rise_(-7, -5]	rise_(-5, -3]	rise_(-3, 0]	rise_(0, 3]	rise_(3, 5]	rise_(5, 7]	rise_(7, 100]
2018-02-27	0	0	0	0	1	0	0	0
2018-02-26	0	0	0	0	0	1	0	0
2018-02-23	0	0	0	0	1	0	0	0
2018-02-22	0	0	0	0	1	0	0	0
2018-02-14	0	0	0	0	1	0	0	0

3.1 读取股票的数据

先读取股票的数据，筛选出p_change数据

```
data = pd.read_csv("./data/stock_day.csv")
p_change= data['p_change']
```

3.2 将股票涨跌幅数据进行分组

```
(1.106, 1.57]      67
(7.198, 10.03]     65
(3.662, 5.11]      65
(0.332, 0.73]      65
(-0.001, 0.332]    65
(2.802, 3.662]     64
(1.57, 2.1]        64
(5.11, 7.198]      63
(0.73, 1.106]      63
(2.1, 2.802]       62
Name: p_change, dtype: int64
```

使用的工具:

- `pd.qcut(data, q):`
 - 对数据进行分组将数据分组, 一般会与`value_counts`搭配使用, 统计每组的个数
- `series.value_counts():` 统计分组次数

```
# 自行分组
qcut = pd.qcut(p_change, 10)
# 计算分到每个组数据个数
qcut.value_counts()
```

自定义区间分组:

- `pd.cut(data, bins)`

```
# 自己指定分组区间
bins = [-100, -7, -5, -3, 0, 3, 5, 7, 100]
p_counts = pd.cut(p_change, bins)
```

3.3 股票涨跌幅分组数据变成one-hot编码

- 什么是one-hot编码

把每个类别生成一个布尔列, 这些列中只有一列可以为这个样本取值为1.其又被称为独热编码。

把下图中左边的表格转化为使用右边形式进行表示:

Sample	Category	Numerical
1	Human	1
2	Human	1
3	Penguin	2
4	Octopus	3
5	Alien	4
6	Octopus	3
7	Alien	4

Sample	Human	Penguin	Octopus	Alien
1	1	0	0	0
2	1	0	0	0
3	0	1	0	0
4	0	0	1	0
5	0	0	0	1
6	0	0	1	0
7	0	0	0	1

- `pandas.get_dummies(data, prefix=None)`
 - data:array-like, Series, or DataFrame

- prefix:分组名字

```
# 得出one-hot编码矩阵
dummies = pd.get_dummies(p_counts, prefix="rise")
```

	rise_(-100, -7]	rise_(-7, -5]	rise_(-5, -3]	rise_(-3, 0]	rise_(0, 3]	rise_(3, 5]	rise_(5, 7]	rise_(7, 100]
2018-02-27	0	0	0	0	1	0	0	0
2018-02-26	0	0	0	0	0	1	0	0
2018-02-23	0	0	0	0	1	0	0	0
2018-02-22	0	0	0	0	1	0	0	0
2018-02-14	0	0	0	0	1	0	0	0

4 小结

- 数据离散化【知道】
 - 可以用来减少给定连续属性值的个数
 - 在连续属性的值域上，将值域划分为若干个离散的区间，最后用不同的符号或整数值代表落在每个子区间中的属性值。
- qcut、cut实现数据分组【知道】
 - qcut:大致分为相同的几组
 - cut:自定义分组区间
- get_dummies实现哑变量矩阵【知道】

5.9 高级处理-合并

学习目标

- 目标
 - 应用pd.concat实现数据的合并
 - 应用pd.merge实现数据的合并

如果你的数据由多张表组成，那么有时候需要将不同的内容合并在一起分析

1 pd.concat实现数据合并

- pd.concat([data1, data2], axis=1)
 - 按照行或列进行合并,axis=0为列索引，axis=1为行索引

比如我们将刚才处理好的one-hot编码与原数据合并

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	...	v_ma20	turnover	rise_[-100, -7]	rise_[-7, -5]	rise_[-5, -3]	rise_[-3, 0]	rise_[0, 3]	rise_[3, 5]
018-2-27	23.53	25.88	24.16	23.53	95578.03	0.63	2.68	22.942	22.142	22.875	...	55576.11	2.39	0	0	0	0	1	0
018-2-26	22.80	23.78	23.53	22.80	60985.11	0.69	3.02	22.406	21.955	22.942	...	56007.50	1.53	0	0	0	0	0	1
018-2-23	22.88	23.37	22.82	22.71	52914.01	0.54	2.42	21.938	21.929	23.022	...	56372.85	1.32	0	0	0	0	1	0
018-2-25	22.76	22.76	22.28	22.02	36105.01	0.36	1.64	21.446	21.909	23.137	...	60149.60	0.90	0	0	0	0	1	0

按照行索引进行
pd.concat([data, dummies], axis=1)

2 pd.merge

- pd.merge(left, right, how='inner', on=None)
 - 可以指定按照两组数据的共同键值对合并或者左右各自
 - left : DataFrame
 - right : 另一个DataFrame
 - on : 指定的共同键
 - how:按照什么方式连接

Merge method	SQL Join Name	Description
left	LEFT OUTER JOIN	Use keys from left frame only
right	RIGHT OUTER JOIN	Use keys from right frame only
outer	FULL OUTER JOIN	Use union of keys from both frames
inner	INNER JOIN	Use intersection of keys from both frames

2.1 pd.merge合并

```
left = pd.DataFrame({'key1': ['K0', 'K0', 'K1', 'K2'],
                    'key2': ['K0', 'K1', 'K0', 'K1'],
                    'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3']})

right = pd.DataFrame({'key1': ['K0', 'K1', 'K1', 'K2'],
                    'key2': ['K0', 'K0', 'K0', 'K0'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})

# 默认内连接
result = pd.merge(left, right, on=['key1', 'key2'])
```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A2	B2	K1	K0	C1	D1
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C2	D2
3	A3	B3	K2	K1	3	C3	D3	K2	K0							

- 左连接

```
result = pd.merge(left, right, how='left', on=['key1', 'key2'])
```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A1	B1	K0	K1	NaN	NaN
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C1	D1
3	A3	B3	K2	K1	3	C3	D3	K2	K0	3	A2	B2	K1	K0	C2	D2
										4	A3	B3	K2	K1	NaN	NaN

- 右连接

```
result = pd.merge(left, right, how='right', on=['key1', 'key2'])
```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A2	B2	K1	K0	C1	D1
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C2	D2
3	A3	B3	K2	K1	3	C3	D3	K2	K0	3	NaN	NaN	K2	K0	C3	D3

- 外链结

```
result = pd.merge(left, right, how='outer', on=['key1', 'key2'])
```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A1	B1	K0	K1	NaN	NaN
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C1	D1
3	A3	B3	K2	K1	3	C3	D3	K2	K0	3	A2	B2	K1	K0	C2	D2
										4	A3	B3	K2	K1	NaN	NaN
										5	NaN	NaN	K2	K0	C3	D3

3 总结

- `pd.concat([数据1, 数据2], axis=**)` 【知道】
- `pd.merge(left, right, how=, on=)` 【知道】
 - `how` -- 以何种方式连接
 - `on` -- 连接的键的依据是哪几个

5.10 高级处理-交叉表与透视表

学习目标

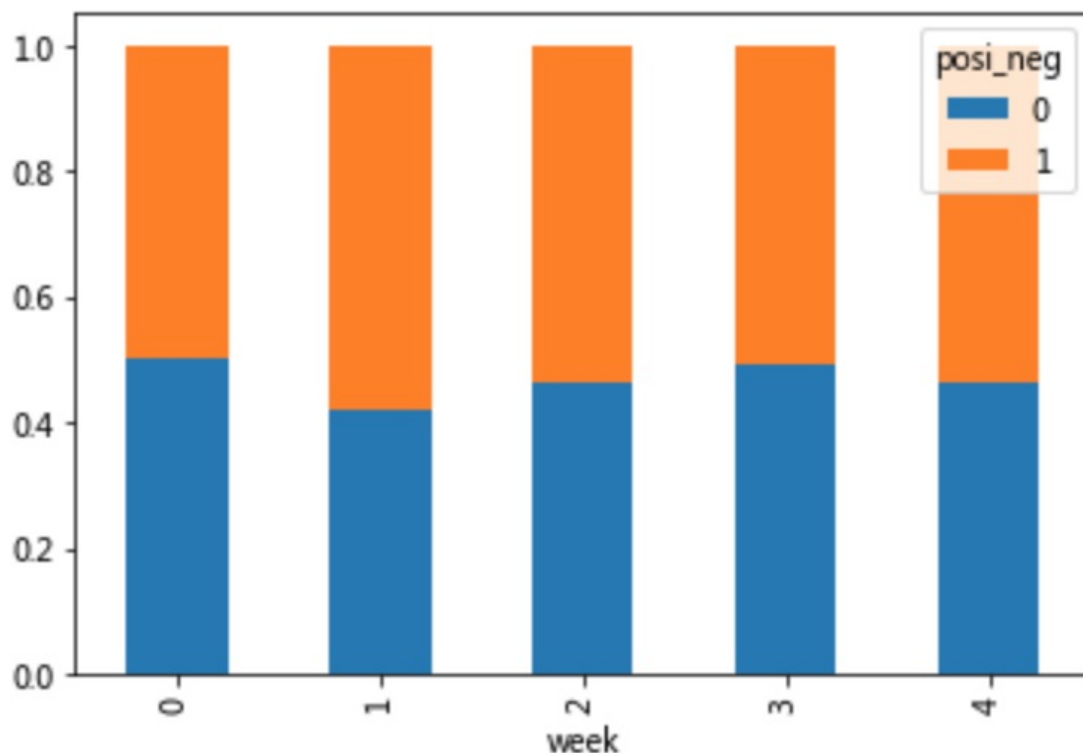
- 目标
 - 应用crosstab和pivot_table实现交叉表与透视表

1 交叉表与透视表什么作用

探究股票的涨跌与星期几有关？

以下图当中表示，**week**代表星期几，**1,0**代表这一天股票的涨跌幅是好还是坏，里面的数据代表比例
可以理解所有时间为星期一等等的数据当中涨跌幅好坏的比例

week	posi_neg	
	0	1
0	0.504000	0.496000
1	0.419847	0.580153
2	0.462121	0.537879
3	0.492188	0.507812
4	0.464567	0.535433



- 交叉表：交叉表用于计算一列数据对于另外一列数据的分组个数(用于统计分组频率的特殊透视表)
 - `pd.crosstab(value1, value2)`
- 透视表：透视表是将原有的**DataFrame**的列分别作为行索引和列索引，然后对指定的列应用聚集函数
 - `data.pivot_table()`
- - `DataFrame.pivot_table([], index=[])`

2 案例分析

2.1 数据准备

- 准备两列数据，星期数据以及涨跌幅是多是坏数据
- 进行交叉表计算

```
# 寻找星期几跟股票涨跌的关系
# 1、先把对应的日期找到星期几
date = pd.to_datetime(data.index).weekday
data['week'] = date

# 2、假如把p_change按照大小去分个类0为界限
data['posi_neg'] = np.where(data['p_change'] > 0, 1, 0)

# 通过交叉表找寻两列数据的关系
count = pd.crosstab(data['week'], data['posi_neg'])
```

但是我们看到count只是每个星期日子的好坏天数，并没有得到比例，该怎么去做？

- 对于每个星期一等的总天数求和，运用除法运算求出比例

```
# 算数运算，先求和
sum = count.sum(axis=1).astype(np.float32)

# 进行相除操作，得出比例
pro = count.div(sum, axis=0)
```

2.2 查看效果

使用`plot`画出这个比例，使用`stacked`的柱状图

```
pro.plot(kind='bar', stacked=True)
plt.show()
```

2.3 使用`pivot_table`(透视表)实现

使用透视表，刚才的过程更加简单

```
# 通过透视表，将整个过程变成更简单一些
data.pivot_table(['posi_neg'], index='week')
```

3 小结

- 交叉表与透视表的作用【知道】
 - 交叉表：计算一列数据对于另外一列数据的分组个数
 - 透视表：指定某一列对另一列的关系

5.11 高级处理-分组与聚合

学习目标

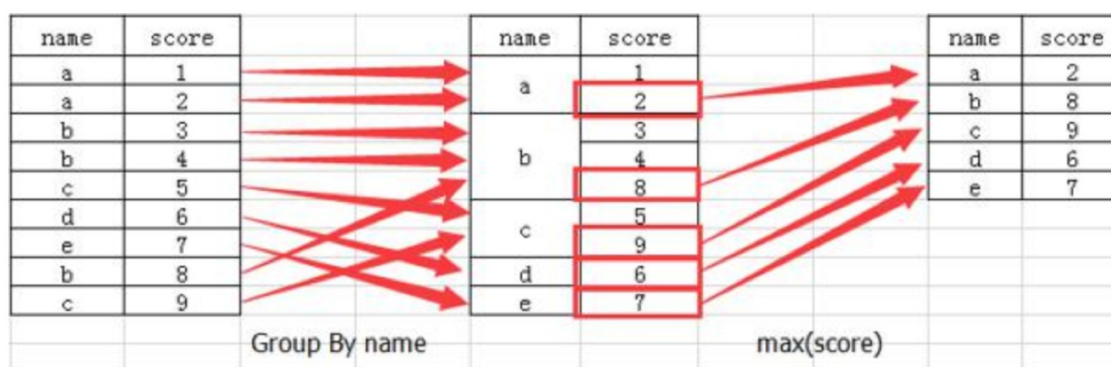
- 目标
 - 应用groupby和聚合函数实现数据的分组与聚合

分组与聚合通常是分析数据的一种方式，通常与一些统计函数一起使用，查看数据的分组情况

想一想其实刚才的交叉表与透视表也有分组的功能，所以算是分组的一种形式，只不过他们主要是计算次数或者计算比例！！看其中的效果：

	Brand	Store Number	Store Name	Ownership Type	Street Address	City	Postcode	Phone Number	Timezone	Longitude	Latitude
Country	State/Province										
AD	7	1	1	1	1	1	1	1	1	1	1
AE	AJ	2	2	2	2	2	2	0	0	2	2
	AZ	48	48	48	48	48	48	7	20	48	48
	DU	82	82	82	82	82	82	16	50	82	82
	FU	2	2	2	2	2	2	1	0	2	2
	RK	3	3	3	3	3	3	0	3	3	3
	SH	6	6	6	6	6	6	0	5	6	6
	UQ	1	1	1	1	1	1	0	0	1	1
AR	B	21	21	21	21	21	21	18	5	21	21
	C	73	73	73	73	73	73	71	24	73	73
	M	5	5	5	5	5	5	2	0	5	5
	S	3	3	3	3	3	3	3	0	3	3
	X	6	6	6	6	6	6	6	0	6	6
AT	3	1	1	1	1	1	1	1	1	1	1
	5	3	3	3	3	3	3	3	3	3	3

1 什么分组与聚合



2 分组API

- DataFrame.groupby(key, as_index=False)
 - key:分组的列数据，可以多个
- 案例:不同颜色的不同笔的价格数据

```
col =pd.DataFrame({'color': ['white','red','green','red','green'], 'object': ['pen','pencil','pencil','ashtray','pen'], 'price1':[5.56,4.28,1.38,0.56,2.75], 'price2':[4.75,4.12,1.68,0.75,3.15]})
```

```

color  object  price1  price2
0  white  pen      5.56   4.75
1  red    pencil   4.20   4.12
2  green  pencil   1.30   1.60
3  red    ashtray  0.56   0.75
4  green  pen      2.75   3.15

```

- 进行分组，对颜色分组，price进行聚合

```

# 分组，求平均值
col.groupby(['color'])['price1'].mean()
col['price1'].groupby(col['color']).mean()

color
green    2.025
red      2.380
white    5.560
Name: price1, dtype: float64

# 分组，数据的结构不变
col.groupby(['color'], as_index=False)['price1'].mean()

color  price1
0  green    2.025
1  red      2.380
2  white    5.560

```

3 星巴克零售店铺数据

现在我们有一组关于全球星巴克店铺的统计数据，如果我想知道美国的星巴克数量和中国的哪个多，或者我想知道中国每个省份星巴克的数量情况，那么应该怎么办？

数据来源: <https://www.kaggle.com/starbucks/store-locations/data>

	Brand	Store Number	Store Name	Ownership Type	Street Address	City	State/Province	Country	Postcode	Phone Number	Timezone	Longitude
0	Starbucks	47370-257954	Meritxell, 96	Licensed	Av. Meritxell, 96	Andorra la Vella	7	AD	AD500	376818720	GMT+1:00 Europe/Andorra	1.53
1	Starbucks	22331-212325	Ajman Drive Thru	Licensed	1 Street 69, Al Jarf	Ajman	AJ	AE	NaN	NaN	GMT+04:00 Asia/Dubai	55.47
2	Starbucks	47089-256771	Dana Mall	Licensed	Sheikh Khalifa Bin Zayed St.	Ajman	AJ	AE	NaN	NaN	GMT+04:00 Asia/Dubai	55.47
3	Starbucks	22126-218024	Twofour 54	Licensed	Al Salam Street	Abu Dhabi	AZ	AE	NaN	NaN	GMT+04:00 Asia/Dubai	54.38
4	Starbucks	17127-178586	Al Ain Tower	Licensed	Khalidiya Area, Abu Dhabi Island	Abu Dhabi	AZ	AE	NaN	NaN	GMT+04:00 Asia/Dubai	54.54

3.1 数据获取

从文件中读取星巴克店铺数据

```

# 导入星巴克店的数据
starbucks = pd.read_csv("./data/starbucks/directory.csv")

```

3.2 进行分组聚合

```

# 按照国家分组，求出每个国家的星巴克零售店数量
count = starbucks.groupby(['Country']).count()

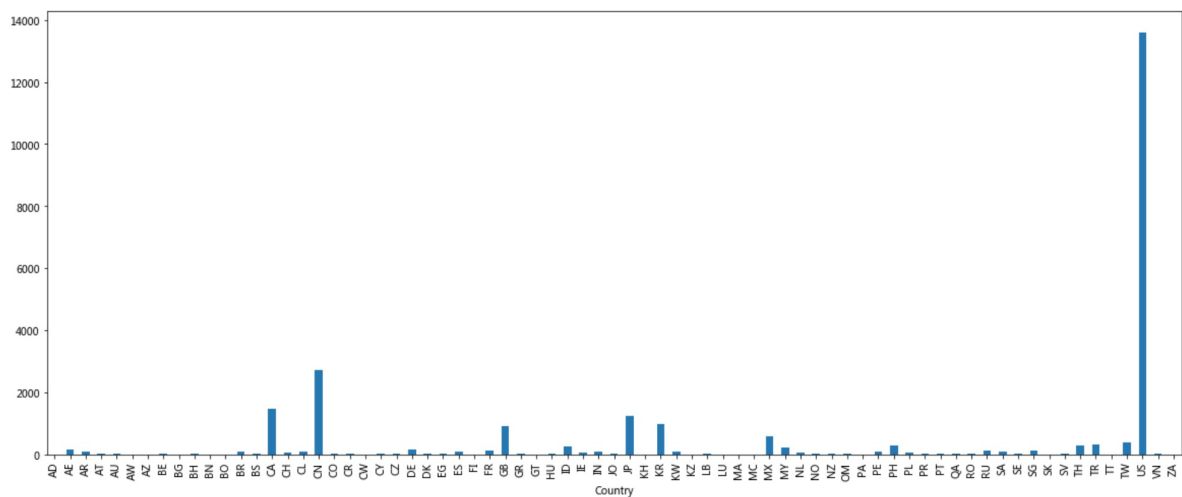
```

画图显示结果

```

count['Brand'].plot(kind='bar', figsize=(20, 8))
plt.show()

```



假设我们加入省市一起进行分组

```
# 设置多个索引, set_index()
starbucks.groupby(['Country', 'State/Province']).count()
```

	Brand	Store Number	Store Name	Ownership Type	Street Address	City	Postcode	Phone Number	Timezone	Longitude	Latitude
Country	State/Province										
AD	7	1	1	1	1	1	1	1	1	1	1
	AE	AJ	2	2	2	2	2	0	0	2	2
	AZ	48	48	48	48	48	48	7	20	48	48
	DU	82	82	82	82	82	82	16	50	82	82
	FU	2	2	2	2	2	2	1	0	2	2
	RK	3	3	3	3	3	3	0	3	3	3
	SH	6	6	6	6	6	6	0	5	6	6
	UQ	1	1	1	1	1	1	0	0	1	1
AR	B	21	21	21	21	21	21	18	5	21	21
	C	73	73	73	73	73	73	71	24	73	73
	M	5	5	5	5	5	5	2	0	5	5
	S	3	3	3	3	3	3	3	0	3	3
	X	6	6	6	6	6	6	6	0	6	6
AT	3	1	1	1	1	1	1	1	1	1	1
	5	3	3	3	3	3	3	3	3	3	3
	9	14	14	14	14	14	14	14	13	14	14

仔细观察这个结构，与我们前面讲的哪个结构类似？？

与前面的MultiIndex结构类似

4 小结

- groupby进行数据的分组【知道】
 - pandas中，抛开聚合谈分组，无意义

5.12 案例

学习目标

- 目标
 - 无

1 需求

现在我们有一组从2006年到2016年1000部最流行的电影数据

数据来源: <https://www.kaggle.com/damianpanek/sunday-eda/data>

- 问题1: 我们想知道这些电影数据中评分的平均分, 导演的人数等信息, 我们应该怎么获取?
- 问题2: 对于这一组电影数据, 如果我们想rating, runtime的分布情况, 应该如何呈现数据?
- 问题3: 对于这一组电影数据, 如果我们希望统计电影分类(genre)的情况, 应该如何处理数据?

2 实现

首先获取导入包, 获取数据

```
%matplotlib inline
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
```

```
#文件的路径
path = "../data/IMDB-Movie-Data.csv"
#读取文件
df = pd.read_csv(path)
```

2.1 问题一:

我们想知道这些电影数据中评分的平均分, 导演的人数等信息, 我们应该怎么获取?

- 得出评分的平均分

使用mean函数

```
df["Rating"].mean()
```

- 得出导演人数信息

求出唯一值, 然后进行形状获取

```
## 导演的人数
# df["Director"].unique().shape[0]
np.unique(df["Director"]).shape[0]

644
```

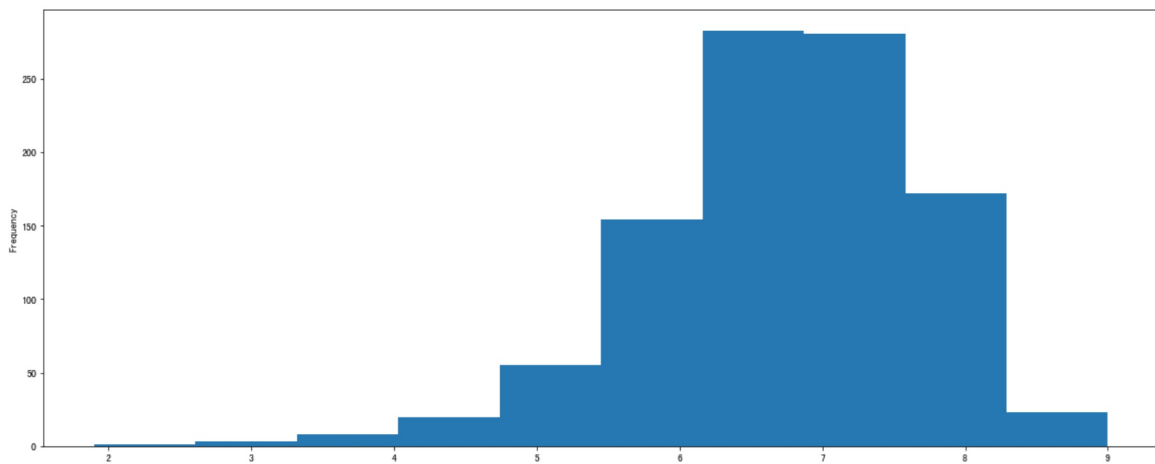
2.2 问题二:

对于这一组电影数据, 如果我们想Rating, Runtime (Minutes)的分布情况, 应该如何呈现数据?

- 直接呈现, 以直方图的形式

选择分数列数据, 进行plot

```
df["Rating"].plot(kind='hist', figsize=(20,8))
```



- Rating进行分布展示

进行绘制直方图

```
plt.figure(figsize=(20,8),dpi=80)
plt.hist(df["Rating"].values,bins=20)
plt.show()
```

修改刻度的间隔

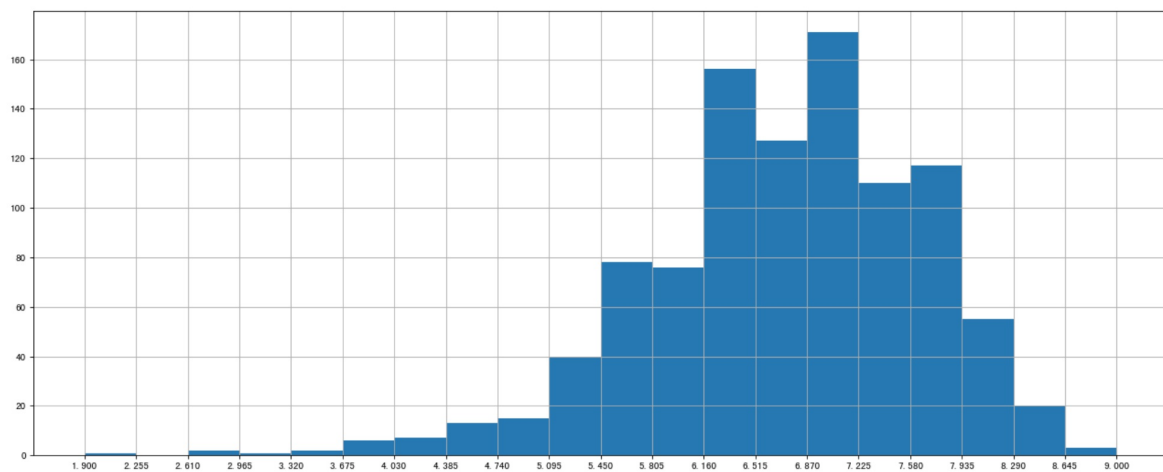
```
# 求出最大最小值
max_ = df["Rating"].max()
min_ = df["Rating"].min()

# 生成刻度列表
t1 = np.linspace(min_,max_,num=21)

# [ 1.9    2.255  2.61   2.965  3.32   3.675  4.03   4.385  4.74   5.095  5.45   5.805  6.16   6.515  6.87   7.225  7.58   7.935  8.29   8.645  9. ]

# 修改刻度
plt.xticks(t1)

# 添加网格
plt.grid()
```



- Runtime (Minutes)进行分布展示

进行绘制直方图

```
plt.figure(figsize=(20,8),dpi=80)
plt.hist(df["Runtime (Minutes)"].values,bins=20)
plt.show()
```

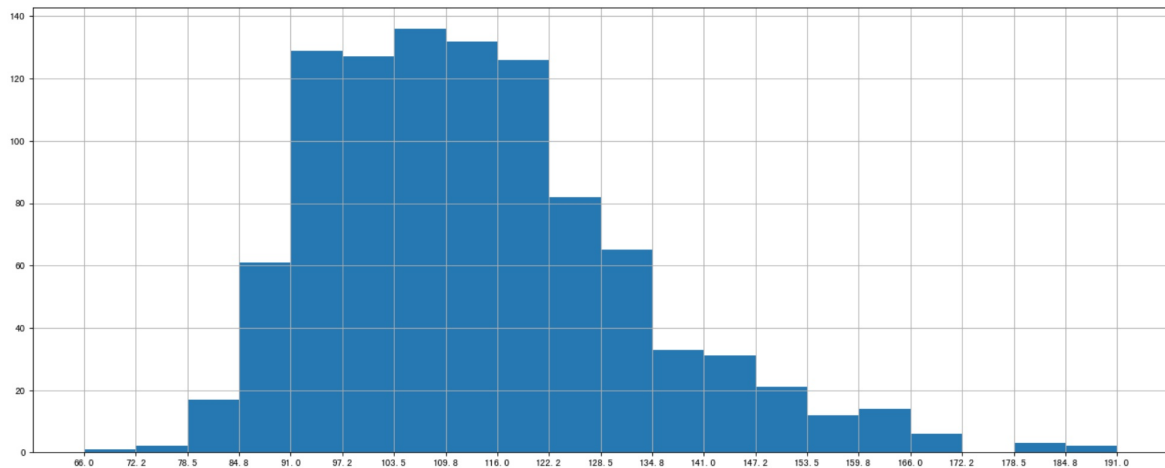
修改间隔

```
# 求出最大最小值
max_ = df["Runtime (Minutes)"].max()
min_ = df["Runtime (Minutes)"].min()

# 生成刻度列表
t1 = np.linspace(min_,max_,num=21)

# 修改刻度
plt.xticks(np.linspace(min_,max_,num=21))

# 添加网格
plt.grid()
```



2.3 问题三：

对于这一组电影数据，如果我们希望统计电影分类(genre)的情况，应该如何处理数据？

- 思路分析
 - 思路
 - 1、创建一个全为0的dataframe，列索引置为电影的分类，temp_df
 - 2、遍历每一部电影，temp_df中把分类出现的列的值置为1
 - 3、求和
 - 1、创建一个全为0的dataframe，列索引置为电影的分类，temp_df

```
# 进行字符串分割
temp_list = [i.split(",") for i in df["Genre"]]
# 获取电影的分类
genre_list = np.unique([i for j in temp_list for i in j])

# 增加新的列
temp_df = pd.DataFrame(np.zeros([df.shape[0],genre_list.shape[0]]),columns=genre_list)
```

- 2、遍历每一部电影，temp_df中把分类出现的列的值置为1

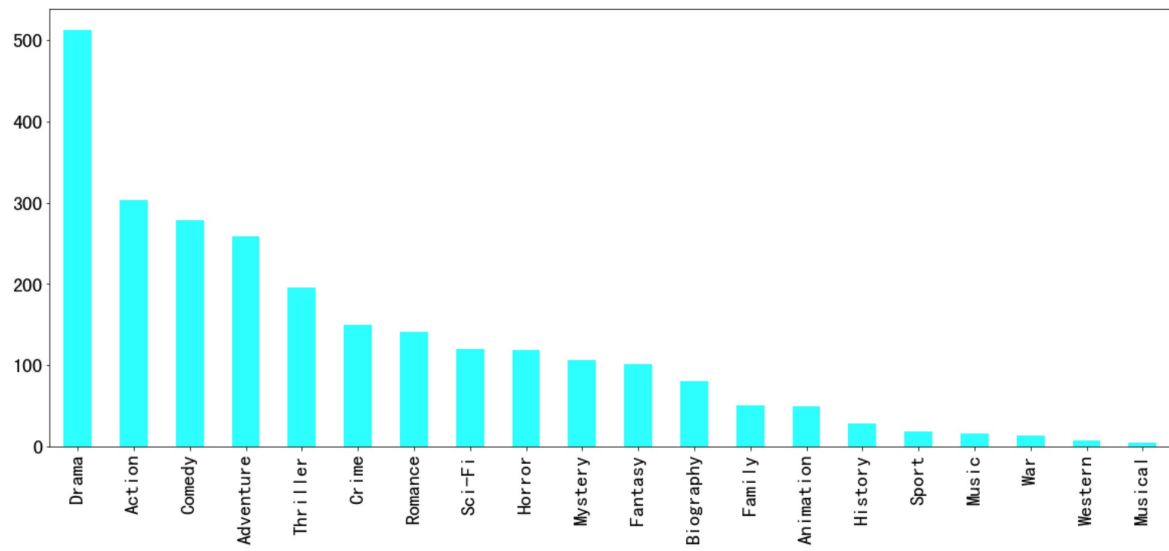
```
for i in range(1000):
    #temp_list[i] ['Action','Adventure','Animation']
    temp_df.ix[i,temp_list[i]]=1
print(temp_df.sum().sort_values())
```

- 3、求和,绘图

```
temp_df.sum().sort_values(ascending=False).plot(kind="bar",figsize=(20,8),fontsize=20,colormap="cool")
```

```
Musical      5.0
Western     7.0
War         13.0
Music       16.0
Sport       18.0
History     29.0
```

```
Animation    49.0
Family       51.0
Biography    81.0
Fantasy     101.0
Mystery     106.0
Horror      119.0
Sci-Fi      120.0
Romance     141.0
Crime       150.0
Thriller    195.0
Adventure   259.0
Comedy      279.0
Action     303.0
Drama      513.0
dtype: float64
```



拓展阅读

完整机器学习项目的流程（拓展阅读）

1 抽象成数学问题

明确问题是进行机器学习的第一步。机器学习的训练过程通常都是一件非常耗时的事情，胡乱尝试时间成本是非常高的。

这里的抽象成数学问题，指的明确我们可以获得什么样的数据，抽象出的问题，是一个分类还是回归或者是聚类的问题。

2 获取数据

数据决定了机器学习结果的上限，而算法只是尽可能逼近这个上限。

数据要有代表性，否则必然会过拟合。

而且对于分类问题，数据偏斜不能过于严重，不同类别的数据数量不要有数量级的差距。

而且还要对数据的量级有一个评估，多少个样本，多少个特征，可以估算出其对内存的消耗程度，判断训练过程中内存是否能够放得下。如果放不下就得考虑改进算法或者使用一些降维的技巧了。如果数据量实在太太大，那就要考虑分布式了。

3 特征预处理与特征选择

良好的数据要能够提取出良好的特征才能真正发挥作用。

特征预处理、数据清洗是很关键的步骤，往往能够使得算法的效果和性能得到显著提高。归一化、离散化、因子化、缺失值处理、去除共线性等，数据挖掘过程中很多时间就花在它们上面。这些工作简单可复制，收益稳定可预期，是机器学习的基础必备步骤。

筛选出显著特征、摒弃非显著特征，需要机器学习工程师反复理解业务。这对很多结果有决定性的影响。特征选择好了，非常简单的算法也能得出良好、稳定的结果。这需要运用特征有效性分析的相关技术，如相关系数、卡方检验、平均互信息、条件熵、后验概率、逻辑回归权重等方法。

4 训练模型与调优

直到这一步才用到我们上面说的算法进行训练。现在很多算法都能够封装成黑盒供人使用。但是真正考验水平的是调整这些算法的（超）参数，使得结果变得更加优良。这需要对算法的原理有深入的理解。理解越深入，就越能发现问题的症结，提出良好的调优方案。

5 模型诊断

如何确定模型调优的方向与思路呢？这就需要对模型进行诊断的技术。

过拟合、欠拟合 判断是模型诊断中至关重要的一步。常见的方法如交叉验证，绘制学习曲线等。过拟合的基本调优思路是增加数据量，降低模型复杂度。欠拟合的基本调优思路是提高特征数量和质量，增加模型复杂度。

误差分析 也是机器学习至关重要的步骤。通过观察误差样本全面分析产生误差的原因:是参数的问题还是算法选择的问题，是特征的问题还是数据本身的问题.....

诊断后的模型需要进行调优，调优后的新模型需要重新进行诊断，这是一个反复迭代不断逼近的过程，需要不断地尝试， 进而达到最优状态。

6 模型融合

一般来说，模型融合后都能使得效果有一定提升。而且效果很好。

工程上，主要提升算法准确度的方法是分别在模型的前端（特征清洗和预处理，不同的采样模式）与后端（模型融合）上下功夫。因为他们比较标准可复制，效果比较稳定。而直接调参的工作不会很多，毕竟大量数据训练起来太慢了，而且效果难以保证。

7 上线运行

这一部分内容主要跟工程实现的相关性比较大。工程上是结果导向，模型在线上运行的效果直接决定模型的成败。不单纯包括其准确程度、误差等情况，还包括其运行的速度(时间复杂度)、资源消耗程度（空间复杂度）、稳定性是否可接受。

这些工作流程主要是工程实践上总结出的一些经验。并不是每个项目都包含完整的一个流程。这里的部分只是一个指导性的说明，只有大家自己多实践，多积累项目经验，才会有自己更深刻的认识。

独立同分布IID(independent and identically distributed)

1.独立同分布(i.i.d.)

在概率统计理论中，如果变量序列或者其他随机变量有相同的概率分布，并且互相独立，那么这些随机变量是独立同分布。

在西瓜书中解释是：输入空间中的所有样本服从一个隐含未知的分布，训练数据所有样本都是独立地从这个分布上采样而得。

2.简单解释 — 独立、同分布、独立同分布

（1）独立：每次抽样之间没有关系，不会相互影响

举例：给一个骰子，每次抛骰子抛到几就是几，这是独立；如果我要抛骰子两次之和大于8，那么第一次和第二次抛就不独立，因为第二次抛的结果和第一次相关。

（2）同分布：每次抽样，样本服从同一个分布

举例：给一个骰子，每次抛骰子得到任意点数的概率都是六分之一，这个就是同分布

（3）独立同分布：i.i.d.，每次抽样之间独立而且同分布

3.机器学习领域的重要假设

IID独立同分布即假设训练数据和测试数据是满足相同分布的，它是通过训练数据获得的模型能够在测试集获得好的效果的一个基本保障。

4.目前发展

机器学习并不总要求独立同分布，在不少问题中要求样本数据采样自同一个分布是因为希望用训练数据集得到的模型可以合理的用于测试数据集，使用独立同分布假设能够解释得通。

目前一些机器学习内容已经不再囿于独立同分布假设下，一些问题会假设样本没有同分布。